

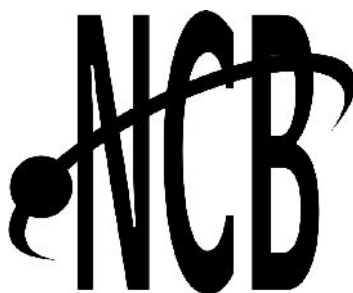


VIDAL PEREIRA

MICROCONTROLADORES PIC 16F E 18F TEORIA E PRÁTICA



MICROCONTROLADORES PIC 16F E 18F TEORIA E PRÁTICA



Instituto NCB

www.newtoncbraga.com.br
contato@newtoncbraga.com.br

Microcontroladores PIC 16F e 18F – Teoria e Prática

Autor: Vidal Pereira da Silva Jr.

São Paulo - Brasil - 2013

Palavras-chave: Eletrônica - Engenharia Eletrônica - Componentes – Microcontroladores

Copyright by
INSTITUTO NEWTON C BRAGA.
1ª edição

Todos os direitos reservados. Proibida a reprodução total ou parcial, por qualquer meio ou processo, especialmente por sistemas gráficos, microfílmicos, fotográficos, reprográficos, fonográficos, videográficos, atualmente existentes ou que venham a ser inventados. Vedada a memorização e/ou a recuperação total ou parcial em qualquer parte da obra em qualquer programa juscibernético atualmente em uso ou que venha a ser desenvolvido ou implantado no futuro. Essas proibições aplicam-se também às características gráficas da obra e à sua editoração. A violação dos direitos autorais é punível como crime (art. 184 e parágrafos, do Código Penal, cf. Lei nº 6.895, de 17/12/80) com pena de prisão e multa, conjuntamente com busca e apreensão e indenização diversas (artigos 122, 123, 124, 126 da Lei nº 5.988, de 14/12/73, Lei dos Direitos Autorais).

Diretor responsável: Newton C. Braga

Diagramação e Coordenação: Renato Paiotti

Dedicatória

À minha esposa Giane
e as minhas filhas Isabella e Nathália,
que são minhas fontes de energia
para viver cada vez mais.

Avisos importantes

1) Sobre as informações aqui apresentadas e garantias de qualquer tipo:

O autor acredita que todas as informações aqui apresentadas estão corretas e podem ser utilizadas para qualquer fim legal. Entretanto, não existe qualquer garantia, explícita ou implícita, de que o uso de tais informações conduzirá ao resultado desejado.

2) Sobre dúvidas relativas ao assunto

A aquisição deste livro não implica no direito do leitor de obter atendimento pessoal sobre dúvidas ou outros questionamentos referentes ao assunto, bem como suporte no uso das ferramentas apresentadas, as quais são gratuitas ou versões de demonstração.

3) Sobre os arquivos para download que acompanha este livro

Os arquivos que acompanham este material têm todos os exemplos já digitados para uso, e mais alguns arquivos auxiliares como databooks de componentes, manuais de uso, entre outros.

Todos os programas são grátis (apenas o compilador C é uma versão demo) e podem ser baixados da internet no site de seus respectivos fornecedores.

Para copiá-los para seu computador e instalar os programas, veja o anexo I no final deste livro.

Para Baixar o compilador acesse: <http://www.ccsinfo.com>

Objetivo deste material

Permitir ao estudante aprender sobre o funcionamento dos microcontroladores PIC em geral, das famílias 16F e 18F, de forma rápida e simples.

Para tal usaremos a linguagem C, o que reduz bastante o tempo de aprendizado e permitindo fácil transição para outros modelos de pic's.

Metodologia

Este material de estudo está dividido em 8 capítulos, que o estudante deve acompanhar na ordem proposta, e 2 anexos auxiliares:

- I) Introdução aos microcontroladores e linguagens de programação
- II) A linguagem C básica com exercícios simples para fixação dos conceitos
- III) Programa mínimo em C para compilação e simulação dos exemplos
- IV) O ambiente de desenvolvimento e simulação 'Mplab' em C com os exemplos básicos dos capítulos II e III
- V) A arquitetura dos Pícs e seus periféricos mais usuais
- VI) A linguagem C implementada pela CCS para os pic's, com detalhes

das diferenças entre as diversas famílias.

- VII) Arquivo de definições dos modelos usados nos exemplos: 18F458, 16F877 e 16F877A
- VIII) Exemplos práticos com esquemas e programas para estudo de alguns dos periféricos estudados (exemplos baseados no 16F877, 16F877A e 18F458)

Sobre a abordagem utilizada neste método de estudo

Desde 1988, ministrando os mais diversos tipos de treinamentos, posso assegurar que a abordagem tradicional usada pela maioria dos autores (geralmente indicada por editoras ou manuais de redação) transmite o assunto, mas de forma seqüencial, um tópico de cada vez.

No método que uso atualmente nos meus treinamentos, pude constatar que se transmitirmos os tópicos de forma paralela, isto é, se formos abordando uma parte de cada área, um pouco por vez, o estudante vai assimilando mais facilmente, pois consegue “enxergar”, passo a passo, o fim do túnel.

Em nosso caso, podemos dividir o treinamento em vários tópicos:

- A linguagem de programação 'C'
- O hardware do PIC
- O ambiente de desenvolvimento
- Esquemas elétricos dos exemplos

Para permitir que o estudante realmente assimile o conhecimento transmitido, vamos abordando todos os tópicos simultaneamente, permitindo ao aluno ir praticando desde o início do treinamento, sem ficar muito tempo apenas na teoria.

Desta forma, ao invés de transmitirmos o conhecimento, primeiro analisando somente o hardware, depois o software, depois as ferramentas e somente ao final os exemplos, vamos mesclando cada um dos tópicos aos poucos, e o aluno com certeza assimilará mais facilmente.

Espero que o leitor aprecie este trabalho, e tenha o melhor aproveitamento possível.

Vidal

Site do autor: www.vidal.com.br

Índice

I - Introdução aos microcontroladores e linguagens de programação.....	12
Os microcontroladores.....	12
A linguagem C	12
Método de estudo.....	12
II - A linguagem C básica.....	14
II.1 - Iniciação à linguagem C.....	14
II.2 - Algumas regras comuns para a programação em 'C'.....	14
II.3 - Modelo básico de um programa em C.....	14
II.4 - Comentários.....	15
II.5 - Diretivas de compilação.....	16
II.6 - Indicador de fim de instrução.....	16
II.7 - Definição de variáveis, constantes e identificadores.....	17
II.7.1 – Sinalizadores de números negativos e positivos.....	17
II.7.2 - Sequência de declaração de variáveis e constantes.....	18
II.7.3 - Atribuindo valores.....	18
II.7.4 – Atribuindo valores iniciais na declaração.....	18
II.7.5 – IMPORTANTE:	18
II.7.6 - Como escrever os nomes de variáveis, constantes e funções.....	19
II.7.7 – Typedef - Redefinindo tipos.....	19
II.8 - Funções e rotinas.....	20
II.8.1 - Funções especialmente desenvolvidas para os PIC's.....	20
II.9 - Expressões numéricas e de string (caracteres).....	21
II.10 - Operadores lógicos e aritméticos básicos da linguagem C.....	21
II.10.1 - Precedencia (ou prioridade) de operadores.....	23
II.10.2 - Conversão de tipos (type casting).....	23
II.11 - Matrizes.....	25
II.11.1 - Matrizes bidimensionais.....	26
II.12 - Controle do programa em C.....	26
II.12.1 - Blocos de declarações.....	26
II.12.2 - Bloco IF (executa se a condição for verdadeira).....	27
II.12.3 - Bloco FOR (executar por um certo número de vezes).....	30
II.12.4 - O condicional WHILE (enquanto).....	31
II.12.5 - O condicional DO . . . WHILE (faça ... enquanto).....	33
II.12.6 – O comando BREAK	34
II.12.7 – O comando CONTINUE.....	34
II.12.8 - O condicional SWITCH.....	35
II.12.9 - O comando RETURN.....	36
II.13 - Abreviações úteis para instruções aritméticas.....	37
II.13.1 - Incremento e Decremento.....	37
II.13.2 - Combinando abreviações.....	38
II.13.3 – Operações com resultado na mesma variável.....	38
II.14 - Variáveis locais, variáveis globais e parâmetros.....	38
II.14.1 - Variáveis Globais.....	38
II.14.2 - Variáveis Locais.....	38
II.14.3 - Variáveis como parâmetros.....	39
II.15 - A variável tipo VOID e os protótipos de funções.....	40
II.15.1 - Protótipos de funções.....	40
II.16 - Estruturas.....	41

II.17 - Unions.....	42
II.18 - A função MAIN ().....	43
II.19 - Exemplos de programas simples.....	43
III - Programa mínimo em C.....	46
IV - Usando o Mplab 7.62 em C.....	50
IV.1 - Conceitos básicos.....	50
IV.2 - O “Projeto” no MpLab.....	50
IV.3 - Criando o projeto com o Project Wizard.....	51
IV.4 - Simulando o programa.....	58
IV.5 – Verificando o registro PORTD durante a simulação.....	60
V - Os Microcontroladores PIC e seus periféricos mais usuais - Famílias 16F e 18F - Teoria de funcionamento.....	63
V.1 - Introdução.....	63
V.2 – Circuito mínimo.....	64
V.3 – Memória de Programa.....	64
V.4 – Memória de dados.....	65
V.5 – Memória EEPROM de dados.....	65
V.6 – Circuito de Reset e Clock.....	66
V.7 – Multiplicação 8 bits x 8 bits por hardware.....	66
V.8 – Interrupções	67
V.8.1 -Trabalhando com interrupções de alta e baixa prioridade.....	68
V.9 – Fusíveis de configuração.....	68
V.10 – O port A e suas funções especiais.....	69
V.10.1 – Algumas funções de acesso ao portA digital.....	69
V.11 – O port B e suas funções especiais.....	70
V.12 – O port C e suas funções especiais.....	70
V.13 – Os ports D e E com suas funções especiais.....	71
V.14 – Interrupções externas.....	71
V.15 – Timer 0.....	72
V.16 – Timer 1.....	73
V.16.1 – Funções para controle do timer 1.....	73
V.17 – Timer 2.....	73
V.18 – Timer 3 - Apenas na linha 18F.....	74
V.19 – O conversor A/D.....	74
V.20 – A comunicação serial Assíncrona.....	76
V.20.1 – Funções para controle da comunicação serial.....	77
V.21 – Tipos de osciladores.....	77
V.22 – O Watch Dog.....	78
V.22.1 - O Watch Dog da família 16F.....	78
V.22.2 - O Watch Dog do pic 18F458.....	79
V.23 – Brown-out Reset.....	80
V.24 – O modo Power-Down, ou ‘Sleep’.....	80
V.25 – Power-up Timer.....	81
V.26 – Oscilator Start-up Timer.....	81
V.27 – Módulo MSSP como SPI.....	82
V.28 – Módulo MSSP como I2C.....	82
V.29 – Módulo CCP como ‘Capture Mode’.....	83
V.30 – Módulo CCP em ‘Compare mode’.....	83
V.31 – Módulo CCP em ‘Pwm Mode’.....	83
V.32 – Low voltage Detector (LVD) -	84

V.33 – Comparador analógico (linha 16F87xA e 18F458).....	85
V.34 – Interfaces CAN, USB, ETHERNET,..... -	86
VI - A linguagem C e os PIC's.....	87
VI.1 - Diretivas de compilação.....	87
VI.1.1 - #asm #endasm.....	87
VI.1.2 - #case.....	87
VI.1.3 - #define 'nome' 'seqüência'.....	88
VI.1.4 - #include <arquivo>.....	88
VI.1.5 - #fuses 'opções'.....	88
VI.1.6 - #ifdef #endif.....	88
VI.1.7 - #INT #####, onde ##### indica o nome da rotina.....	89
VI.1.8 - #Priority – Prioridades por software.....	89
VI.1.9 - #ROM.....	90
VI.1.10 - #use delay (clock= 'valor do clock em Hz').....	90
VI.1.11 - #use Fast_IO(port).....	90
VI.1.12 - #use standard_IO(port) - Default.....	90
VI.1.13 - #use rs232 (BAUD = taxa, XMIT = pinoTx, RCV = pinoRx, BITS = n)	91
VI.1.14 - #byte nome = endereço do byte	91
#bit nome = endereço do byte . número do bit.....	91
VI.2 - Funções escritas para os PIC's.....	91
VI.2.1 - Funções matemáticas.....	92
VI.2.2 - Funções de manipulação de bit.....	92
VI.2.3 - Funções de tempo.....	93
VI.2.4 - Funções para interrupções.....	93
VI.2.5 - Funções para o canal A/D.....	94
VI.2.6 - Funções para EEPROM de dados interna.....	95
VI.2.7 - Funções do Timer 0 (São similares para os demais timers).....	95
VI.2.8 - Funções do canal de comunicação serial.....	96
VI.2.9 - Funções de uso geral.....	97
VII - Os arquivos de definições dos PICs.....	98
VIII – Exemplos práticos dos principais periféricos estudados no cap V.....	99
VIII.1 – Introdução.....	99
VIII.2 – Usando displays de cristal liquido como auxilio de	99
VIII.2.1 - O display LCD 16X2.....	99
VIII.3 – Exemplos do Capítulo II.....	101
VIII.4 – Exemplo dos Capítulo III.....	101
VIII.5 – Exemplos práticos para treinamento.....	101
VIII.5.1 – Usando saídas digitais.....	102
VIII.5.2 – Usando entradas e saídas digitais.....	103
VIII.5.3 – Usando o conversor de analógico para digital com resolução de 8 bits....	103
VIII.5.4 – Display LCD 16x2 com interface de 4 bits.....	105
VIII.5.5 - Usando o conversor de analógico para digital com resolução de 10 bits e visualização dos resultados no LCD.....	105
VIII.5.6 – Usando a EEPROM de dados.....	106
VIII.5.7 – Usando o timer 0 em 8 bits com clock externo.....	107
VIII.5.8 – Usando o timer 1 (16 bits) com clock interno.....	107
VIII.5.9 – Usando a interrupção externa INT0 no pino RB0.....	108
VIII.5.10 – Enviando um byte pelo canal serial.....	109
VIII.5.11 – Recebendo um byte pelo canal serial.....	110
VIII.5.12 – Comunicação serial SPI por hardware.....	111

VIII.5.13 – Comunicação serial I2C.....	111
VIII.5.14 – CCP em modo “PWM”	112
VIII.5.15 – Multiplexação de displays de 7 segmentos.....	113
VIII.5.16 – Teclado matricial 3x4.....	114
VIII.5.17 – Comunicação serial SPI por software.....	114
Anexos.....	116

I - Introdução aos microcontroladores e linguagens de programação

Os microcontroladores

Desde meu primeiro livro, em 1.988, utilizo o termo “Microcomputador-de-um-só-chip” para definir os microcontroladores.

A principal característica do microcontrolador esta em reunir em um só chip todos os periféricos necessários para o projeto e fabricação de dispositivos eletrônicos dos mais diversos tipos, desde simples sinalizadores e luzes pisca-pisca até equipamentos médicos sofisticados.

Hoje temos microcontroladores de apenas 6 pinos, minúsculos, ideais para incluir inteligência em dispositivos mecânicos em geral (dispositivos mecatrônicos) e até chips com as mais de 80 pinos, com as mais variadas capacidades, diversos tipos de interfaces (USB, Ethernet, CAN, ...); conversores analógico-digitais, entre outros.

Para a sua empreitada neste mundo maravilhoso de projetos com microcontroladores, alguns pré-requisitos são necessários:

1. Conhecimentos de nível médio de [eletrônica analógica](#) e [digital](#);
2. Facilidade de uso de computadores da linha PC baseados em Windows, para edição de textos e manipulação de arquivos;
3. Prática de [montagens eletrônicas](#) para a parte experimental;
4. Noções de programação em qualquer linguagem ou conhecimentos de lógica de programação.

A linguagem C

Neste treinamento utilizaremos a linguagem C para programar os PIC's, e não o assembler.

A principal vantagem do uso da linguagem C esta no fato de que a crescente complexidade dos microcontroladores vai tornando a programação em assembler cada vez mais difícil, dificultando para o projetista a mudança de modelos, como, por exemplo, a migração, na linha microchip, da linha 16F para a linha 18F.

Com o compilador C, as constantes mudanças de arquitetura interna do chip, das instruções, e dos algoritmos de desenvolvimento de software, passa a ser muito mais simples, pois a recompilação de cada rotina ou função especial (por exemplo, a programação dos registros internos para uso do conversor analógico-digital), que com certeza é diferente nas linhas 16F e 18F, passa a ser transparente para o desenvolvedor.

Desta forma, ao invés de consumir tempo reescrevendo rotinas ou todo um programa, o projetista apenas vai revisar as funções do seu programa para ajustar-se aos periféricos do novo modelo, dedicando seu tempo a escrever as funções baseadas em lógica, e não perdendo-se em detalhes de bits e bancos de memória.

Método de estudo

Neste treinamento não vamos estudar na forma tradicional.

Vamos primeiro ver a linguagem C básica, sem se preocupar com sua aplicação nos pics, apenas analisando os detalhes para compilar os programas mínimos apresentados.

Em seguida, veremos o ambiente de desenvolvimento Mplab, da Microchip, e o compilador C da CCS (versão demo), numa apresentação gráfica de como se usa a ferramenta.

Depois estudaremos os principais periféricos das famílias 16F e 18F, baseados nos modelos 16F877, 17F877A e 18F458.

Após este estudo voltaremos ao estudo da linguagem C, agora já detalhando os comandos desenvolvidos para os PIC's.

Nesta parte do treinamento dedicaremos nosso maior tempo, pois já poderemos estudar e utilizar simultaneamente o pic, o compilador C, o ambiente Mplab em compilação e sobretudo em simulação.

Teremos então vários exemplos completos, com esquema e software detalhado.

Para enriquecer ainda mais este trabalho, temos arquivos para download com todos os programas já digitados e prontos para serem executados, com a versão demo do compilador PCWH, com o Mplab e com o software de gravação IC-PROG, e os esquemas individuais de cada experiência e de um gravador para os pic's.

II - A linguagem C básica

II.1 - Iniciação à linguagem C

A principal vantagem de se usar linguagens de alto nível (no nosso caso a linguagem C) esta na menor interação do projetista com o hardware, no que diz respeito ao controle do mesmo (ajuste de bancos de registradores, seqüências de inicialização, etc...).

Como exemplo vamos ver a seqüência de escrita na EEPROM de dados do 18F458 em assembler:

```
MOVLW      EE_ADDRESS
MOVWF      EEADR
MOVLW      EE_DATA
MOVWF      EEDATA
BCF         EECON1, EEPGD
BCF         EECON1, CFGS
BSF         EECON1, WREN
BCF         INTCON, GIE
MOVLW      55h
MOVWF      EECON2
MOVLW      0AAh
MOVWF      EECON2
BSF         EECON1, WR
BSF         INTCON, GIE
```

e em C:

```
write_eeprom ( EE_ADDRESS , EE_DATA );
```

II.2 - Algumas regras comuns para a programação em 'C'

- Use letras minúsculas em todo o programa (recomendado)
- Todas as funções e variáveis devem ser declaradas
- Palavras reservadas não podem ser usadas como identificadores de variáveis e funções
- Sempre escreva comentários em seus programas

II.3 - Modelo básico de um programa em C

Quatro elementos estão presentes em um programa C:

```
Comentários
Diretivas de compilação
Definições de dados
Blocos com instruções e funções
```

Modelo básico (desconsiderando ser um programa para pic's):

```

#include <...>                (Diretivas de compilação)

// comentário ocupando uma linha

/* comentários entre '/' e '*' e '*' e '/'      podem ocupar mais de
uma linha */

int8 i, j;                    (Variáveis de 8 bits)
float Tempo;                  (Variável de ponto flutuante)

void main( )
{
    instruções do programa principal
}

void delay( )
{
    instruções da função (rotina) delay
}

```

II.4 - Comentários

Comentários são informações que você, durante a escrita do programa fonte (*), vai inserindo e que permitem a você (programador) e a outros entenderem o significado do que esta sendo feito.

É boa prática comentar o máximo possível de linhas, e até mesmo incluir grandes blocos de comentários, explicando o porque do que esta sendo feito, pois após um certo tempo, nem mesmo o criador do programa lembrará de tudo o que estava pensando no momento da escrita.

O compilador ignora tudo que estiver definido como comentário.

(*) O programa fonte em C deve ter terminação “.C”, por exemplo: teste.c

Existem dois tipos de comentários:

- Comentários que ocupam apenas 1 linha

Este tipo de comentário é iniciado com duas barras conjuntas: //

Neste caso, tudo que estiver após as duas barras será ignorado pelo compilador, até o final da linha.

Exemplo:

```
x = x + 2;           // soma 2 à variável x
```

• Comentários com múltiplas linhas

Este estilo de comentário é iniciado com a seqüência `/*` e finalizado pela seqüência `*/`.

Neste caso, tudo que estiver ENTRE estas duas seqüências será ignorado pelo compilador, não importando quantas linhas exista entre os dois marcadores.

É ideal para excluir temporariamente trechos de código.

Exemplo:

```
x = x + 2;
```

```
/* tempo++;
```

```
a = SQRT(25);          */
```

```
x = 0;
```

No exemplo acima, as linhas `tempo++;` e `a=SQRT(25);` serão ignoradas no momento da compilação.

II.5 - Diretivas de compilação

Geralmente são instruções para o compilador, e em alguns casos, devido as necessidades, implicarão na inclusão de alguns trechos de código já pronto.

As diretivas informam, por exemplo, o processador para o qual o código deverá ser gerado, o valor do clock que será usado pela cpu, etc..

As diretivas sempre começam com `#`.

Um bom exemplo é a diretiva que inclui no processo de compilação as definições do chip.

Por exemplo: `#include <18F458.H>`

A terminação `.H` indica um Hheader File da linguagem C, ou seja, um cabeçalho.

II.6 - Indicador de fim de instrução

O compilador C não é um compilador de linha, como o assembler.

O compilador C procura o sinal de que a instrução ou o bloco de instruções já acabou.

Este sinal é o `;` (ponto e virgula) para uma instrução isolada ou o `}` para o bloco (mais tarde falaremos sobre blocos de instruções).

No exemplo abaixo, as duas maneiras são corretas, pois o `;` é que sinaliza o fim da instrução.

```
x = x + 25;
```

```
x =  
x + 25  
;
```

É claro que você vai usar o exemplo da esquerda, pois seu código deve estar sempre bem legível.

II.7 - Definição de variáveis, constantes e identificadores

Todas as variáveis e constantes usadas no programa devem ser devidamente definidas, com um nome e um tipo. O mesmo vale para identificadores de funções e rotinas.

Os dados básicos podem ser de 8, 16 e 32 bits de comprimento, e devido as características peculiares dos microcontroladores, variáveis de 1 bit também podem ser definidas.

Embora as variáveis possam ser designadas pelos nomes tradicionais da linguagem C (char, long int, ...) vamos usar os nomes abaixo, permitidos pelo compilador da CCS, que são equivalentes e muito mais simples de entender.

<u>Variáveis</u>	<u>Identificador</u>	<u>Faixa</u>
• variável inteira de 8 bits:	INT8	(de 0 à 255)
• variável inteira de 16 bits:	INT16	(de 0 à 65.535)
• variável inteira de 32 bits:	INT32	(de 0 à 4.294.967.296)
• variável de ponto flutuante:	FLOAT	(- 1.5 E 45 a +3.4 E +38)
• variável de 1 bit:	INT1	(pode assumir 0 ou 1)

II.7.1 – Sinalizadores de números negativos e positivos

As variáveis do tipo INT8, INT16 e INT32 podem ou não ter sinal negativo.

Para garantir que a variável seja sempre positiva, usamos ***unsigned*** antes da definição da mesma. Caso seja necessário tratar números negativos, usaremos ***signed***.

Ex.: ***unsigned int16*** tempo; // tempo irá de 0 à 65.535
signed int16 espaço; // espaço irá de -32.768 à 32.767

Detalhes importantes:

- Observe que o indicativo ***signed*** diminui o alcance da variável pela metade.
- O compilador da CCS usa como padrão, quando não indicado, o tipo ***UNSIGNED***.
- Com variáveis ***FLOAT*** não precisamos usar indicativos de sinal, pois o alcance já vai de um valor negativo ao positivo.
- Se desejar garantir total compatibilidade com outros compiladores, sempre use os qualificadores nas definições das variáveis.

II.7.2 - Seqüência de declaração de variáveis e constantes

Variáveis:

Para se declarar uma variável temos a seguinte ordem das definições:

Sinal Tipo Nome

unsigned int8 tempo; *// a variável 'tempo' vai de 0 à 255*
(opcional)

Um grupo de variáveis de mesmo tipo pode ser declarada na mesma linha.

int8 i, j, k; *// declara que i, j e k são do tipo int8.*

Constantes:

CONST define um 'label' ou 'nome' para valores que não serão alterados pelo programa:

Exemplo: **float const PI = 3.14;**

II.7.3 - Atribuindo valores

A atribuição de valores é feita pelo sinal de igualdade, '='.

Exemplo: **tempo = 123;** *// aqui a variável tempo passa a ter o valor 123 decimal.*

II.7.4 – Atribuindo valores iniciais na declaração

Podemos também indicar o valor inicial das variáveis na declaração das mesmas.

Desta forma, o compilador já colocará no início do programa as instruções necessárias para alocar os valores.

Sua principal utilidade está em garantir que não vamos esquecer de inicializar os valores.

Exemplo:

uint8 velocidade_inicial = 100; *// declaramos e já demos um valor*

II.7.5 – IMPORTANTE:

Devemos sempre ajustar os valores iniciais de cada variável do programa antes de usá-las, pois o valor no reset é INDETERMINADO.

II.7.6 - Como escrever os nomes de variáveis, constantes e funções

Todo 'label' (nome ou identificador), seja de variável, constante ou função:

- Deve começar por letra ou por sublinhado ' _ '.
- Ter no máximo 32 caracteres.
- Não usar caracteres especiais ou de controle (! \ ? %).
- Nomes de funções e rotinas internas também não podem ser utilizados.

- Exemplos de definições:

Corretas	Incorretas
Teste	3local >>> começa com número !
teste	parte!dois
TESTE	^
_12A	caractere inválido (!)
x_2_5	

IMPORTANTE: Este compilador C, inicialmente, NÃO diferencia letras minúsculas de maiúsculas.

Nos exemplos acima, Teste, teste e TESTE são a mesma variável para o compilador.

DICA: Use nomes de variáveis óbvios e em português, assim não terá problemas de incompatibilidade.

Por exemplo:

```
Int8   velocidade = 10;           // variável já inicializada
Int32  tempo_de_resposta;        // esta variável deveria ser inicializada no programa
Float  distancia = 1.2;
```

II.7.7 – Typedef - Redefinindo tipos

A declaração TYPEDEF é usada para definirmos novos tipos baseados em outros já existentes

Vamos ver através de um exemplo simples:

```
typedef signed int8 sint8;
```

Neste exemplo, vamos fazer com que o compilador "enxergue" o novo tipo, **SINT8**, como se tivéssemos digitado **SIGNED INT8**

Podemos redefinir quantos tipos quisermos para o mesmo tipo original, e quando criamos um novo tipo com TYPEDEF o tipo original continua valendo.

Neste exemplo criamos dois novos tipos que tem o mesmo padrão:

```
typedef signed int8 sint8;
typedef signed int8 maismenos8;
```

Após estas declarações, para usar uma variável de 8 bits com sinal, podemos usar em nosso programa **signed int8** ou **sint8** ou **maismenos8**.

Se colocarmos este trecho de código no início de nossos programas, estaremos redefinindo as variáveis de 8, 16 e 32 bits para facilitar a escrita dos programas:

```
typedef    unsigned int8    uint8;           // U de unsigned
typedef    unsigned int16   uint16;
typedef    unsigned int32   uint32;
typedef    signed int8      sint8;          // S de signed
typedef    signed int16     sint16;
typedef    signed int32     sint32;
```

II.8 - Funções e rotinas

Chamamos função um trecho de programa que realiza determinada operação (função que lê teclado, função que escreve no display, ...) bem como as funções pré-definidas (SQRT, ABS, ACOS, ...).

Muitas vezes as palavras “função” e “rotinas” referem-se a mesma operação.

No exemplo abaixo, temos a função MAIN (falaremos dela depois) que é a rotina executada no reset do programa.

A função MAIN chama uma ROTINA para ler teclas, e a rotina de ler teclas chama uma FUNÇÃO para calcular o valor absoluto da tecla.

```
LeTeclas()
{
    ---
    ---
    X = ABS(tecla_lida);
    ---
    ---
}

main ()           // no reset o processador começa a rodar aqui
{
    ---
    ---
    LeTeclas();
    ---
    ---
}
```

II.8.1 - Funções especialmente desenvolvidas para os PIC's

No capítulo VI, referente ao compilador da CCS, veremos várias funções que existem para os PIC's e como funcionam.

II.9 - Expressões numéricas e de string (caracteres)

Vamos ver neste item os tipos de expressões permitidas para valores numéricos e para a manipulação de caracteres.

Números Decimais: _____ Não podem começar por ' 0 ' (zero)

Exemplos: 123; 2; 14534; 3.14; ...

Números Octais: _____ Devem começar por ' 0 ' (zero)
(Pouco utilizados)

Exemplos: 045; 07;...

Números Binários: _____ Devem iniciar por ' 0b '

Exemplo: 0b10101010

Números Hexadecimais: _____ Devem iniciar por ' 0x '

Exemplo: 0x32; 0xA9; ...

String de 1 caractere: _____ Entre Apóstrofos ' '

Exemplo: 'z'; 'A';

String de vários caracteres: _____ Entre aspas " "

Exemplo: "teste de escrita"

II.10 - Operadores lógicos e aritméticos básicos da linguagem C

Temos aqui os principais operadores lógicos e aritméticos da linguagem C:

Operadores aritméticos

- + Adição
- ++ Incremento da variável indicada (D++ é equivalente a D = D + 1)
- - Subtração
- -- Decremento da variável indicada (X-- é equivalente a X = X - 1)
- * Multiplicação
- / Divisão (parte inteira da divisão para variáveis inteiras)
Se a variável de resultado não for inteira o operador " / " coloca na mesma apenas a parte inteira do resultado.

Exemplo: x=5/2; // x= 2 e o resto 1 é descartado
- % Resto da divisão (quando realizamos divisão de inteiros)
Se a variável de resultado for inteira o operador " % " coloca na mesma apenas o RESTO do resultado.

Operações de deslocamento binário

- << 'n'

Desloca o valor indicado 'n' bits para a esquerda, descartando os 'n' bits MAIS significativos e acrescentando 'n zeros' à direita.

Exemplo: **X = 0b00001010** // X = 10 decimal
X = X << 1; // X = **00010100** = 20 decimal
X = X << 2; // X = **01010000** = 80 decimal

- >> 'n'

Desloca o valor indicado 'n' bits para a direita, descartando os 'n' bits MENOS significativos e acrescentando 'n zeros' à esquerda.

Exemplo: **X = 0b00001010** // X = 10 decimal
X = X >> 2; // X = **00000010** = 2 decimal

(10 dividido por 4 resulta 2 com resto 1, que foi descartado)

II.10.1 - Precedencia (ou prioridade) de operadores

Precedencia refere-se a “ordem sequencial” com que uma operação é executada pelo compilador.

Se numa instrução tivermos

X = a + b * c;

Qual seria a primeira operação realizada ? Seria a soma de a e b ?

Prioridade	Operação
1	() ++ --
2	& * + - ~ ! ++ --
3	* / %
4	+ -
5	<< >>
6	< > <= >=
7	== !=
8	&
9	^
10	
11	&&
12	
13	= *= /= %= += -= <<= >>=
	\$= ^= =

Em nosso exemplo, a sequencia real será:

Primeiro: **b * c** gerando um resultado parcial que chamaremos de P

Depois: **Z = a + P**

II.10.2 - Conversão de tipos (type casting)

Muitas vezes ocorre de precisarmos copiar uma variável em outra, ou até mesmo realizar operações aritméticas entre elas.

Nestes momentos temos de ter cuidado com as conversões entre os tipos de variáveis, pois o compilador não vai avisar caso algum valor venha a ser “truncado” na operação.

Apenas para simplificar usaremos os nomes `var8` e `var16` para variáveis de 8 e 16 bits, e para facilitar usaremos valores em hexadecimal.

Exemplo 1: copiar variável de 8 bits para uma de 16 bits

```
var8 = 0x12;  
var16 = var8;           // resultado = 0x0012
```

O compilador ajustou a variável de 16 bits com ‘zeros’ a esquerda.

Exemplo 2: copiar variável de 16 bits para uma de 8 bits

```
Var16 = 0x1234;  
Var8 = var16;           // resultado = 0x34
```

Aqui o compilador “matou” a parte alta (0x12) e usou apenas a parte baixa na operação.

DICA: Sempre que for usar tamanhos diferentes certifique-se de transformar todos para o mesmo tipo, pois os erros de cálculo decorrentes das conversões automáticas são imprevisíveis e NÃO são sinalizados pelo compilador

Ao invés de deixar o compilador fazer a conversão, podemos especificar o tipo de conversão que desejamos com o seguinte formato:

(tipo) valor

Este formato de escrita é chamado de TYPE CASTING.

Quando escrevemos desta forma temos uma conversão temporária em memória para uso apenas na operação desejada.

Se multiplicarmos duas variáveis de 8 bits, o resultado será outra variável de 8 bits, mesmo que o resultado seja direcionado para uma variável de 16 bits, porque a aritmética é realizada ANTES do resultado ser escrito na variável final.

```
int8    a = 0;  
int8    b = 0;  
int16   c = 0;
```

```
a = 100;  
b = 100;
```

```
c = a * b;
```

Neste caso o resultado sera 16. Vejamos:

```
100 * 100 = 10000
```

Em hexadecimal temos um valor de 16 bits igual a 0x2710. Como o resultado será de 8 bits, apenas o byte final será aproveitado, resultando 0x10 que em decimal da 16

Como resolver com o type casting ?

```
int8    a = 0;
int8    b = 0;
int16   c = 0;
```

```
a = 100;
b = 100;
```

```
c = (int16) a * b;           // primeiro passa a variavel 'a' para 16 bits, logo a
                             // multiplicação será em variáveis de 16 bits
```

Estude o exemplo que está na pasta

c:\apostilaPICrev4\cap2\typecasting

II.11 - Matrizes

Define-se como matriz um grupo de dados que podem ser agrupados num mesmo nome, diferenciando-se apenas pela posição no grupo.

Como exemplo temos uma rua com várias casas. A rua seria nossa matriz, por exemplo, Rua Paraná. Cada casa tem um número que indica sua posição na rua (na matriz).

Exemplo: casa 32 da Rua Paraná

Poderíamos escrever Rua Parana [32]

Em C, a definição de uma variável ou de uma constante como matriz é feita apenas pela inclusão de seu tamanho entre colchetes [].

Exemplo:

Matriz para os 20 valores de temperatura lidos, variáveis de 8 bits:

```
int8 temperatura [20];           // reserva espaço de memória para
                                 // 20 bytes que indicarão a temperatura
```

Para acessar os elementos da matriz basta escrever o nome da variável com o índice do valor desejado.

Exemplo:
 valor = temperatura [12]; // o 13º elemento da matriz temperatura
 // é copiado para a variável valor.

Podemos ainda utilizar variáveis para acessar os elementos da matriz.

Exemplo:
 int8 qual; // usaremos esta variável como índice
 int8 dados [20]; // declaração da matriz em memória, com
 // 20 elementos

Em nosso programa podemos fazer o seguinte:

```
qual = 10;                       // ajustamos o índice no 11º elemento
dados [qual] = 40;               // e escrevemos na matriz na posição
                                 // indicada na variável
```

IMPORTANTE:

1. O índice de acesso a matriz vai de 0 até tamanho-1.
No nosso exemplo, irá de 0 à 19.
2. CUIDADO! Se você usar como índice da matriz um numero maior que o seu limite, o compilador usará uma posição de memória RAM de outra variável.

Exemplo:

```
qual = 30;  
dados [qual] = 10;           // onde este valor será armazenado, se nossa  
                             // matriz tem apenas 20 elementos?
```

II.11.1 - Matrizes bidimensionais

Podemos ainda usar matrizes bidimensionais, ou seja , como em um tabuleiro de xadrez.

Exemplo: `int16 tabela [10][10];`

Neste exemplo criamos uma matriz com 100 elementos (10 X 10) de 16 bits, ou seja, usamos 200 posições de memória de dados.

II.12 - Controle do programa em C

Um dos pontos mais importantes do aprendizado da linguagem C esta na sua sintaxe, isto é, como o programa deve ser escrito.

Até agora vimos apenas teorias sobre variáveis, dados, e funções.

Veremos a partir de agora como tudo isto é integrado de forma a produzir um resultado útil.

Para tal vamos estudar as estruturas de montagem de programas e ver como se controla o fluxo de “ações” que o programa deve tomar.

Para facilitar o aprendizado não vamos nos preocupar agora com o PIC e nem com as funções criadas para o mesmo, mas apenas com a forma como escrevemos o programa em C.

II.12.1 - Blocos de declarações

Sempre que mais de uma instrução tiver de ser executada para uma certa rotina, as mesmas deverão estar contidas dentro de um *BLOCO* de declarações.

Um bloco de declarações tem início com a abertura de uma chave ‘ { ‘ e é finalizado pelo fechamento da chave ‘ } ‘.

Como um bloco não termina no fim da linha, mas sim ao fechar a chave, podemos escrever o mesmo de forma mais clara, colocando seus elementos em várias linhas e tabulando as colunas conforme os blocos vão se integrando, permitindo melhor colocação de comentários e visualização do programa.

Exemplo 1: { x = 1; tempo = x * 2; }

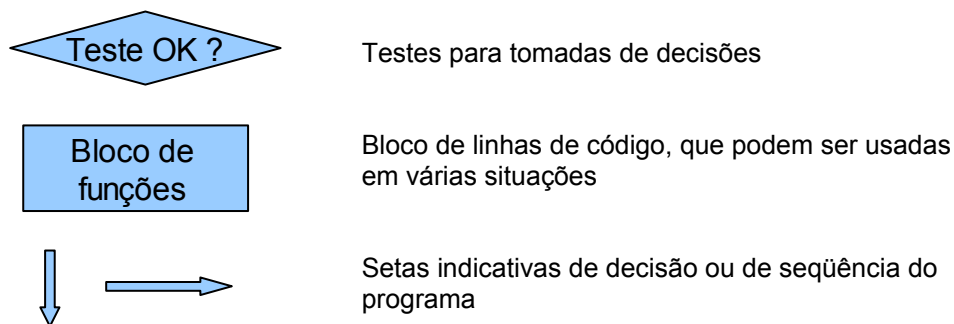
Exemplo 2: {
 x = 1; // posso colocar um
 tempo = x * 2; // comentário para cada
 } // linha

Os dois exemplos acima realizam a mesma tarefa, mas o exemplo 2 é mais fácil de ser lido e posteriormente entendido.

II.12.1.1 – Utilização de fluxogramas para representar a seqüência das instruções

Vamos sempre que possível utilizar fluxogramas simples para auxiliar nosso entendimento do fluxo do programa.

Em nossos fluxogramas temos os desenhos básicos:



II.12.2 - Bloco IF (executa se a condição for verdadeira)

Podemos entender o bloco IF como um teste simples, onde o programa fará um teste e em função da resposta ao mesmo (falso ou verdadeiro), executará uma ou outra ação, prosseguindo a partir daí.

Temos duas opções básicas, sendo que a condição de teste deverá estar entre parênteses:

- | | |
|---|--|
| - | SE (teste = ok!) “ <i>executa esta(s) declaração(ões)</i> ” |
| - | SE (teste = ok!) “ <i>executa esta(s) declaração(ões)</i> ”
SENÃO “ <i>executa esta(s) outras declaração(ões)</i> ” |

Temos vários formatos possíveis para o IF

- IF simples, com apenas uma declaração caso o teste seja verdadeiro

if (A == 0) A = 10; // SE a variável A estiver zerada, atribui 10 à mesma.

- IF com mais de uma declaração caso o teste seja verdadeiro.

Neste caso o grupo de declarações deverá estar num BLOCO, entre chaves ‘ { } ’.

```

if ( tempo > 10 )
{
    tempo = 0;
    contador = contador + 1;
}

```

- IF com exceção (se o teste falha, executa outra declaração ou bloco).
Pode na exceção executar uma instrução apenas ou um bloco.

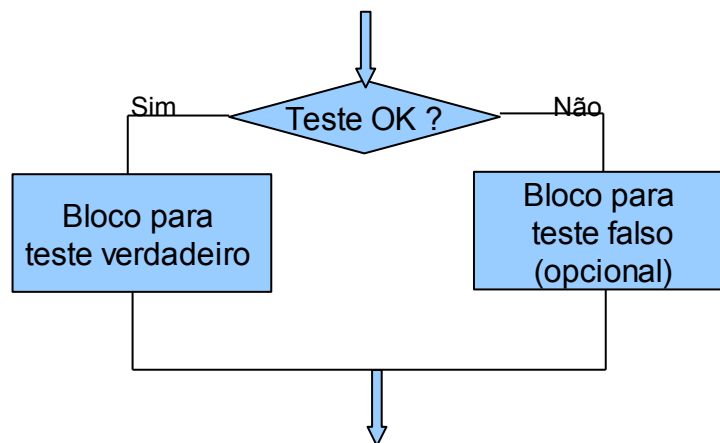
```

if ( teste ok )
    declaração individual ou bloco
else
    declaração individual ou bloco da exceção

```

Importante: A instrução (declaração) simples não precisa estar na mesma linha do IF ou do ELSE. (Ver item II.11.2.1, nos exemplos de IF's aninhados).

Fluxograma genérico para o controle IF:



Podemos então resumir numa tabela todas as combinações dos IF's:

- if (teste a realizar)
 instrução para teste "OK"
- if (teste a realizar)
 {
 grupo de instruções para teste "OK"
 }
- if (teste a realizar)
 instrução para teste "OK"
 else
 instrução para teste "NÃO OK"
- if (teste a realizar)
 {
 grupo de instruções para teste "OK"
 }


```

else
    instrução para teste "NÃO OK"

•if (teste a realizar)
    instrução para teste "OK"
else
{
    grupo de instruções para teste "NÃO OK"
}

•if (teste a realizar)
{
    grupo de instruções para teste "OK"
}
else
{
    grupo de instruções para teste "NÃO OK"
}

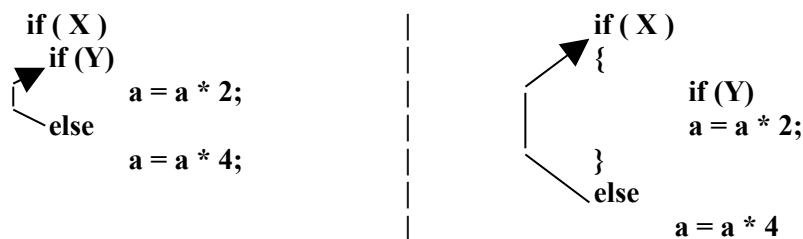
```

II.12.2.1 - IF's aninhados (embutidos um no outro)

Chamamos a estrutura de "IF's aninhados" quando a instrução a ser executada para um teste (seja verdadeiro ou falso) é na verdade outro IF.

Vamos ver dois exemplos que ajudarão a esclarecer o assunto.

Exemplo 1: Observe os dois trechos de programa a seguir:



No trecho da esquerda, o 'else' refere-se ao **if (Y)**, pois esta "mais próximo" deste. Somente se os if's (X) e (Y) resultarem falsos é que a linha $a = a * 4$ será executada. Se o if (X) resultar *falso*, nenhuma operação será realizada.

No trecho da direita, o else refere-se ao **if (X)**, pois o if (Y) esta dentro de um bloco, não sendo visível para o else.

Se o if (X) resultar *verdadeiro* mas o if(Y) resultar *falso*, nenhuma operação será realizada.

Exemplo 2: Vários IF's seqüenciais

<pre> if (posição == 1) peso = 1; else if (posição == 2) peso = 2; else if (posição == 3) peso = 4; else if (posição == 4) peso = 8; else peso = 0; </pre>	<pre> // Vê se posição = 1. // É 1. Faz peso = 1. // Não é 1. Vê se posição= 2. // É 2. Faz peso = 2. // Não é 2. Vê se posição = 3. // É 3. Faz peso = 4. // Não é 3. Vê se posição = 4. // É 4. Faz peso = 8. // Não é 4. Faz peso = 0. </pre>
---	--

II.12.3 - Bloco FOR (executar por um certo número de vezes)

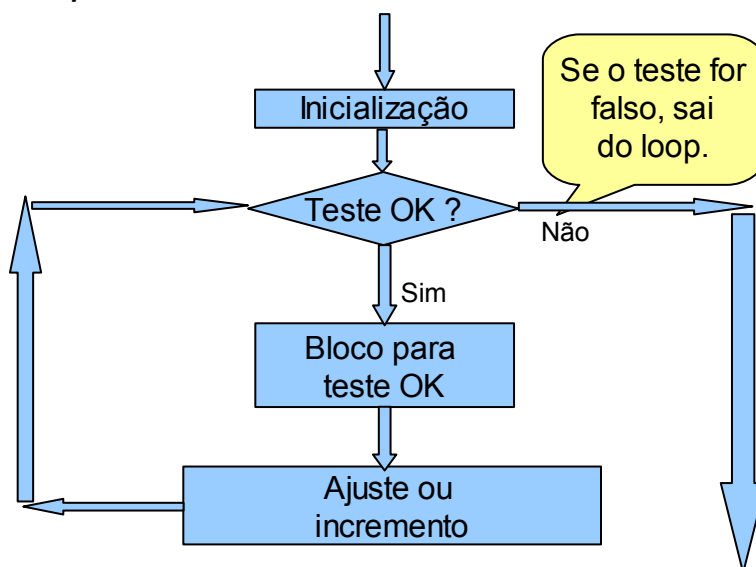
A idéia do bloco FOR é executar uma instrução ou um bloco de instruções repetidamente, por um número de vezes determinado pela chamada do loop.

Sua sintaxe é a seguinte:

```

for ( inicialização ; condição de teste ; ajuste ou incremento )
    instrução ;
ou
for ( inicialização ; condição de teste ; ajuste ou incremento )
{
    ( grupo de instruções )
}

```

Fluxograma genérico para o controle FOR:

Continua o programa

Exemplo 1:

Para melhor entendimento, vejamos um exemplo de um loop que conta de 1 a 100, e escreve este valor numa variável chamada CONTADOR.

```
for ( i = 1; i < 101 ; i++ )
    CONTADOR = i;
```

Exercício proposto: faça numa folha de papel a simulação do FOR acima para ver se entendeu o conceito

Exemplo 2:

Se desejarmos escrever na variável 'contador' e ainda somar estes números, podemos usar o seguinte programa:

```
int8    contador;           // declarei contador como variável de 8 bits
int16   soma;              // declarei a variável soma como 16 bits.
int8    i;                  /* a variável que é usada no loop também
.                               precisa ser declarada. Neste caso, 1
.                               byte é suficiente */
.
.
soma = 0;                   // faço soma = 0 para inicializar a variável
for ( i = 1; i < 101; i++ )   // i vai de 1 à 100
{
    contador = i;           // escreve ' i ' em contador
    soma = soma + i;        // a soma anterior é somada a ' i '
}
```

II.12.3.1 - Loop infinito com FOR

Podemos criar um loop infinito com a declaração

```
for ( ; ; )    instrução ou função que será executada indefinidamente
```

ou

```
for ( ; ; )
{
    Grupo de instruções que serão executadas indefinidamente
}
```

Lembre-se que o programa de um microcontrolador não tem fim, sempre esta rodando.

Geralmente nosso programa sempre terá ao menos 1 loop infinito.

II.12.4 - O condicional WHILE (enquanto)

O WHILE executa a instrução ou o bloco de instruções “enquanto” a condição de teste for verdadeira.

“Se logo no início do teste a condição resultar ‘falsa’, nada será executado.”

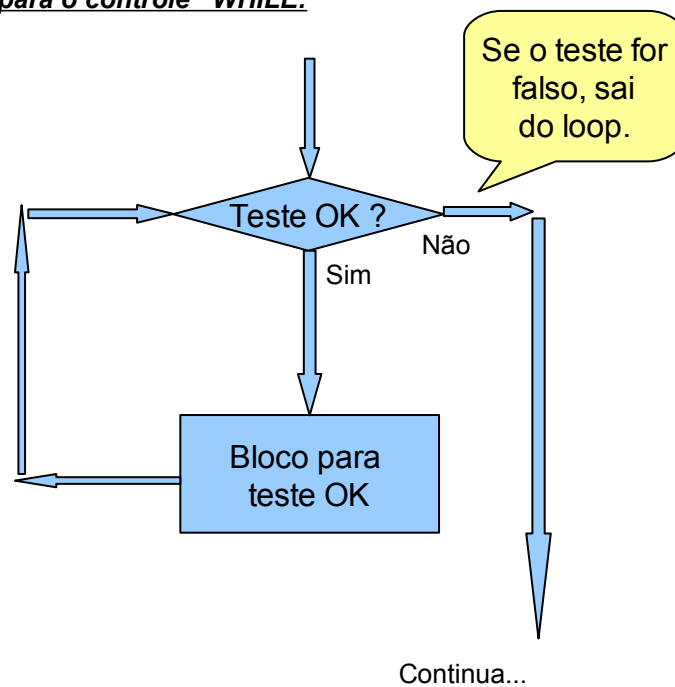
Sintaxe:

```

while ( teste )
    instrução para teste verdadeiro
ou
while ( teste )
{
    ( grupo de instruções para teste verdadeiro)
}

```

Fluxograma genérico para o controle WHILE:



Vamos proceder novamente a soma dos números de 1 a 100 como no exemplo anterior.

```

i = 1;                               // nunca esquecer de inicializar
soma = 0;                             // todas as variáveis do programa.

while ( i < 101 )                     // faça enquanto i < 101
{
    soma = soma + i;
    i++;                             /* como o WHILE apenas faz o teste
                                     devo incrementar a variável */
}

```

II.12.4.1 - Loop infinito com WHILE

```
while ( 1 )           // ou podemos escrever while (true)
{
    ( declarações executadas eternamente )
}
```

A condição booleana “1” sempre será verdadeira (1 = true), logo o bloco do while será executado eternamente, MAS aconselha-se usar o FOR(;;) para evitar que o compilador emita um aviso sobre o fato da condição sempre resultar verdadeira.

II.12.5 - O condicional DO . . . WHILE (faça ... enquanto)

O condicional DO . . . WHILE funciona de forma semelhante ao WHILE, exceto pelo seguinte fato:

“pelo menos uma vez a instrução ou o bloco serão executados”.

Sua sintaxe é:

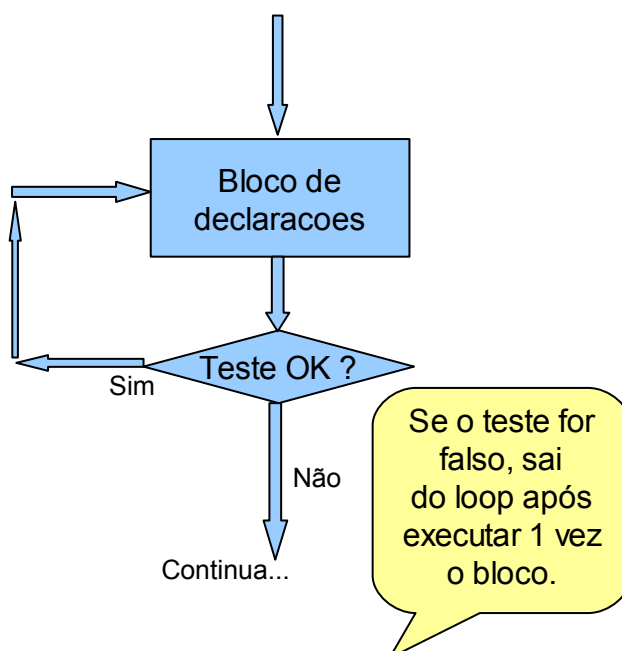
```
do          instrução para teste verdadeiro
while ( teste ) ;

ou

do
{
    ( grupo de instruções para teste verdadeiro )
}
while ( teste ) ;
```

A instrução (ou o bloco) é executada “pelo menos uma vez” porque neste formato o teste dentro do While será realizado APÓS a execução do bloco, enquanto que no bloco While comum o teste é executado ANTES.

Fluxograma genérico para o controle DO...WHILE:



Il.12.6 – O comando BREAK

O comando **BREAK** permite a saída de qualquer loop.

Ao encontrar o break, o programa pula para a instrução logo após o loop.

Exemplo:

[illegible]

Este programa vai somando 2 à variável soma, até que i seja igual a 5, independente do loop ir até 50. Neste caso o valor final da soma será 10.

Estude o exemplo que esta na pasta **break** no arquivo para download no link <http://www.newtoncbraga.com.br/cap2.zip>

II.12.7 – O comando CONTINUE

O comando `BREAK` visto anteriormente permite a “saída” do loop a qualquer momento, encerrando sua execução.

Já o comando CONTINUE funciona de forma similar, mas sem sair do loop, apenas pulando para o próximo valor do mesmo.

Ao encontrar o comando CONTINUE, o programa “pula” todas as instruções entre o CONTINUE e o ponto onde faz a atualização do loop

Exemplo:

[illegible]

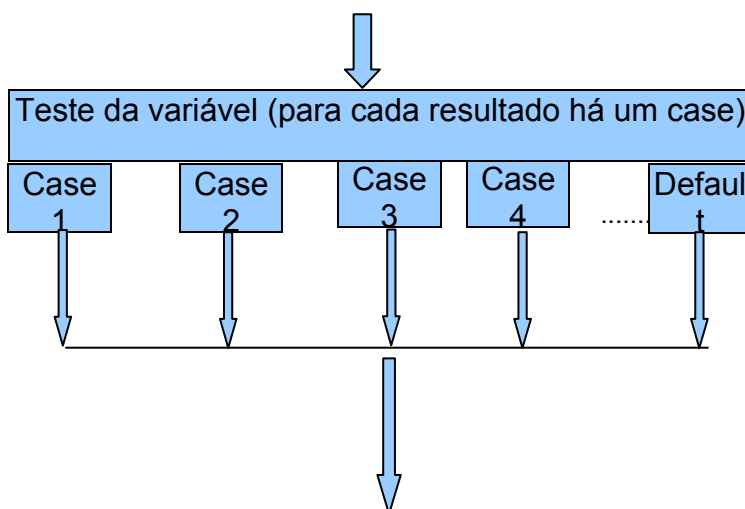
Estude o exemplo que esta na pasta **continue** no arquivo para download no link
<http://www.newtoncbraga.com.br/cap2.zip>

II.12.8 - O condicional SWITCH

O SWITCH testa a variável e conforme seu valor “pula” (isto é, realiza um desvio) diretamente para o grupo de instruções conforme a cláusula CASE.

Caso nenhuma das condições previstas nos vários CASE sejam satisfeitas, executa o bloco DEFAULT, se houver (o *default* não é obrigatório).

Fluxograma genérico para o controle SWITCH:



Mesmo que apenas uma instrução seja usada para um certo CASE, devemos incluir a instrução **BREAK**, que faz com que o programa vá imediatamente para o fim do SWITCH, continuando a partir daí (ver item II.12.6 acima).

Caso o BREAK não seja colocado, o programa continuará pelo CASE logo abaixo do que foi chamado (ou no DEFAULT), resultando em erro de sequência lógica.

Seu formato geral é:

```

switch ( variável )
{
    case constante1
        ( instrução ou grupo de instruções )
        break;

    case constante2
        ( instrução ou grupo de instruções )
        break;

    .
    .
    .

    default:
        ( instrução ou grupo de instruções para falso geral )
        break;
}
  
```

Vamos ver um exemplo que faz o mesmo que o IF aninhado que vimos anteriormente.

```
switch ( posição )
{
    case 1:                // CASO posição = 1 ....
        peso = 1;
        break;            // sai do CASE.
    case 2:
        peso = 2;
        break;
    case 3:
        peso = 4;
        break;
    case 4:
        peso = 8;
        break;
    default:                // CASO posição NÃO seja 1, 2, 3
                           // ou 4, executa este bloco
        peso = 0;
        break;
}
```

Em alguns casos, embora nem sempre seja recomendável, podemos escrever várias declarações em uma só linha quando apenas 1 ou 2 instruções são usadas:

```
switch ( posição )
{
    case 1: peso = 1; break;           // CASO peso=1, ....
    case 2: peso = 2; break;           // CASO peso=2, ....
    case 3: peso = 4; break;
    case 4: peso = 8; break;
    default: peso = 0; break;
}
```

II.12.9 - O comando RETURN

O comando RETURN encerra imediatamente a execução da função e executa um retorno para a função de chamada.

É o mesmo que encontrar o fim da função, mas de forma proposital.

O comando RETURN permite ainda indicar um valor para ser devolvido para a função de chamada.

Se nenhum valor for indicado, e a função de chamada estiver esperando um retorno, o valor recebido será indefinido.

Exemplo: Chamamos a função dobrar enviando um valor e a mesma RETORNA o resultado

```
int8    i;
int8    dobro = 0;

int8 dobrar (int8 valor)           // recebe um valor e devolve o dobro
```



```
{  
    return valor+valor;  
}  
  
void main (void)  
{  
    for( i=1 ; i<11 ; i++ )        // i vai de 1 a 10  
    {  
        dobro = dobrar ( i );  
    }  
}
```

IMPORTANTE:

- 1) Veremos adiante que todo programa deve ter a função MAIN (). Só para constar, no reset o processador será desviado sempre para a função MAIN().
- 2) No item II.14.3 vamos detalhar a idéia de passar valores entre as funções

Estude o exemplo que esta na pasta **return** no arquivo para download no link
<http://www.newtoncbraga.com.br/cap2.zip>

II.13 - Abreviações úteis para instruções aritméticas

II.13.1 - Incremento e Decremento

Duas delas foram mostradas no item II.8 (Operadores lógicos e aritméticos básicos da linguagem C), a saber:

- ++ Incremento da variável indicada

O operador ‘++’ facilita a escrita de instruções para o incremento de variáveis.

Ao invés de escrever: tempo = tempo + 1;

podemos escrever: tempo++; ou ++tempo;
(ver item I.12.2 a seguir)

- -- Decremento da variável indicada

O operador ‘--’ facilita a escrita de instruções para o decremento de variáveis.

Ao invés de escrever: tempo = tempo - 1;

podemos escrever: tempo--; ou --tempo;

II.13.2 - Combinando abreviações

Podemos combinar numa única linha abreviações e atribuições, de forma a facilitar a escrita do programa.

Importante: Os iniciantes na linguagem devem usar inicialmente as instruções básicas para não se confundirem.

Vamos explicar com alguns exemplos:

$$Y = ++ X; \quad \left\{ \begin{array}{l} X = X + 1; \\ Y = X; \end{array} \right.$$

Como o ++ esta 'ANTES' do X, primeiro o X será incrementado para depois Y receber o valor.

Alterando a posição do incremento, teremos:

$$Y = X ++; \quad \left\{ \begin{array}{l} Y = X; \\ X = X + 1; \end{array} \right.$$

Como o ++ esta 'DEPOIS' do X, primeiro o valor de X será copiado para Y e depois incrementado.

Estude o exemplo que esta na pasta **abreviações** no arquivo para download no link <http://www.newtoncbraga.com.br/cap2.zip>

II.13.3 – Operações com resultado na mesma variável

Sempre que utilizarmos no resultado a mesma variável, podemos abreviar a operação, o que reduz o trabalho de digitação.

Instrução normal

Tempo_de_resposta = Tempo_de_resposta * 10;

Instrução abreviada

Tempo_de_resposta *= 10;

II.14 - Variáveis locais, variáveis globais e parâmetros

Existem duas maneiras principais para a alocação de variáveis em memória:

II.14.1 - Variáveis Globais

As variáveis globais têm "validade" para todas as rotinas e funções existentes no programa. Isto significa que qualquer rotina ou função poderá utilizá-la.

II.14.2 - Variáveis Locais

As variáveis locais tem "validade" apenas dentro das rotinas ou funções onde foram criadas. Isto significa que apenas a rotina ou função onde a mesma foi criada poderá utilizá-la.

Vamos ver um fragmento de código como exemplo:

```

int8 A,B,C;           // A, B e C são variáveis globais, pois estão
                      // fora de qualquer função ou rotina.

main ( )
{
    A = 0;           // Faz A=0
    Tempo( );        // chama rotina Tempo
    Espera( );        // chama rotina Espera
}
.
.
Tempo( )
{
    int8 J;           // cria uma variável chamada J.
                      // Somente a rotina Tempo( ) poderá utilizá-la

    J = A;
}
.
Espera( )
{
    int8 X;           // cria uma variável chamada X.
                      // Somente a rotina Espera( ) poderá utilizá-la

    X = A;
    X = J;           // >>>>>>>>>> ERRO: A variável J não existe para esta rotina
}

```

Vantagens: A principal vantagem no uso das variáveis locais está em que a memória RAM ocupada pela mesma poderá ser compartilhada por outras variáveis locais, permitindo um “aumento” relativo no espaço de memória, pois as variáveis locais de duas rotinas que nunca se comunicam poderão estar no mesmo endereço físico de memória.

Este trabalho de alocação de memória é feito automaticamente pelo compilador. Cabe ao programador apenas declarar as variáveis corretamente (locais ou globais).

II.14.3 - Variáveis como parâmetros

Outra facilidade da linguagem C está na passagem de valores de uma rotina ou função para outra.

Estes valores são conhecidos por “parâmetros”.

Vamos imaginar uma rotina que calcula o quadrado de um certo número.

Podemos chamar a rotina assim:

```

...
Número = 5;           // atribui um valor a variável 'número'
Quadrado( );          // chama a função quadrado e o resultado será
                      // armazenado na variável 'result'

X = result;
.
.
void Quadrado( )
{
    Result = Número * Número;
}

```

Ou podemos chamar de forma mais elegante:

```
X = Quadrado(5);           // chama a função 'quadrado' enviando
                           // como parâmetro o número desejado
                           // e o resultado será automaticamente
                           // armazenado na variável usada na
                           // chamada, X

...
int16 Quadrado(int8 H)      // crio uma variável local para receber o
                           // parâmetro
{
    return(H * H);         // retorna uma variável 'int16'
}
```

II.15 - A variável tipo VOID e os protótipos de funções

Existe um tipo de variável especial chamada VOID. Seu significado seria mais ou menos do tipo VAZIO, e não ocupa espaço de memória.

É usada principalmente na chamada de rotinas ou no retorno das mesmas, para indicar que nenhum valor foi “enviado” ou será “retornado” pela função.

Para melhor entendimento, vejamos uma rotina que não precisa de parâmetros e nada retorna:

```
LeTeclas();               // chama a rotina sem enviar nada
.
void LeTeclas()           // este void indica que a rotina não
{                          // envia parâmetros de volta
.
}
```

II.15.1 - Protótipos de funções

Os protótipos de funções indicam ao compilador:

- qual o tipo de variável a função retorna
- quantas e também quais os tipos das variáveis a função recebe (mais conhecidas por parâmetros ou argumentos)

Caso durante a chamada da função o programador escreva mais ou menos argumentos, ou erre o tipo (declarou int e manda float) o compilador sinaliza o erro, ajudando no desenvolvimento do programa

Outra facilidade esta na ‘localização’ da função.

Vejamos:

Por padrão, durante o processo de compilação, o compilador C sempre busca as funções **acima** do ponto onde são chamadas. Para evitar erros de compilação, como por exemplo o compilador não encontrar uma rotina porque a mesma esta abaixo do ponto de chamada OU em outro arquivo (como uma biblioteca), podemos avisar o compilador da existência das rotinas usando o conceito de protótipo.

Esta declaração da rotina, feita apenas com seu nome, sem suas instruções e finalizada pelo “ ; ” é conhecida por **protótipo**.

Exemplos:

```
void LeTeclas (void);      // Este protótipo indica que a rotina LeTeclas não
                           // recebe parâmetros e também não retorna
                           // valores.

void Tempo (int8 yyy);     // Este protótipo indica que a rotina Tempo
                           // receberá um valor do tipo int8, que receberá o
                           // nome de yyy para seu uso, mas não retornará
                           // nada.
```

II.16 - Estruturas

Uma estrutura é um agrupamento de variáveis formando uma nova variável, mais complexa, que dá ao programador mais flexibilidade na manipulação de dados.

Podemos resumir que uma estrutura é um conjunto de variáveis, de vários tamanhos, que são agrupadas em uma matriz, mas cujos elementos são chamados pelos seus nomes mais o nome da matriz.

Vejam um exemplo que vai esclarecer tudo.

Imagine que você quer criar um conjunto de dados para salvar os seguintes elementos relativos a 3 veículos:

- velocidade
- distância
- tempo

Você poderia criar assim:

```
int8   velocidade1, velocidade2, velocidade3;
int16  distancia1, distancia2, distancia3;
int32  tempo1, tempo2, tempo3;
```

Uma maneira mais elegante e fácil de ser visualizada é pela utilização de uma estrutura, onde agruparemos os tipos básicos dos dados em um conjunto com um nome genérico, usado apenas para referenciar ao compilador o tipo de dados, e que chamaremos de “veículo”.

```
struct  DadosGerais      // nome genérico da estrutura
{
    int8   velocidade;
    int16  distancia;
    int32  tempo;
};

struct DadosGerais  veiculo1,veiculo2,veiculo3;
                  // nome das variáveis que conterão os dados da estrutura
```

Para usar as variáveis não precisaremos mais usar os nomes individuais, mas sim o nome do conjunto + o ponto decimal + o nome da variável.

Por exemplo, para escrever 10 na variável velocidade do veículo 2, usaremos a seguinte notação:

```
veiculo2.velocidade = 10;
```

Neste caso a variável velocidade “escrita de forma individual” não será reconhecida pelo compilador.

```
velocidade = 10;           // ERRO DE COMPILAÇÃO
```

Agora imagine que você quer fazer isto para 100 veículos.

Ao invés de criar uma estrutura e colocar nela 100 variáveis, ou seja,

```
struct    DadosGerais      // nome genérico da estrutura
{
    int8    velocidade;
    int16   distancia;
    int32   tempo;
};

struct veiculo1,veiculo2,veiculo3,.....veiculo99,veiculo100;
```

podemos utilizar uma matriz na estrutura, assim:

```
struct    DadosGerais      // nome genérico da estrutura
{
    int8    velocidade;
    int16   distancia;
    int32   tempo;
}
veiculo [100] ;           // temos aqui 100 “veículos”
                           // declarados
```

e a velocidade do veículo 13 pode ser salva da seguinte forma:

```
veiculo [13] . velocidade = 100;
```

Podemos ver nestes exemplos que estruturas e matrizes nos dão uma enorme flexibilidade na manipulação de variáveis complexas e ainda torna o programa mais ‘legível’ para o usuário.

Estude o exemplo que esta na pasta **estruturas** no arquivo para download no link <http://www.newtoncbraga.com.br/cap2.zip>

II.17 - Unions

Embora a declaração de uma UNION seja similar a declaração de estruturas, a UNION tem como característica principal o fato de que todas as variáveis da mesma “COMPARTILHAM” a mesma área de memória. (na estrutura as variáveis são sequenciais e independentes).

Vejamos um exemplo que vai esclarecerá tudo.

Imagina que você tem uma variável float (ponto flutuante que ocupa 4 bytes de ram) e deseja salvá-la na eeprom de dados.

Com o exemplo da UNION vamos mostrar como fazer isso facilmente.

Primeiro vamos criar uma union que vai compartilhar a variável float com uma matriz de 4 bytes:

```

union pacote           // nome genérico da union
{
    float Valor32;     // variável float de 32 bits
    int8 bytes[4];     // matriz para abrigar os 4 bytes do float
};

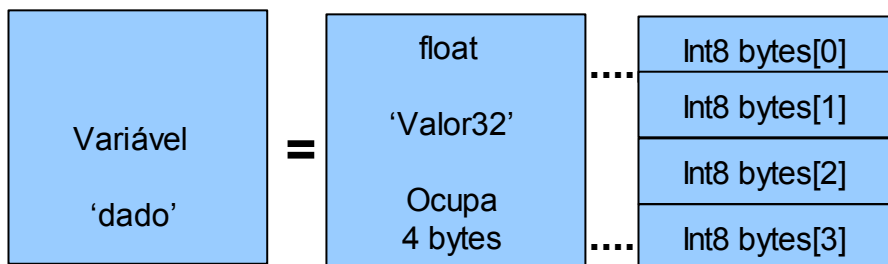
```

```

union pacote dado;     // dado é a variável que UNE os dados

```

Mapa de memória ocupada pela union:



Exemplo de utilização:

```

valor32 = 132.23;           // valor ponto flutuante. Como salvar na
                             // eeprom ?

                             // com o auxilio da union
                             // salvamos byte a byte
                             // os 4 bytes que compoem
                             // a union

write_eeprom ( 0 , bytes [ 0 ] );
write_eeprom ( 1 , bytes [ 1 ] );
write_eeprom ( 2 , bytes [ 2 ] );
write_eeprom ( 3 , bytes [ 3 ] );

```

Estude o exemplo que esta na pasta **unions** no arquivo para download no link <http://www.newtoncbraga.com.br/cap2.zip>

II.18 - A função MAIN ()

Todo programa em C deve ter uma função chamada MAIN (), que é a rotina que será executada quando o programa for chamado (em nosso caso após o reset da cpu).

Exemplo de um programa que nada faz, mas que pode ser compilado:

```

#include <16F877A.H>

void main (void)           // a função MAIN não recebe ou devolve valores
{
    ;
}

```

II.19 - Exemplos de programas simples

Veremos neste item alguns programas pequenos e simples, sem função técnica específica, apenas para ilustrar os conceitos vistos até agora. Use-os para treinar o uso do Mplab e do compilador C (Cap. III).

Programa 1: Calcula a soma dos números de 1 a 100.

```
#include <16F877A.H>

int8   i;           // variável i usada no loop (0 a 255)
int16  soma;        // soma pode ir de 0 a 65535

void main( )
{
    for ( ;; )
    {
        soma = 0;           // inicializei a variável soma
        for ( i = 1; i < 101; i++)
            soma = soma + i;
    }
}
```

Programa 2:

Conta indefinidamente de 1 a 10. Sempre que chegar a 10 incrementa a variável X e recomeça.

```
#include <16F877A.H>
int8   i, x;        // declarei que i e x são
                    // variáveis do tipo int8

void main ( void)
{
    i = 0; x = 0;    // inicializei as variáveis
    for ( ;; )      // fica eternamente neste loop
    {
        i++;        // incrementa i
        if ( i == 10 ) // vê se i = 10.
        {
            x++;    // É. Incrementa x e faz i = 0.
            i = 0;
        }
    }
}
```

Programa 3:

Incrementa indefinidamente a variável chamada contador e escreve nos leds colocados no port D

```
#include <16F877A.H>
int8   contador;    // declarei que 'contador' é uma
                    // variável do tipo int8 (8 bits)

void main ( void)
{
    contador = 0;
    for ( ;; )      // fica eternamente neste loop
    {
        output_D ( contador);
        contador ++;
    }
}
```


Estes 3 programas estão nas pastas ex1, ex2 e ex3 no arquivo para download no link <http://www.newtoncbraga.com.br/cap2.zip>


```
#ifndef 877
    #include <16F877.H>           // Define processador 16F877
    #IFDEF mododebugger
        #device icd=true
        #Fuses HS,NOWDT,PUT,NOPROTECT,NOBROWNOUT,NOLVP,NOCPD,NOWRT
    #else
        #Fuses HS,NOWDT,PUT,PROTECT,NOBROWNOUT,NOLVP,NOCPD,NOWRT
    #ENDIF
#endif

#ifndef 877A
    #include <16F877A.H>         // Define processador 16F877A
    #IFDEF mododebugger
        #device icd=true
        #Fuses HS,NOWDT,PUT,NOPROTECT,NOBROWNOUT,NOLVP,NOCPD,NOWRT
    #else
        #Fuses HS,NOWDT,PUT,PROTECT,NOBROWNOUT,NOLVP,NOCPD,NOWRT
    #ENDIF
#endif
```

Neste trecho de código temos quatro ‘defines’ (veremos com detalhes no capítulo VI), um que indica **452, 458, 877 e 877A**.

APENAS UM PODERÁ ESTAR HABILITADO.

Os outros “defines”, referentes aos modelos que NÃO serão usados, devem ser precedidos do marcado de comentário, “//”. Em nossos exemplos, todos estarão pré-habilitados para o 16F877A.

Em vários pontos do programa teremos os indicadores de “compilação condicional”, isto é, o compilador decidirá, baseado em qual ‘define’ esta ativo, qual trecho de código será compilado. Desta forma podemos ter um único programa fonte para mais de um modelo de pic.

```
//=====//  
// Programa modelo para os pics 18F452, 18F458, 16F877 e 16F877A  
// Apenas para estudo da linguagem e do ambiente de desenvolvimento  
//=====//  
// VIXEM Microcontroladores  
// Curso de Microcontroladores PIC  
// Revisão: 02/01/2008  
//=====//  
  
//=====//  
// SELECAO DO TIPO DE PROCESSADOR  
// >>>>>>>>>>>>>>>>>> Apenas 1 define pode existir.  
// Coloque como 'comentario' os que nao estiver usando  
  
##define 458  
##define 452  
##define 877  
#define 877A  
  
//=====//  
//=====//  
// OS AJUSTES SAO AUTOMATICAMENTE REALIZADOS  
// CONFORME O MODELO DO PROCESSADOR SELECIONADO  
#ifdef 458  
    #include <18F458.H> // Define processador 18F458  
    #IFDEF mododebugger  
        #device icd=true  
        #fuses noWRTD, noWRTB, noCPD, noCPB, noWRT, noWRTC, noEBTR, noEBTRB  
        #fuses NOOSCSN, noSTVREN, NOLVP, noPROTECT, noWDT, WDT128  
        #fuses noBROWNOUT, BORV27, HS, PUT, debug  
    #else  
        #fuses noWRTD, noWRTB, CPD, noCPB, noWRT, noWRTC, noEBTR, noEBTRB  
        #fuses NOOSCSN, noSTVREN, NOLVP, PROTECT, noWDT, WDT128  
        #fuses noBROWNOUT, BORV27, HS, PUT, nodebug  
    #ENDIF  
#endif  
#ifdef 452  
    #include <18F452.H> // Define processador 18F452  
    #IFDEF mododebugger  
        #device icd=true  
        #fuses noWRTD, noWRTB, noCPD, noCPB, noWRT, noWRTC, noEBTR, noEBTRB  
        #fuses NOOSCSN, noSTVREN, NOLVP, noPROTECT, noWDT, WDT128  
        #fuses noBROWNOUT, BORV27, HS, PUT, debug  
    #else  
        #fuses noWRTD, noWRTB, CPD, noCPB, noWRT, noWRTC, noEBTR, noEBTRB  
        #fuses NOOSCSN, noSTVREN, NOLVP, PROTECT, noWDT, WDT128  
        #fuses noBROWNOUT, BORV27, HS, PUT, nodebug  
    #ENDIF  
#endif  
#ifdef 877  
    #include <16F877.H> // Define processador 16F877  
    #IFDEF mododebugger  
        #device icd=true  
        #Fuses HS, NOWDT, PUT, NOPROTECT, NOBROWNOUT, NOLVP, NOCPD, NOWRT  
    #else  
        #Fuses HS, NOWDT, PUT, PROTECT, NOBROWNOUT, NOLVP, NOCPD, NOWRT  
    #ENDIF  
#endif  
#ifdef 877A  
    #include <16F877A.H> // Define processador 16F877A  
    #IFDEF mododebugger  
        #device icd=true  
        #Fuses HS, NOWDT, PUT, NOPROTECT, NOBROWNOUT, NOLVP, NOCPD, NOWRT  
    #else
```

```

    #Fuses HS,NOWDT,PUT,PROTECT,NOBROWNOUT,NOLVP,NOCPSD,NOWRT
    #ENDIF
#endif

#include delay(clock=1000000)

////////////////////////////////////
//                                VARIAVEIS
////////////////////////////////////

void main()
{
    // Inicializacao de hardware
    setup_adc_ports(no_analogs);           // se nao usar A/D, devemos ajustar os ports para digital

#ifndef 877
    setup_comparator(NC_NC_NC_NC);        // o 877 nao tem comparador
// se nao usar o comparador, devemos desligar o mesmo
#endif

    /////////////////////////////////// DEMAIS AJUSTES DE SEU PROGRAMA

    /////////////////////////////////// PROGRAMA PRINCIPAL
    for (;;)
    {

    }
}
//*****
//                                Fim do programa fonte

```

IV - Usando o Mplab 7.62 em C

IV.1 - Conceitos básicos

O MpLab é um ambiente integrado para o estudo e desenvolvimento com a família PIC de microcontroladores.

Sua principal característica é a total integração de seus módulos com o ambiente Windows, e principalmente com o compilador C da CCS, permitindo a fácil copia de arquivos e trechos de arquivos de um aplicativo para outro.

Para se escrever um programa no MpLab, precisamos antes assimilar o conceito de “projeto”.

IV.2 - O “Projeto” no MpLab

Entende-se por **projeto** um conjunto de arquivos e informações que diz ao MpLab qual a situação de um certo trabalho em particular.

Por exemplo, num certo projeto designado EXEMPLO.PJT temos duas janelas abertas, exemplo.c e exemplo.lst.

Em outro projeto designado TESTE.PJT temos três janelas abertas, teste.c, teste.lst. e File Register Window.

Com o gerenciamento de projetos presente no MpLab não precisamos “lembrar” quais as janelas que cada projeto estava usando no momento em que encerramos o trabalho, pois ao carregarmos o projeto desejado todas as informações relativas ao mesmo serão recuperadas.

IMPORTANTE: *Vá ajustando as janelas para que fiquem mais ou menos nas mesmas posições e dimensões apresentadas neste capítulo, para facilitar seu aprendizado.*

IV.3 - Criando o projeto com o Project Wizard

Inicie o Mplab.

Na tela inicial selecione *Project* > *Project Wizard* como na figura 1 e na janela que vai se abrir selecione AVANÇAR.

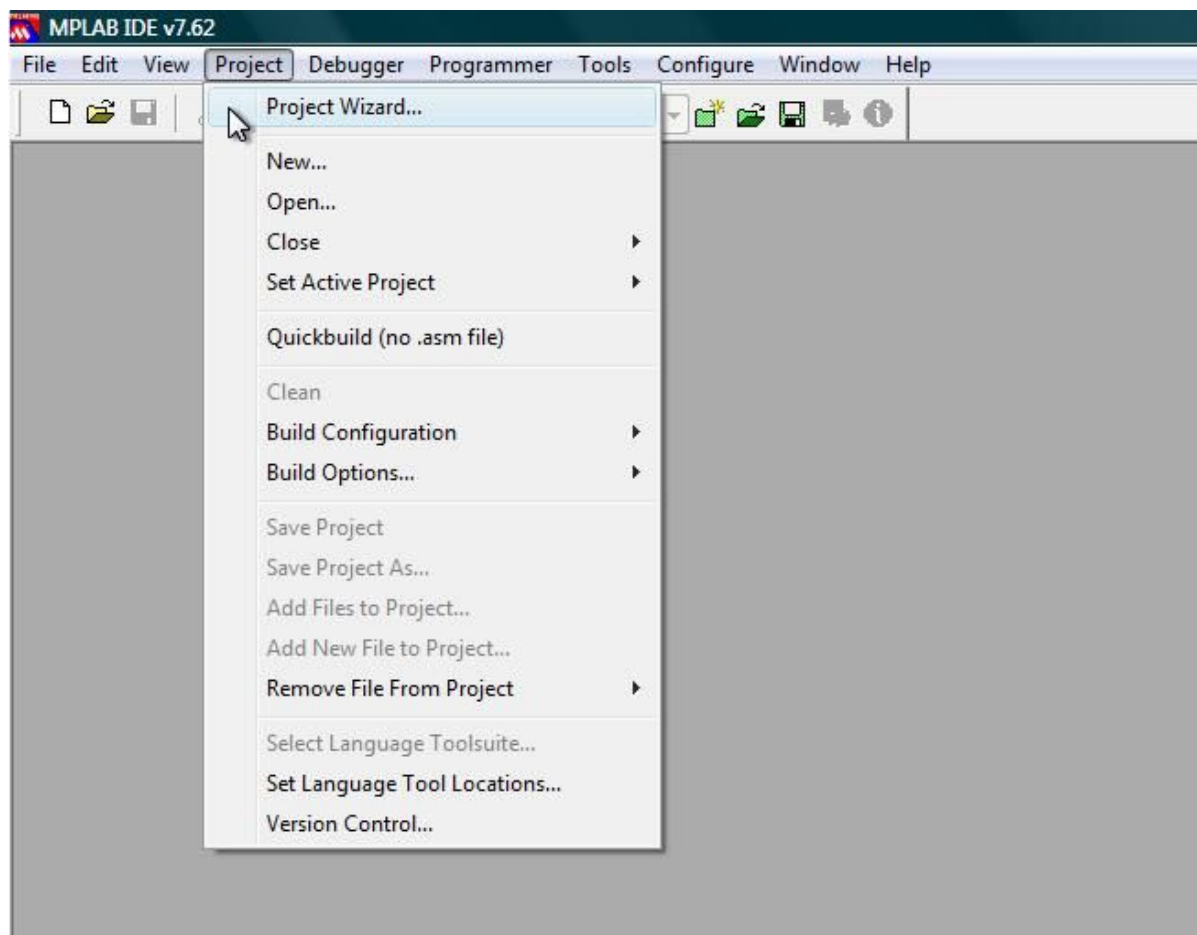


Figura 1

Caso o PIC indicado não seja o 18F458, selecione no menu conforme figura 2

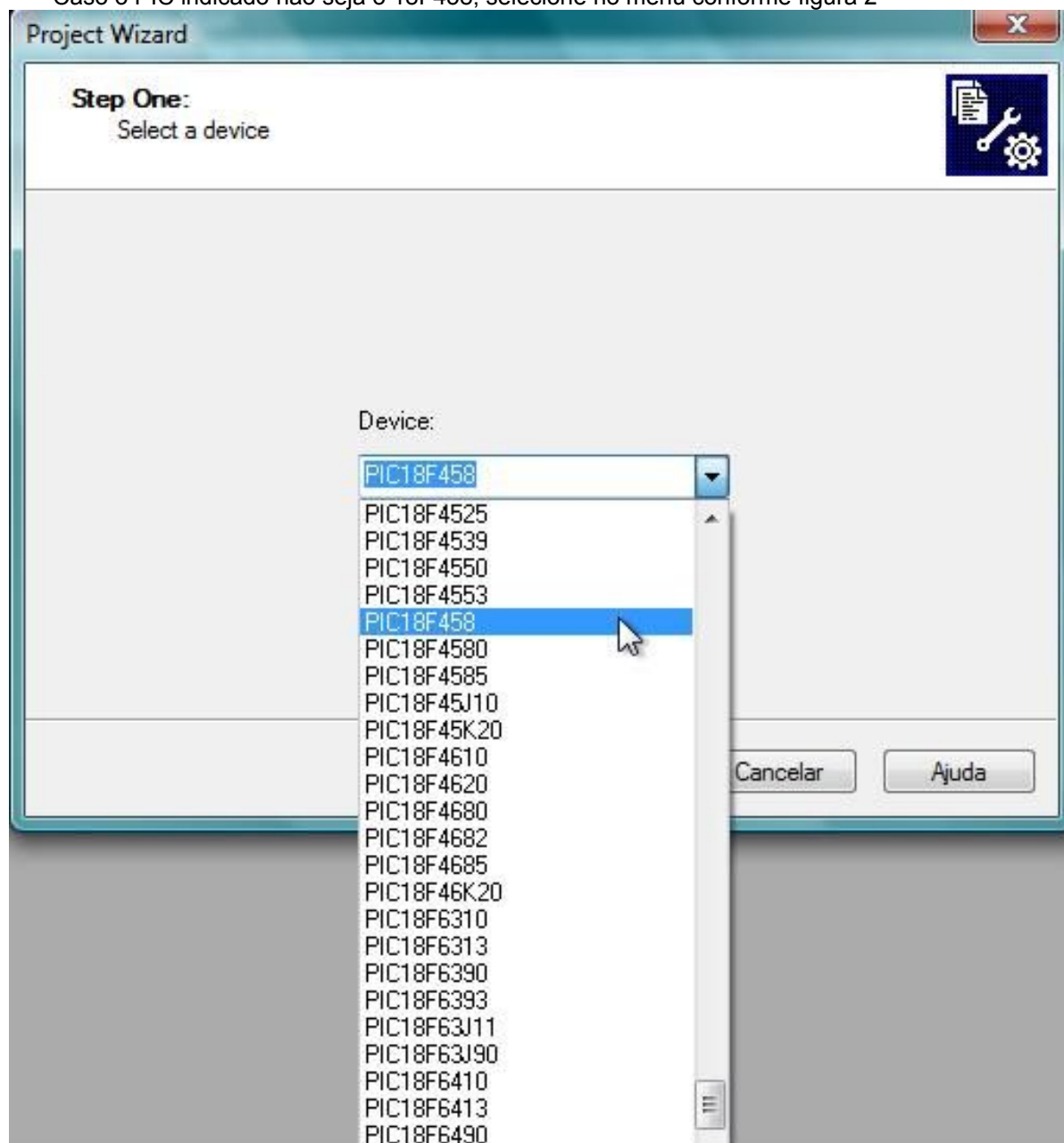


Figura 2

E de um click no botão “Avançar”.

Ao pressionar o botão “Avançar”, iremos para a janela de seleção do tipo de compilador. Na primeira vez você provavelmente obterá a janela da figura 3 (que indica o compilador padrão – MPASM Toolsuite):

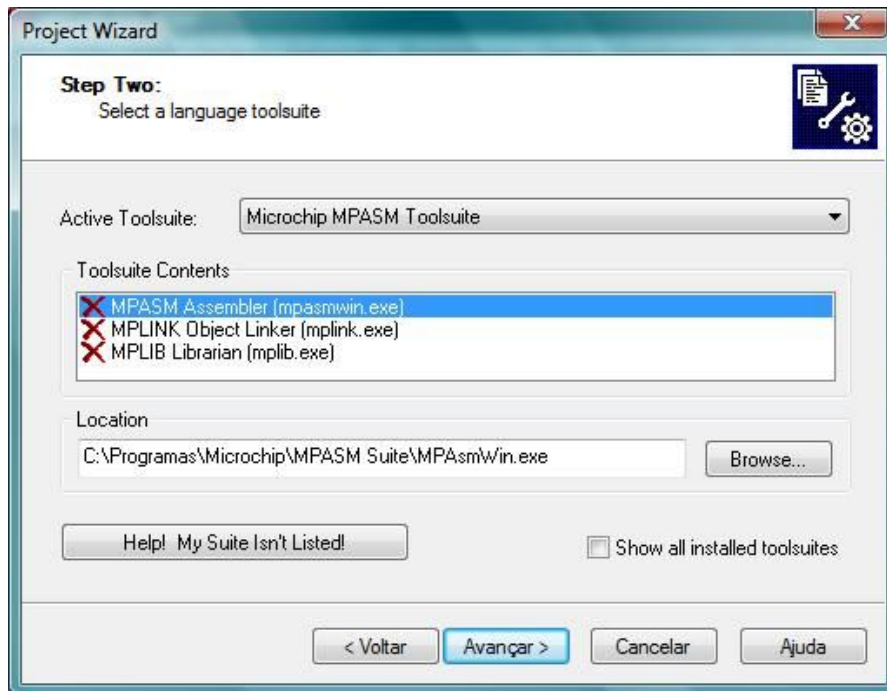


Figura 3

Se for o caso, selecione na linha Active Toolsuite a opção **CCS c Compiler** e obterá a tela a seguir:

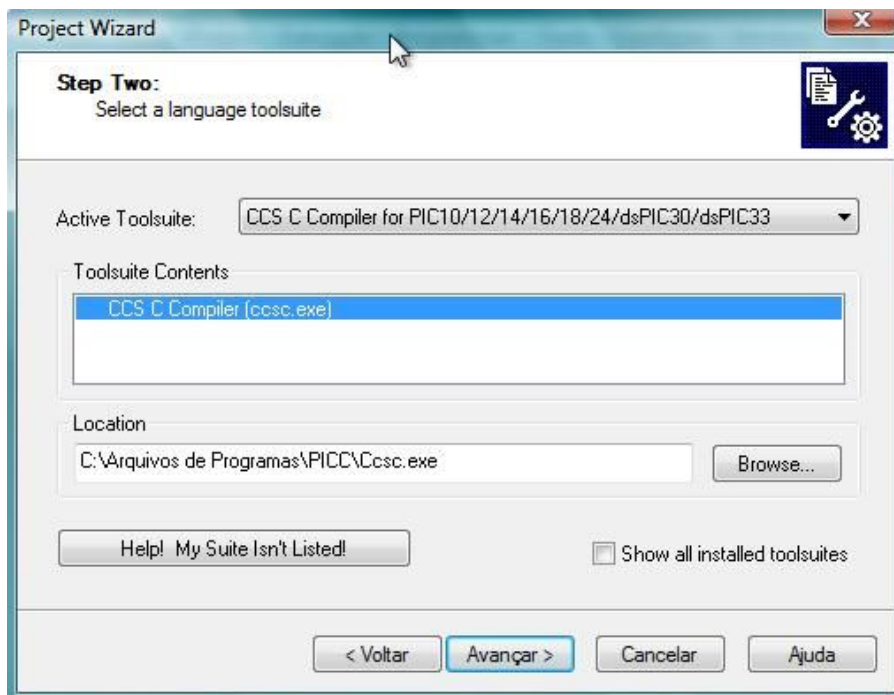


Figura 3a

Caso os ajustes não estejam iguais aos acima, ajuste um a um e pressione o botão “Avançar” mais uma vez. Aparecerá a janela da figura 4, para ajuste do nome e local do projeto

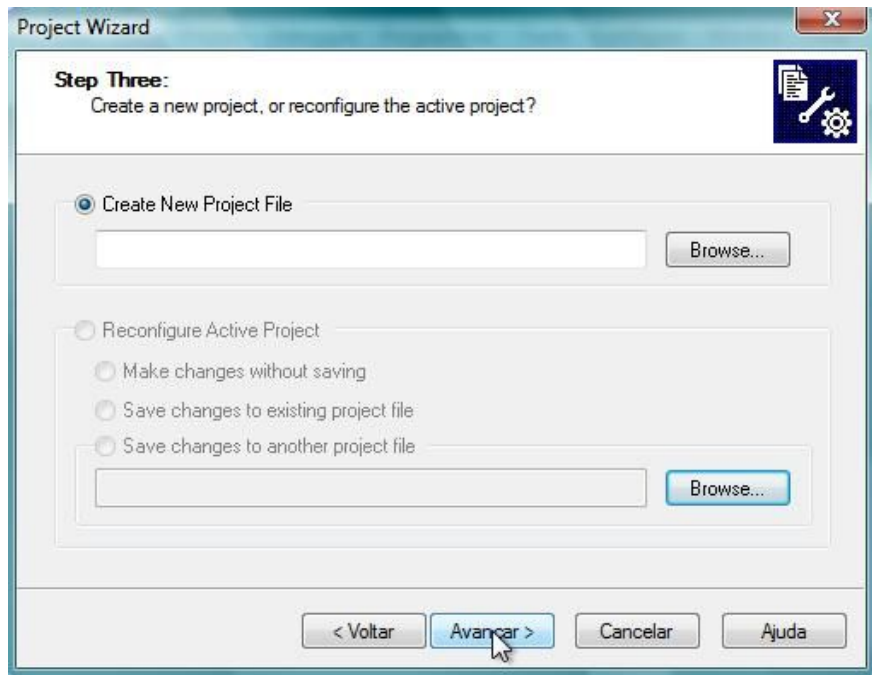


Figura 4

Selecione pelo botão BROWSE qual diretório (ou pasta) deseja usar, ou crie uma pasta para este estudo, chamada CURSOPIC, no drive C, e preencha o nome do projeto como CURSOPIC. Ao final sua tela devera estar assim:

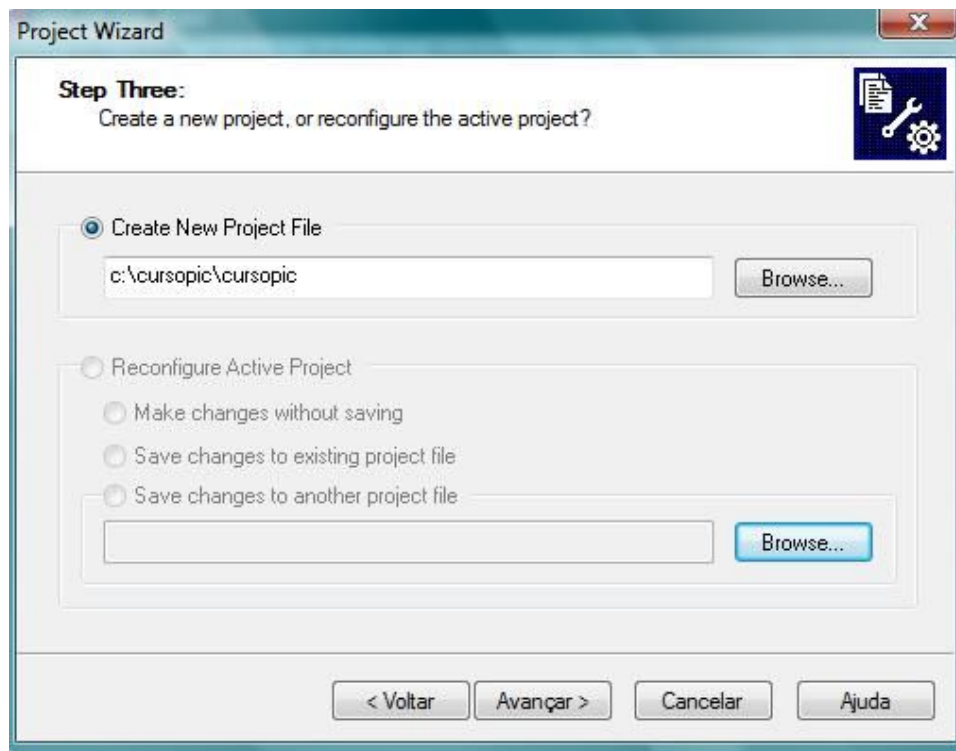


Figura 5

Na janela que vai aparecer apenas selecione “Avançar” mais uma vez, pois ainda não criamos o arquivo do programa fonte.

Depois selecione “Concluir” e o Mplab voltara a tela inicial.

Para continuar, selecione no menu principal a seqüência: *File > New* e obteremos uma janela padrão como na figura 6:

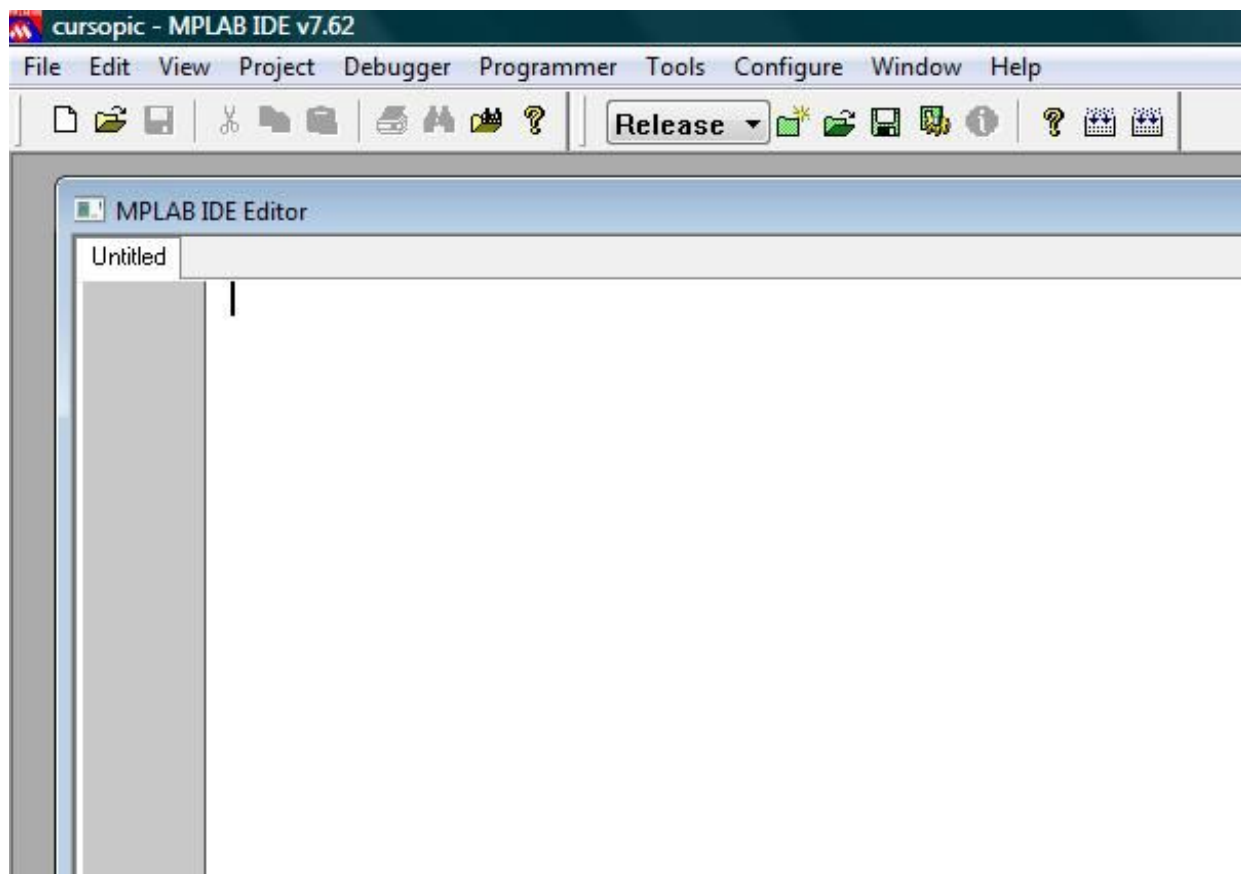


Figura 6

Nesta janela digite o texto exatamente como abaixo:

```
#include <18f458.h>

void main ( )
{
    for ( ; ; )
    {
        output_high (pin_d7);
        output_low (pin_d7);
    }
}
```

e depois selecione no menu *File > Save as...* ajustando conforme figura 7 abaixo:

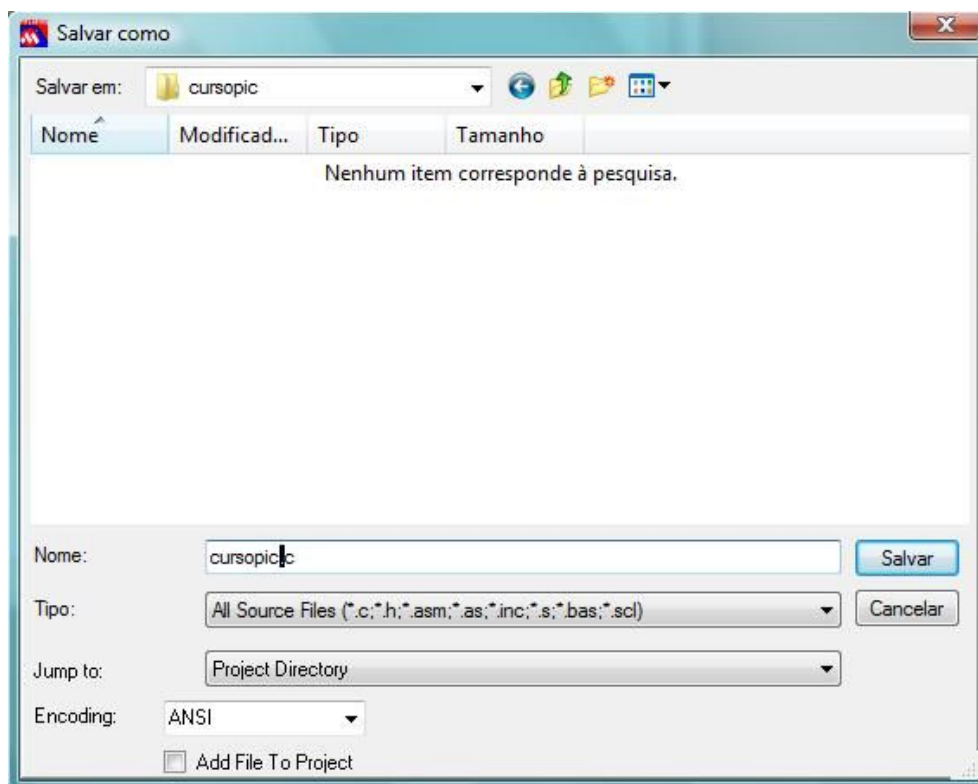


Figura 7

Selecione salvar e o mplab ficará assim (ou similar – redimensione suas janelas se necessário):

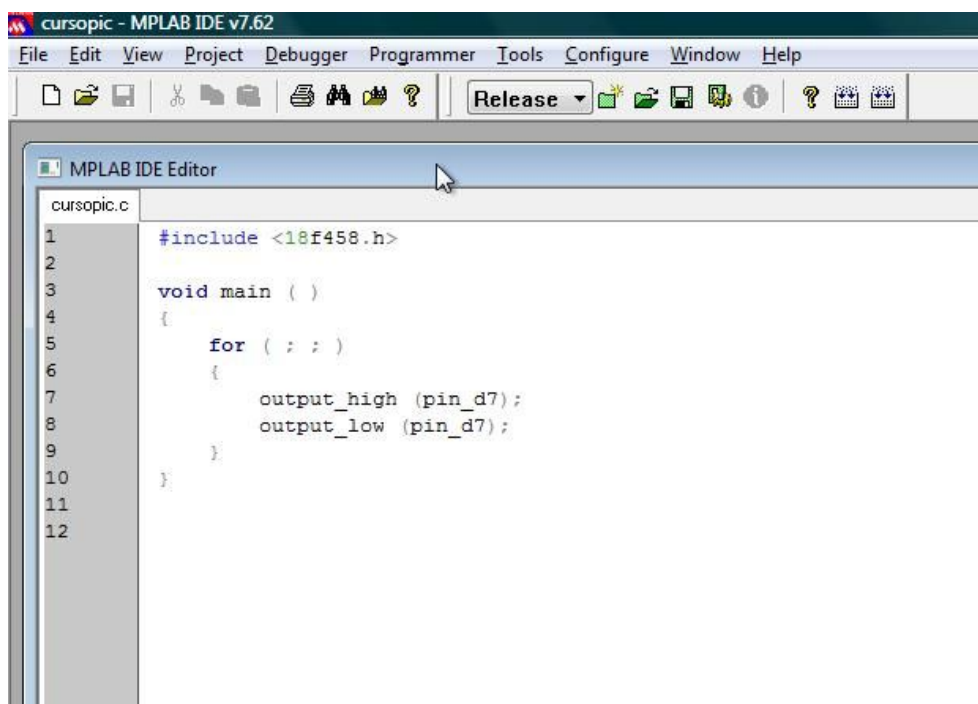


Figura 8

Para podermos compilar o programa, devemos avisar ao Mplab que este fonte será usado, e para tal usamos a sequência

Project > Add Files to Project....

Na janela da figura 9, selecione o fonte “cursopic.c” e de um click em “Abrir”:

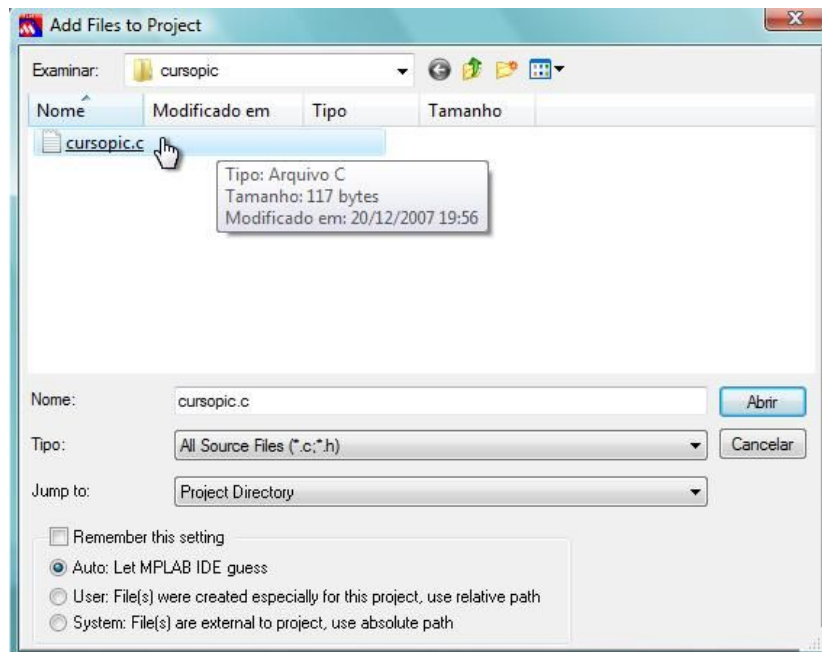


Figura 9

Neste ponto, por segurança (vai que o windows trava !?!?!?) salvamos tudo:

Project > Save Project

Agora podemos continuar.....

Para saber se esta tudo correto, pressione **F10** para compilar o programa. Caso nenhum erro tenha sido cometido, obteremos a janela da figura 10:

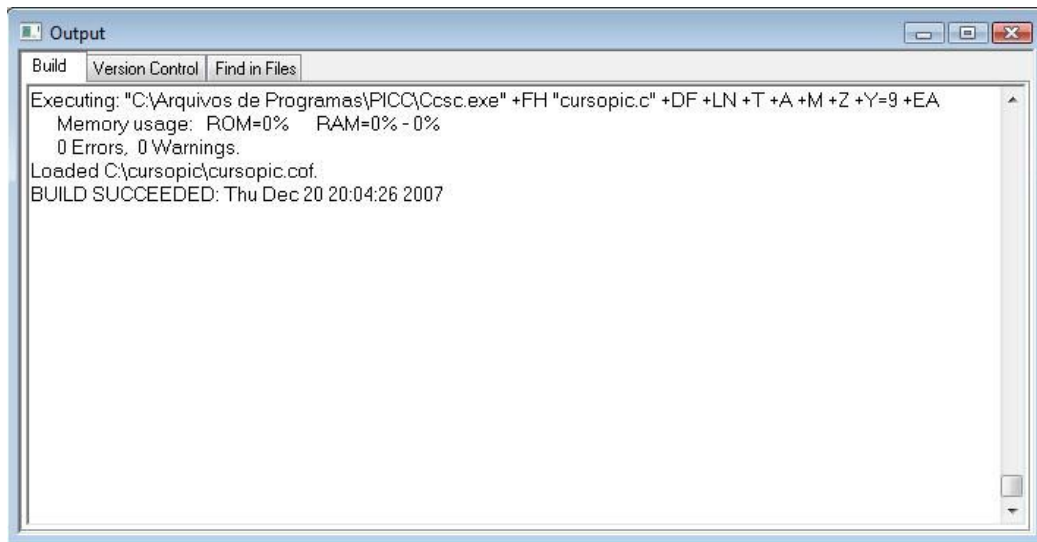


Figura 10

Em caso de erros, verifique na janela **"Output"**, na aba **"Build"** qual é o primeiro erro indicado.

Caso seja necessário, com o mouse desloque a janela ate chegar no primeiro erro indicado pelo compilador.

Dê um click-duplo exatamente sobre a linha de erro.

O Mplab vai alternar as janelas, e irá para a janela do programa fonte, com o cursor já colocado sobre a linha onde encontrou o erro.

Corrija este erro e tecla F10 novamente para compilar, repetindo tantas vezes quantas o compilador indicar erro, até não haver mais erro algum.

Importante: Algumas vezes a linha que indica estar com erro esta correta. Neste caso, verifique a linha anterior, pois a falta de um simples ' ; ' ao fim de uma linha gera erro na próxima.

IV.4 - Simulando o programa

Para podermos simular o programa no Mplab, como se estivesse gravado no pic, devemos ajustar o Mplab para trabalhar com o **Mplab Sim**.

Veja na figura 11 como configurar o exemplo para usar o simulador:

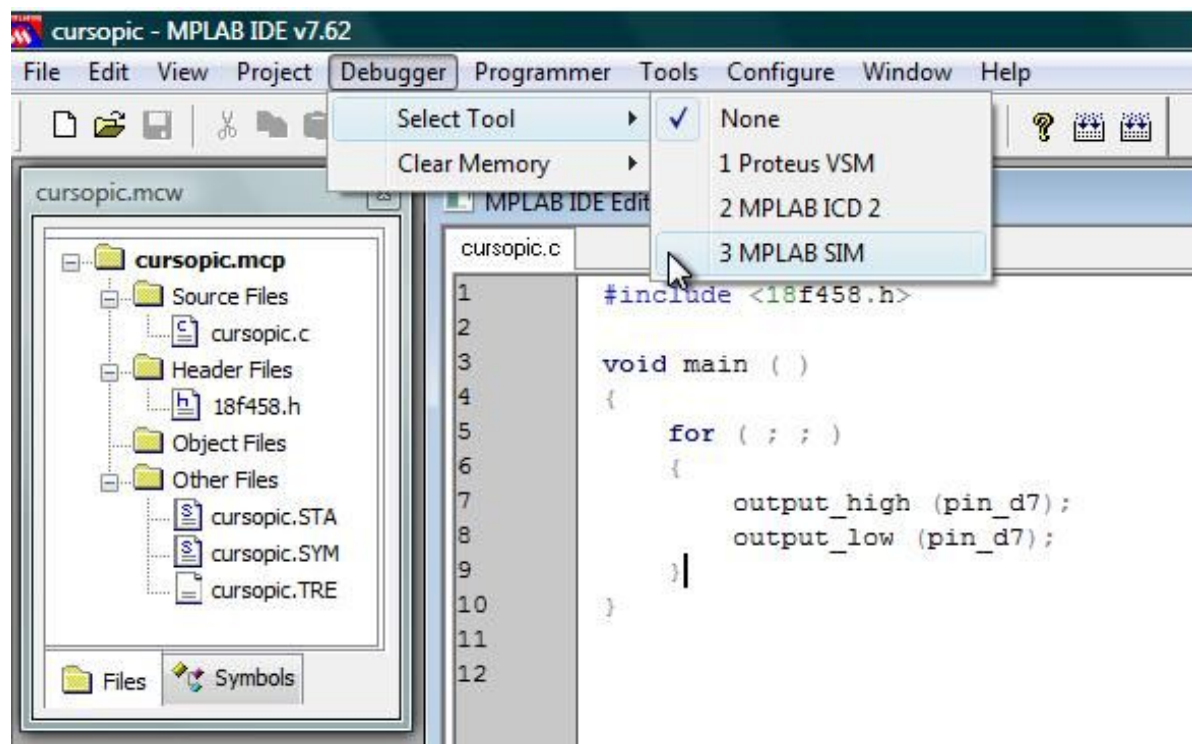


Figura 11

Neste instante observe que na barra de comandos do Mplab apareceram alguns botões novos (figura 12, NO CANTO SUPERIOR DIREITO):

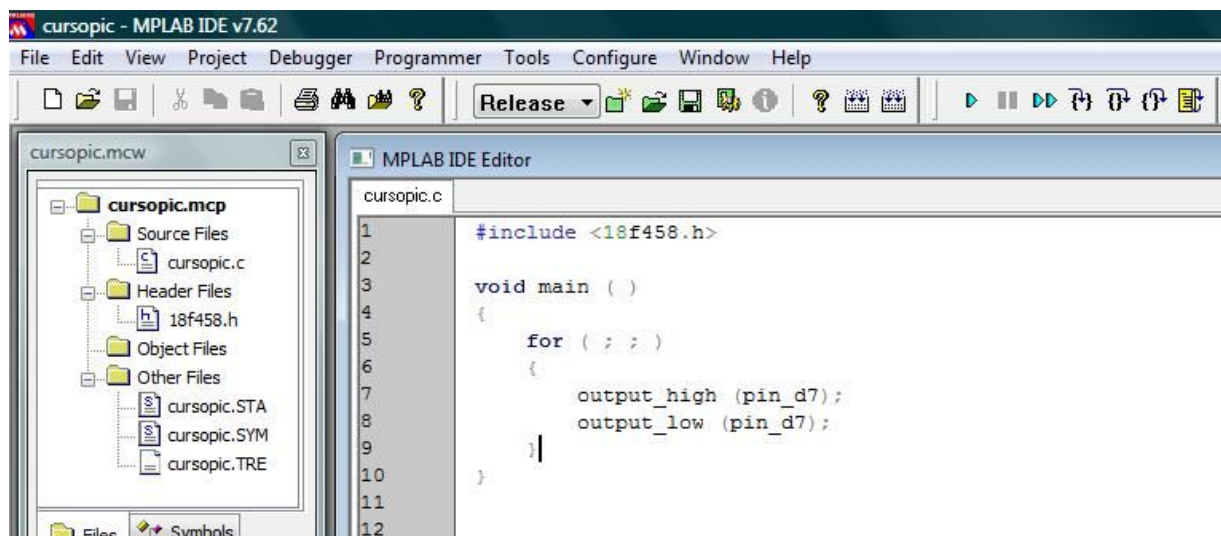


Figura 12

Os principais botões são:

	Botão RUN.	>>> Executa rapidamente sem atualização na tela
	Botão PAUSE.	>>> Para a simulação
	Botão ANIMATE.	>>> Roda a simulação passo a passo
	Botão RESET.	>>> Reinicia o processador
	Botão STEP.	>>> A cada clique executa um passo de programa e para esperando outro comando.

De um click no botão ANIMATE e veja na tela do programa fonte um cursor indicando qual linha esta sendo executada (simulada).

Depois de um click no botão PAUSE para parar a simulação e continuar o estudo.

IV.5 – Verificando o registro PORTD durante a simulação

No menu principal seleccione View > Watch, conforme figura 13 a seguir:

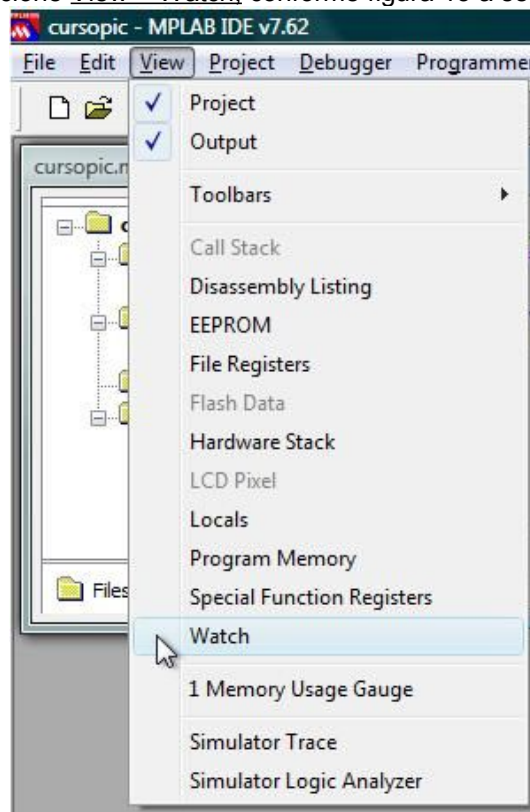


Figura 13

Você obterá a janela da figura 14:

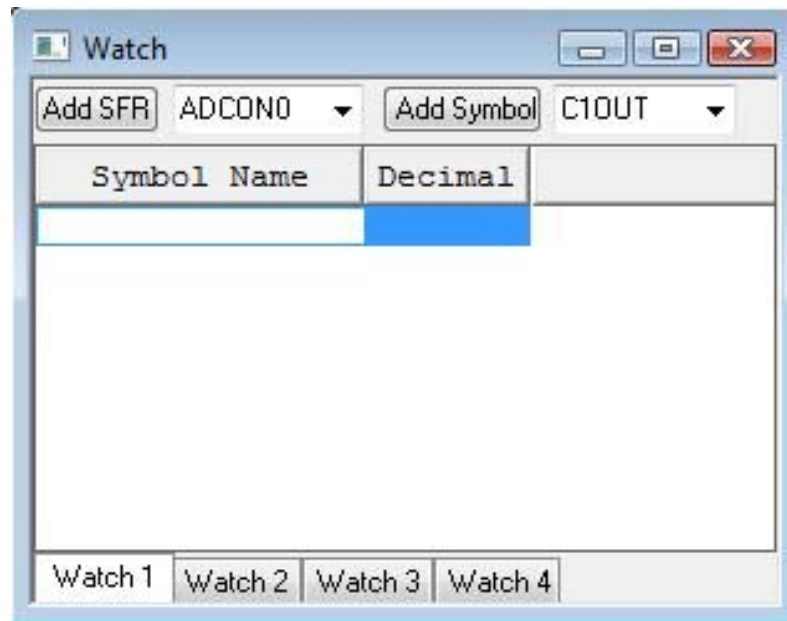


Figura 14

O botão **Add SFR**, e sua respectiva janela de seleção (que esta indicada com ADCON0), serve para visualizarmos os registros “do pic”, e o botão **Add Symbol** com sua janela de seleção (indicando CCP_1), serve para vermos as variáveis criadas no programa ou definidas no arquivo de definições (por exemplo, no arquivo 18F458.H).

Vamos selecionar o PORTD para visualização. Com o mouse selecione a caixa onde inicialmente esta ADCON0 e escolha o PORTD e depois de um clique em **Add SFR** (figura 15):

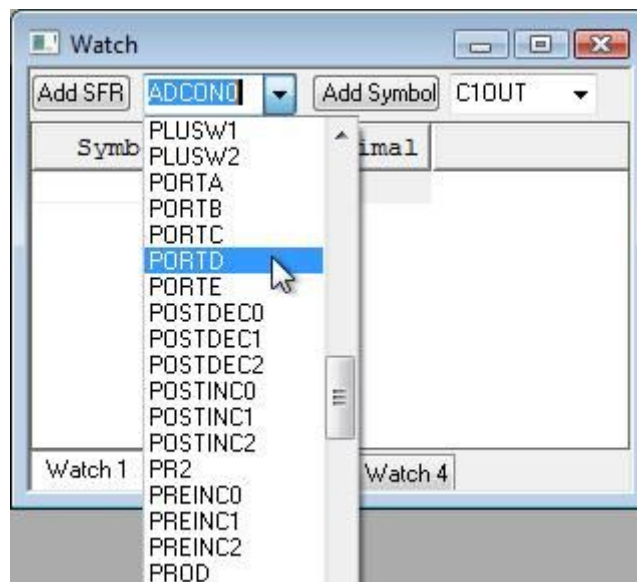


Figura 15

Suas janelas deverão estar como na figura 16 (ajuste-as com o mouse para ficarem parecidas):

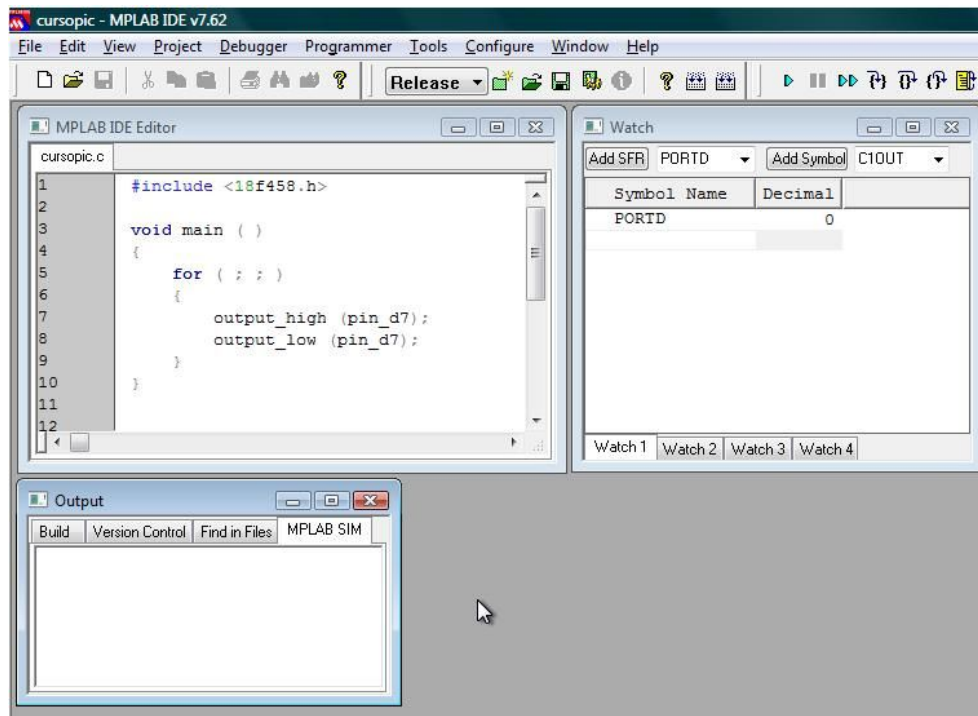


Figura 16

Para ver o PORTD sendo ajustado pelo programa, de um clique no botão ANIMATE e observe tudo ocorrendo simultaneamente:

- na tela do fonte, o cursor indica cada linha sendo executada;
- na janela Watch você verá o valor do PORTD alternando entre 0 e 128

Para visualizar o PORTD em binário ou decimal, de um clique com o botão DIREITO do mouse SOBRE a variável PORTD, e no item propriedades, selecione o formato desejado e quais opções deseja.

Praticando é que se aprende

Repita esta seqüência com outros nomes de arquivos usando todos os exemplos dados nos capítulos II e III para treinar o uso do Mplab. Modifique, escreva comandos novos, escreva comandos errados para ver as mensagens de erro, enfim, PRATIQUE !

Após ter baixado e copiado os arquivos conforme indicado no anexo I, veja as pastas:

Cap2\.... onde temos os exemplos 1, 2, 3 e regras dados no capítulo II;

Cap3\.... onde temos o programa mínimo para simulação no mplab.

V - Os Microcontroladores PIC e seus periféricos mais usuais - Famílias 16F e 18F - Teoria de funcionamento

V.1 - Introdução

Neste capítulo vamos descrever o hardware dos microcontroladores PIC, iniciando pela memória de programa e depois passando para cada periférico.

Sempre que um periférico for exclusivo de uma certa família (por exemplo, a multiplicação por hardware existe apenas na família 18F) daremos destaque a esta diferença. Quando não mencionado, considere que o periférico existe em toda a linha pic.

Usaremos como base em nosso estudo o pic 16F877 e o 18F458, suportados pela versão de demonstração do compilador C (o 16F877A tem como diferença do 16F877 apenas o comparador analógico).

Para ver as diferenças em relação aos demais modelos de pic, consulte o databook do mesmo, que pode ser encontrado no site www.microchip.com

Os pics estudados tem como principais características:

- Podem rodar até 10 MIPS (milhões de instruções por segundo) – 18F
- Até 34 pinos para uso como entrada ou saída
- Capacidade de drenar/fornecer 25 mA/pino
- Várias fontes de interrupção
- Timers programáveis de 8 e 16 bits
- módulos PWM
- Comunicação I²C ou SPI
- Comunicação Serial Assíncrona TTL
- Conversor A/D de 8 ou 10 bits de resolução com até 8 entradas
- Comparador analógico
- Watch Dog programável
- Modo Sleep para economia de energia
- Proteção contra leitura do código

V.2 – Circuito mínimo

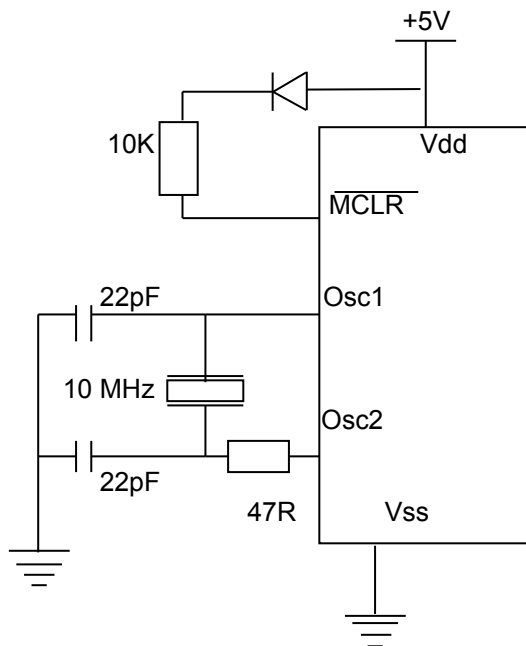


Figura 1 – Circuito mínimo com o PIC

V.3 – Memória de Programa

A memória de programa tem por objetivo armazenar as “instruções” a serem executadas pelo processador.

Esta memória tem como principal característica não ser volátil, isto é, não perde as informações quando a energia é desconectada. Em contrapartida, não pode ser usada como memória temporária de dados.

A complexidade interna do hardware acaba criando para o estudante uma certa dificuldade inicial, por isto o uso da linguagem C torna-se uma opção atraente, pois além de ser mais fácil a implementação do programa aplicativo (também conhecido por firmware), precisamos apenas conhecer o funcionamento básico de cada periférico, visto que o compilador C já possui implementadas todas as funções para tal.

O mapa de gravação da memória de programa será automaticamente preenchido pelo compilador.

16F877 e 18F877A: Temos 8.192 palavras de programação de 14 bits

18F458: Temos 16.384 palavras de programação de 16 bits

V.4 – Memória de dados

É na memória de dados que o processador realiza todas as suas funções de cálculo e armazenamento de informações transitórias.

Esta memória tem como principal característica 'ser' volátil, isto é, perde as informações quando a energia é desconectada. Em contrapartida, pode ser alterada a vontade e rapidamente (na mesma velocidade de execução do programa) pelo firmware para armazenar resultados de cálculos, estado dos pinos de entrada/saída, entre outros.

16F877 e 16F877A:

Nos microcontroladores 16F877 e 16F877A temos 368 posições de memória RAM com 8 bits de comprimento, isto é, os valores podem variar entre 0 e 255 (ver anexo II sobre sistemas de numeração)

18F458:

No microcontrolador 18F458 temos 1.536 posições de memória RAM com 8 bits de comprimento, isto é, os valores podem variar entre 0 e 255 (ver anexo II sobre sistemas de numeração)

A memória RAM é automaticamente organizada pelo compilador C, e o mesmo nos permite trabalhar com variáveis de vários tamanhos com números inteiros de 8, 16, e 32 bits, bem como com números de ponto flutuante, conforme vimos anteriormente no capítulo II.

V.5 – Memória EEPROM de dados

Esta é uma memória especial que temos no microcontrolador, pois reúne algumas características muito úteis das memórias de programa e de dados.

A principal característica da eeprom de dados, ou apenas EEPROM, esta em poder ser alterada a vontade pelo firmware para armazenar valores, e de NÃO perder estes resultados ao se desconectar a energia

Mas, como tudo tem um preço, o acesso para 'gravar' na EEPROM não é feito de forma rápida como na RAM (apenas a leitura da EEPROM pode ser feita rapidamente,) podendo a escrita levar vários milissegundos.

Assim sendo, os usos mais indicados para a EEPROM são:

- Gravação de parâmetros de configuração
- Senhas
- Dados de contadores, ...

Tanto no 16F877, 16F877A e no 18F458 temos 256 posições de EEPROM com 8 bits de comprimento, e como na memória RAM, os valores podem variar entre 0 e 255.

A rotina de acesso a memória EEPROM é automaticamente organizada pelo compilador C, onde devemos apenas nos lembrar de que a cada escrita o processador vai esperar um tempinho até o fim da mesma.

V.6 – Circuito de Reset e Clock

Conforme vimos na figura 1, para funcionar corretamente o pic precisa, além da alimentação, de mais 2 circuitos auxiliares:

- Circuito de reset
- Circuito de clock

Circuito de Reset: O circuito de reset tem por objetivo gerar no pino MCLR\ um sinal ascendente e rápido, de 0 a 5Vcc.



Existem várias configurações possíveis para esta ligação, até mesmo um fio direto do Vcc para o pino MCLR\.

Estamos usando o resistor e o diodo para permitir a fácil conexão de gravadores in-circuit ou debuggers à linha de reset.

Circuito de clock: A CPU dos microcontroladores PIC usa 4 pulsos de clock para gerar um ciclo de máquina interno.

Desta forma, com cristal de 20 MHz (valor máximo típico), o clock interno será de 5 MHz, e teremos ciclo de máquina de 200 ns.

18F458:

Para a família 18F, temos a configuração HS-PLL, onde a frequência do cristal terá seu valor quadruplicado internamente. Assim sendo, com cristal de 10 MHz, o clock interno será de 40 MHz (limite operacional da linha 18F), e teremos internamente um sinal de 10 MHz, ou um ciclo de máquina de 100 ns. (Tcy = 100 ns).

V.7 – Multiplicação 8 bits x 8 bits por hardware

Apenas na linha 18F

O pic 18F458 possui em seu circuito interno um multiplicador por hardware, isto é, podemos realizar a multiplicação de duas variáveis de 8 bits em apenas 1 Tcy (1 ciclo de máquina).

Esta característica permite o uso de complexas operações matemáticas com uma velocidade considerável.

Apenas para ilustração, uma rotina de multiplicação escrita sem usar esta característica consome aproximadamente 69 Tcy, ou seja, é 69 vezes mais lenta, além de ocupar 13 posições de memória de programa.

O interessante está em que ao se usar a linguagem C, o compilador já otimiza o programa final, usando onde for possível a multiplicação por hardware.

V.8 – Interrupções

Neste tópico veremos apenas uma breve teoria sobre o que são interrupções e seu mecanismo de funcionamento.

Nos exemplos práticos teremos mais detalhes sobre as interrupções de alguns periféricos.

A interrupção

Uma interrupção nada mais é que uma “breve parada” na atividade atual para executar “outra atividade”, onde ao final desta o processador retornará a atividade anterior. No caso das interrupções, esta ‘parada’ é solicitada por um evento externo ou interno ao processador, de forma assíncrona, isto é, não sabemos quando vai ocorrer.

Diferente de rotinas que são chamadas pelo programa conforme desejado, onde neste caso sabemos o ponto de ‘parada’.

Podemos imaginar uma interrupção como o simples ato de pararmos uma atividade, como ler ou assistir televisão, para atender a campainha que tocou (não sabemos de antemão o instante em que a campainha vai tocar).

De uma maneira bem simples, podemos considerar que após atendermos a esta “interrupção”, voltaremos ao que estávamos fazendo (não vamos levar em conta a incrível capacidade humana de abandonar uma atividade para se dedicar apenas a outra. No microcontrolador não funciona desta maneira).

Quantidade de interrupções

16F877 e 16F877A:

O pic 16F877 possui 14 fontes de interrupção e o 16F877A possui 15.

18F458:

O pic 18F458 possui 21 fontes de interrupção, e com exceção da interrupção externa INT0, todas podem ter alta ou baixa prioridade, e ainda podemos não utilizar esta característica, quando as interrupções terão todas a mesma prioridade (compatível com linha 16XXX).

Mascaramento

Este termo indica a capacidade de se habilitar ou desabilitar as interrupções.

Na linha PIC todas as interrupções podem ser a qualquer momento habilitadas ou desabilitadas pelo software.

Se uma interrupção estiver desabilitada, o seu hardware correspondente poderá até ajustar o bit que sinaliza o evento, mas o processo de interrupção não ocorrerá.

Prioridade

16F877 e 16F877A:

A linha 16FXXX possui APENAS 1 nível de prioridade para as interrupções.

18F458:

A linha 18FXXX possui 2 níveis de prioridade para as interrupções: alta ou baixa.

Resumidamente podemos dizer que uma interrupção qualquer sempre poderá interromper o programa, se estiver habilitada.

18F458:

Mas, se esta interrupção for de baixa prioridade, também poderá ser interrompida, com a chegada de um pedido de alta prioridade.

Já uma interrupção de alta prioridade nunca será interrompida, nem mesmo por outra interrupção de alta prioridade.

V.8.1 -Trabalhando com interrupções de alta e baixa prioridade.

Apenas na linha 18F

Até o presente momento vimos o tratamento de interrupções da mesma forma que ocorre na linha 16FXXX, isto é, apenas 1 interrupção ativa por vez.

Veremos agora como se ajusta uma ou mais interrupções para serem consideradas de alta prioridade, e assim poderem “interromper” o atendimento de uma interrupção de baixa prioridade.

Para definir uma interrupção como sendo de alta prioridade, devemos na declaração da mesma indicar com o parâmetro HIGH:

Exemplo: interrupção do timer 0 de alta prioridade:

```
#INT_timer0 HIGH
void Relogio()
{
    // rotinas da interrupção do timer 0
}
```

V.9 – Fusíveis de configuração

Os fusíveis de configuração tem por objetivo ajustar o funcionamento do chip, como tipo de oscilador, proteção de código, uso do watch dog, entre vários outros.

Como alguns pics tem muitos fusíveis, vamos detalhar apenas os mais importantes.

Nos programas de exemplo a linha de fusíveis sempre estará configurada para o mais usual.

Caso o leitor deseje se aprofundar no assunto consulte o databook do pic na seção “Configuration Words”.

Ajustes mais importantes

Oscilator type: Indica com que tipo de oscilador estaremos trabalhando, como, por exemplo, RC, cristal de baixa velocidade, de alta velocidade, cristal com o PLL interno que quadruplica o clock, ...

Brown Out: Liga ou desliga a capacidade do chip entrar em reset caso a tensão caia abaixo do valor programado.

Watch Dog: Liga ou desliga o monitor interno, que serve para resetar o processador em caso de eventual travamento do software.

Code protect: Permite proteger todo o código ou parte do mesmo.

As funções do watch dog, brown out e outros serão vistas em tópicos dedicados mais a frente.

Um ajuste típico dos fusíveis do pic 16F877 é o seguinte:

- Todos os ajustes em apenas 1 linha

#fuses PROTECT, HS, CPD, NOLVP, NOWDT, NOBROWNOUT, PUT

- Cada ajuste em uma linha (pode facilitar seu entendimento)

```
#fuses PROTECT          //
#fuses HS                //
#fuses CPD               //
#fuses NOLVP             // gravacao em baixa tensao desab.
#fuses NOWDT            //
#fuses NOBROWNOUT       //
#fuses PUT               // power up timer habilitado
```

V.10 – O port A e suas funções especiais

Neste tópico estudaremos o port A, suas funções de entrada e saída digital e sua multiplexação com o conversor de analógico para digital (A/D).

O uso do conversor A/D será estudado em um capítulo a parte.

Pinos presentes no port A:

- RA0 / AN0 - Pino de entrada ou saída digital.
Também pode ser a entrada analógica '0'
- RA1 / AN1 - Pino de entrada ou saída digital.
Também pode ser a entrada analógica '1'
- RA2 / AN2 - Pino de entrada ou saída digital.
Também pode ser a entrada analógica '2'
- RA3 / AN3 - Pino de entrada ou saída digital.
Também pode ser a entrada analógica '3'
- RA4 / T0CKi - Pino de entrada ou saída (saída OPEN-DRAIN) digital.
Também pode ser a entrada do timer 0 para sinal externo
- RA5 / AN4 - Pino de entrada ou saída digital.
Também pode ser a entrada analógica '4'

Para uso do port A como entrada e saída digital, temos funções específicas em 'C', que permitem ler e escrever no port A inteiro ou em bits individuais.

V.10.1 – Algumas funções de acesso ao portA digital

```
Escrever 1 em um pino:    output_high (pin_a3);
Escrever 0 em um pino:   output_low (pin_a3);
Escrever no portA inteiro: output_A (valor_desejado_8_bits);
```

Ler o estado de 1 pino: VarBit = input (pin_a2);
 Ler o portA inteiro (8 bits): Var8 = input_A ();

Importante: Todas as funções acima servem para os ports B, C, D e E.
 Para uma referência completa ver a parte VI – A linguagem C e os pic's

V.11 – O port B e suas funções especiais

Neste tópico estudaremos o port B, suas funções de entrada e saída digital e sua multiplexação com as interrupções externas.

Pinos presentes no port B:

RB0 / INT0 -	Pino de entrada ou saída digital. Também pode ser a entrada de interrupção externa '0'
RB1 / INT1 -	Pino de entrada ou saída digital. Apenas na linha 18F Também pode ser a entrada de interrupção externa '1'
RB2 / INT2 / CanTx -	Pino de entrada ou saída digital. Apenas na linha 18F Também pode ser a entrada de interrupção externa '2' ou CAN Tx
RB3 / CanRX -	Pino de entrada ou saída digital. Apenas na linha 18F Também pode ser usado para o CAN Rx
RB4 -	Pino de entrada ou saída digital.
RB5 -	Pino de entrada ou saída digital.
RB6 / PGC -	Pino de entrada ou saída digital. Usado na gravação.
RB7 /PGD -	Pino de entrada ou saída digital. Usado na gravação.

Assim como no portA, para uso do port B como entrada e saída digital, temos funções específicas em 'C', que permitem ler e escrever no port B inteiro ou em bits individuais.

Para uma referência completa ver a parte VI – A linguagem C e os pic's

V.12 – O port C e suas funções especiais

Neste tópico estudaremos o port C, suas funções de entrada e saída digital e sua multiplexação com o timer 1, PWM, comunicação I²C, SPI e serial assíncrona.

Pinos presentes no port C:

RC0 / T1CKi	-	Pino de entrada ou saída digital. Entrada de clock externo para o timer 1
RC1 /	-	Pino de entrada ou saída digital.
RC2 / CCP1	-	Pino de entrada ou saída digital. Saída do pwm
RC3 / Sck/Scl	-	Pino de entrada ou saída digital. Saída de clock Spi ou I ² C
RC4 / Sdi/Sda	-	Pino de entrada ou saída digital. Entrada de dado Spi ou entrada/saída I ² C
RC5 / Sdo	-	Pino de entrada ou saída digital. saída de dado Spi
RC6 / Txd	-	Pino de entrada ou saída digital. saída serial assíncrona
RC7 / Rxd	-	Pino de entrada ou saída digital. Entrada serial assíncrona

V.13 – Os ports D e E com suas funções especiais.

Neste tópico estudaremos os ports D e E, suas características de entrada e saída digital e a multiplexação do port E com o canal analógico.

Pinos presentes no port D:

RD0	-	Pino de entrada ou saída digital.
RD1	-	Pino de entrada ou saída digital.
RD2	-	Pino de entrada ou saída digital.
RD3	-	Pino de entrada ou saída digital.
RD4	-	Pino de entrada ou saída digital.
RD5	-	Pino de entrada ou saída digital.
RD6	-	Pino de entrada ou saída digital.
RD7	-	Pino de entrada ou saída digital.

Pinos presentes no port E:

RE0 / AN5	-	Pino de entrada ou saída digital. Também pode ser a entrada analógica '5'
RE1 / AN6	-	Pino de entrada ou saída digital. Também pode ser a entrada analógica '6'
RE2 / AN7	-	Pino de entrada ou saída digital. Também pode ser a entrada analógica '7'

V.14 – Interrupções externas

Um dos melhores periféricos presentes em qualquer microcontrolador é formado pelas chamadas “entradas de interrupção externa”.

Através de 1 ou mais pinos (1 na família 16F – 3 na família 18F), o processador pode receber do mundo externo um sinal indicando a ocorrência de um evento.

Para comparação, imagine que sua campainha esta quebrada. Teoricamente você deveria ficar a cada instante dando uma olhada na porta, para ver se tem alguém. Este processo, conhecido por varredura, é também muito utilizado, dependendo da aplicação ou das limitações do chip em uso.

Com o uso de interrupções, o seu programa não precisará de rotinas de varredura. Quando um dos sinais de interrupção externa for ativo, o processador para o programa principal e vai atender a rotina de interrupção, voltando depois ao ponto onde estava.

16F877 e 16F877A: Temos 1 pino com capacidade de interrupção externa:

- RB0 / INT – É a chamada interrupção externa

18F458: Temos 3 pinos com capacidade de interrupção externa:

- RB0 / INT0 – É a chamada interrupção externa 0
- RB1 / INT1 – É a chamada interrupção externa 1
- RB2 / INT2 – É a chamada interrupção externa 2

Cada uma das interrupções pode ser individualmente habilitada ou desabilitada a qualquer momento, permitindo total flexibilidade do nosso programa.

V.14.1 – Funções para controle das interrupções externas

```
Enable_interrupts ( int_ext0 );           // para habilitar
Disable_interrupts (int_ext2 );          // para desabilitar

Ext_int_edge ( 0, L_TO_H );              // para ajustar int0 na subida do sinal
```

V.15 – Timer 0

Em microcontroladores, os timers são de grande utilidade pois podem realizar contagem de eventos externos ou ainda, usando o clock da máquina como referência, criar bases de tempo precisas, como por exemplo, contar de 2 ms em 2 ms.

Tem ainda a capacidade de pedir interrupções ao fim de certos eventos, como, por exemplo no 'overflow' ou estouro de contagem. Temos então um hardware muito versátil, que alia a contagem de tempos ou eventos com pedido de interrupções, liberando o nosso programa para realizar outras funções.

O timer 0 tem as seguintes características comuns as duas famílias (16F e 18F):

- Registro de contagem de 8 bits.
- Pode ser lido ou alterado a qualquer momento
- Divisor de frequência (prescaler) ajustável
- Sinal interno (pino RA4 livre) ou externo (pino RA4 como entrada)
(Sinal externo é o padrão após o reset)
- Gera pedido de interrupção no 'overflow'
- Sensível a borda do sinal externo na subida ou descida

18F458:

- O registro de contagem pode ser configurado para de 8 bits ou 16 bits
(ajuste realizado por software)

Quando usamos sinal interno o timer 0 incrementa seu valor a cada ciclo de máquina (Tcy), sem considerar o prescaler.

O prescaler pode dividir os pulsos de entrada (sinal externo ou interno) por 1 (prescaler desabilitado), 2, 4, 8, 16, 32, 64, 128 ou 256.

16F877 e 16F877A:

Na linha 16F o prescaler do timer 0 É COMPARTILHADO com o Watch Dog (ver ítem V.22)

V.15.1 – Funções para controle do timer 0

```
Setup_timer_0 ( fonte_de_sinal , prescaler );  
Set_timer0 ( valor_inicial_de_contagem );  
X = get_timer0 ( );
```

V.16 – Timer 1

O timer 1 tem as seguintes características:

- Timer de 16 bits
- Pode ser lido ou alterado a qualquer momento
- Divisor de frequência (prescaler) ajustável
- Sinal interno (pino RC0 livre) ou externo (pino RC0 como entrada)
- Gera pedido de interrupção no 'overflow'
- Sinal externo ativo na borda de subida
- Pode usar cristal próprio (até 200 KHz) nos pinos RC0 e RC1

Quando usamos sinal interno o timer 1 incrementa seu valor a cada ciclo de máquina (Tcy), sem considerar o prescaler.

O prescaler pode dividir os pulsos de entrada (sinal externo ou interno) por 1 (prescaler desabilitado), 2, 4 ou 8.

Assim como para as interrupções, diferente do assembler, onde o programador deve se preocupar com o controle e verificação dos bits de controle, disposição do código e outros, quando programamos em 'C' este trabalho não existe.

Toda a organização de memória, ajuste de hardware e outros é automaticamente realizada pelo compilador C.

V.16.1 – Funções para controle do timer 1

```
Setup_timer_1 ( fonte_de_sinal 'ORed' prescaler );  
Set_timer1 ( valor_inicial_de_contagem );  
X = get_timer1 ( );
```

V.17 – Timer 2

O timer 2 tem as seguintes características:

- Timer de 8 bits
- Registro de comparação de 8 bits
- Pode ser lido ou alterado a qualquer momento
- Divisores de frequência (prescaler e postscaler) ajustáveis
- Somente sinal interno
- Gera pedido de interrupção quando a contagem atinge o valor do registro de comparação, e recomeça do 0 neste instante.
- Gera a base de tempo do módulo PWM.

O prescaler pode dividir os pulsos de contagem por 1, 4 ou 16.

O postscaler pode dividir os pulsos na saída do timer por 1, 2, 3, 4, 5, ..., 14, 15 e 16, aumentando a base de tempo nos pedidos de interrupção.

A saída do timer 2, *antes do postscaler*, pode gerar a base de tempo para o módulo PWM.

Assim como para as interrupções, diferente do assembler, onde o programador deve se preocupar com o controle e verificação dos bits de controle, disposição do código e outros, quando programamos em 'C' este trabalho não existe.

V.17.1 – Funções para controle do timer 2

```
Setup_timer_2 ( modo_e_prescaler , período , postscaler );  
Set_timer2 ( valor inicial de contagem );  
X = get_timer2 ( );
```

V.18 – Timer 3- **Apenas na linha 18F**

O timer 3 do pic 18F458 funciona de forma idêntica ao timer 1.

A única diferença está no fato de que o timer 3 NÃO tem pinos para entrada de sinal ou para cristal externo.

Neste caso o timer 3 pode usar a mesma base de tempo externa do timer 1 (pino RC0 ou cristal em RC0 e RC1) ou rodar de forma autônoma pelo clock interno, Tcy.

Apenas as funções de acesso devem endereçar este timer em particular.

V.19 – O conversor A/D

Neste tópico estudaremos o conversor de analógico para digital, mais conhecido por conversor A/D ou apenas A/D.

O objetivo do conversor A/D é converter um sinal analógico, em nosso exemplo, de 0 a 5 V, em equivalentes digitais.

Como o conversor pode ser ajustado para conversão de 8 ou 10 bits, vejamos os dois casos:

A) Conversão com resolução de 8 bits

Neste caso, com 8 bits temos $2^8 = 256$ possibilidades, o que dá aproximadamente 20 mV por conversão ($V_{ref} = 5V$). Assim sendo, poderíamos fazer a seguinte tabela:

Valor da Entrada	Valor Binário de 8 bits
0,000 V	00000000
0,020 V	00000001
0,040 V	00000010
...	...
4,980 V	11111110
5,000 V	11111111

Para usarmos o conversor A/D com resolução de 8 bits devemos colocar o resultado em uma variável de 8 bits.

B) Conversão com resolução de 10 bits

Neste caso, com 10 bits temos $2^{10} = 1.024$ possibilidades, o que dá aproximadamente 5 mV por conversão ($V_{ref} = 5V$). Assim sendo, poderíamos fazer a seguinte tabela:

Valor da Entrada	Valor Binário de 10 bits (usar variável de 16 bits)
0,000 V	0000000000
0,005 V	0000000001
0,010 V	0000000010
...	...
4,995 V	1111111110
5,000 V	1111111111

Para usarmos o conversor A/D com resolução de 10 bits devemos colocar o resultado em uma variável de 16 bits (onde os 6 bits mais significativos serão zerados).

V.19.1 – Funções para controle do conversor analógico/digital

```
Setup_Adc_Ports ( seleção_dos_pinos_analogicos ); // (*)
Setup_Adc ( modo_de_funcionamento );
Set_Adc_channel ( qual_canal_vai_converter );
X = read_adc ( );
```

Conforme a resolução desejada usar uma das diretivas a seguir:

```
#device adc=8      >>>> para que o compilador ajuste o conversor para
                    resolução de 8 bits
```

```
#device adc=10     >>>> para que o compilador ajuste o conversor para
                    resolução de 10 bits
```

(*) Consultar o arquivo “.H “ do pic utilizado para ver quais as configurações possíveis para os pinos analógicos.

MUITO IMPORTANTE

Após o reset o hardware do conversor A/D é automaticamente configurado para o modo analógico, e os pinos que servem para entrada analógica já estão configurados para tal.

Assim sendo, mesmo que não desejemos usar o conversor analógico, devemos configura-lo.

Verifique que em todos os exemplos práticos temos no início da função MAIN() o comando:

```
Setup_adc_ports ( 'parametro' )
```

Onde o “parâmetro” depende de nossa configuração.

V.20 – A comunicação serial Assíncrona

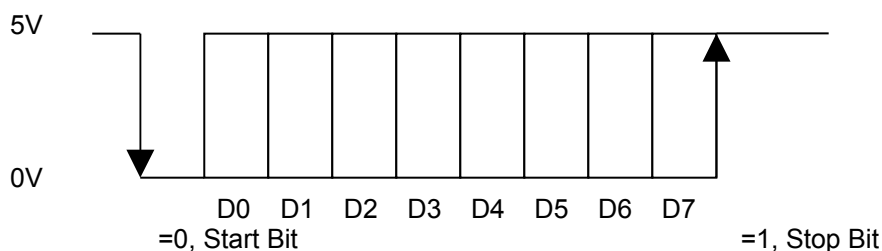
Estudaremos aqui qual o princípio de funcionamento da comunicação serial assíncrona.

Este tipo de comunicação é muito utilizada quando queremos comunicar nosso sistema baseado em microprocessador com outros equipamentos, mais usualmente um computador tipo PC.

As principais características desta comunicação são:

- Ligação ponto a ponto;
- Pode ser realizada com apenas 2 fios de sinal e um terra de referência;
- Com conversores apropriados podemos ampliar o alcance;
- Enviamos e recebemos byte a byte;
- Tratamento simples.

Formato mais usual dos dados (considerando o PIC, nível TTL):



Transmissão:

Para transmitir, o transmissor interno do Pic inicia a comunicação descendo (=0) a linha de dados, pino TX, por um tempo dT , depois vai colocando no pino TX, a intervalos “ dT ”, cada um dos bits do byte a ser enviado, iniciando pelo menos significativo (D0), e depois do bit D7, leva o pino de volta para o nível 1, encerrando o envio do byte.

Recepção:

Quando o receptor interno do Pic ‘sente’ a descida do sinal no pino RX, inicia um processo de varredura automática neste pino, e a cada intervalo fixo copia o nível do pino (0 ou 1) para um registro interno. Após receber o bit D7, espera o Stop Bit, que deve ser nível 1. Em caso afirmativo, salva o byte recebido no registro RCREG e sinaliza um pedido de interrupção.

Tempo de cada bit:

Cada bit terá a duração equivalente a $1 / \text{Taxa de transmissão}$.

Para uma comunicação em 9.600 bits por segundo, cada bit terá a duração de aproximadamente 104 μs .

Conversores de tensão:

Como a maioria dos equipamentos (PC's e outros) usa níveis baseados no padrão RS-232, devemos no caso do pic utilizar um chip conversor de tensão, em nosso caso o Max232 ou equivalente.

Este chip, alimentado apenas com +5V, pode gerar internamente as tensões de +9V e -9V necessárias para atender esta especificação.

Para mais detalhes ver o esquema do exemplo de comunicação.

Protocolos de troca de dados:

Veremos nos exemplos de recepção e transmissão um exemplo bem simples de comunicação entre o Pic e um microcomputador PC, com todos os detalhes das rotinas de interrupção e tratamento dos dados.

V.20.1 – Funções para controle da comunicação serial

Devemos antes do inicio das rotinas, usar as seguintes definições:

```
#use delay (.....)           para indicarmos a velocidade da cpu
#use Rs232 (.....)           para indicarmos os pinos, velocidade, ....
```

e no programa devemos usar estas funções:

```
x = Getchar ( );              // para ler um dado recebido pelo serial
putchar ( dado );             // para enviar um dado pelo serial
```

Maiores detalhes de todas as funções de ajuste serão vistos no capítulo VI

V.21 – Tipos de osciladores

Existem várias possibilidades para as fontes de clock para o pic:

Com cristal ou ressonador (necessidade de bases de tempo mais precisas):

LP	>	Cristal de baixa potência (até 200 KHz)
XT	>	Cristal ou ressonador normal (até 4.0 MHz)
HS	>	Cristal de alta velocidade (acima de 4.0 Mhz, até 20 MHz)

16F877 e 16F877A:

> Nos modos acima aceita clock fornecido por fonte externa

18F458:

HS + PLL	>	Cristal de alta velocidade com PLL para quadruplicar o clock, que resultará em clock equivalente a 40 MHz (máximo)
----------	---	--

Com Resistores e capacitores (Para aplicações que não necessitam de clock de precisão)

RC	>	Rede Resistor/Capacitor externa com saída do 'clock/4' em Osc2
----	---	--

18F458:

RCIO > Rede Resistor/Capacitor externa com mais um pino de entrada e saída (o pino osc2 Torna-se RA6)

Com clock externo

18F458:

EC > Clock Externo com saída do 'clock/4' em Osc2
ECIO > Clock externo com mais um pino de entrada e saída (o pino osc2 Torna-se RA6)

Ciclo de máquina:

Em todos os nossos exemplos o modo usado será o HS, com cristal de 10 MHz, e Tcy = 400 ns, onde Tcy é o tempo de cada ciclo de máquina, calculado assim:

$$T_{cy} = \frac{4}{\text{clock em Hertz}}$$

V.22 – O Watch Dog

O watch dog é um timer interno que roda livremente, independente do clock do sistema, sem qualquer componente externo.

Neste caso, continua funcionando mesmo durante o modo Power Down, ou Sleep.

Princípio de funcionamento:

Se o watch dog estiver habilitado, de tempos em tempos o nosso programa deverá “dar um clear” no mesmo, caso contrario o watch dog provocará um reset no sistema.

Seu objetivo primário é o de reinicializar o programa em caso de travamento.

Devemos em nosso programa colocar a instrução `Restart_Wdt()` naqueles pontos onde o programa sempre deverá passar em caso de normalidade.

Como o watch dog é diferente nas duas famílias, vamos estudar separadamente cada caso:

V.22.1 - O Watch Dog da família 16F

IMPORTANTE: o Watch Dog da linha 16F compartilha o prescaler com o timer 0, de forma que se o prescaler estiver DIRECIONADO para o timer 0, o watch dog NÃO terá o clock dividido, rodando no seu tempo nominal, e se o prescaler estiver direcionado para o watch dog, o timer 0 NÃO TERA DIVISÃO de frequência.

Ajuste do Watch Dog:

O watch dog da família 16F tem ajuste APENAS nos fusíveis, e estes ajustes são:

- Ligar:

Quando ligado via fusível na programação (WDT), SEMPRE ficará ligado, não podendo ser desligado por software. (acrescentar a seguinte linha de configuração)

#fuses WDT

- Desligar:

Quando estiver desligado nos fusíveis (NOWDT), será como se não existisse. (acrescentar a seguinte linha de configuração)

#fuses NOWDT

- Tempo de reset:

O tempo nominal do watch dog é de 18 ms, e podendo ser multiplicado por 1, 2, 4, 8, 16, 32, 64 ou 128, via fusíveis (não pode ser alterado pelo programa).

Seu ajuste é feito na função de ajuste do timer 0.

V.22.1.1 – Funções para controle do watch dog da linha 16F

Setup_counter (ajuste do timer0, ajuste do prescaler)

```
Restart_wdt ( );           // para zerar o contador e reiniciar a
                           // contagem
```

V.22.2 - O Watch Dog do pic 18F458

Ajuste do Watch Dog:

O watch dog da família 18F tem ajustes tanto nos fusíveis como via programação, e estes ajustes são: ligado / desligado E ajuste do tempo de reset.

- Ligar e Desligar:

Quando ligado via fusível na programação (WDT), SEMPRE ficará ligado, não podendo ser desligado por software. (acrescentar na linha #fuses a palavra **WDT**)

Quando estiver desligado nos fusíveis (NOWDT), poderá ser ligado ou desligado via software. (acrescentar na linha #fuses a palavra **NOWDT**)

- Tempo de reset:

O tempo nominal do watch dog é de 18 ms, e pode ser multiplicado por 1, 2, 4, 8, 16, 32, 64 ou 128, via fusíveis (não pode ser alterado pelo programa)

Acrescentar na linha #Fuses uma das opções abaixo:

WDT1 WDT2 WDT4 WDT8 WDT16 WDT32 WDT64 WDT128

V.22.2.1 – Funções para controle do watch dog

```
Setup_wdt ( ligar ou desligar );           // somente no caso da opção NOWDT
```

```
Restart_wdt ( );           // para zerar o contador e reiniciar a
                           // contagem
```

V.23 – Brown-out Reset

Este circuito interno permite ajustar o pic para ser “resetado” caso a tensão de alimentação caia abaixo de um determinado valor.

Toda a configuração é feita via linha de fusíveis e NÃO pode ser alterada pelo programa durante o funcionamento do chip.

As palavras de ajuste para os fusíveis são:

16F877 e 16F877A:

NOBROWNOUT > Não usará o brown-out reset

BROWNOUT > USARÁ o brown-out reset

Na família 16F a tensão de brown out é fixa e determinada pelo parâmetro **Vbor**, tipicamente em 4.0 V .

18F458:

NOBROWNOUT > Não usará o brown-out reset (os ajustes de tensão são ignorados)

BROWNOUT > USARÁ o brown-out reset (o ajuste de tensão é necessário)

BORV20 > O reset ocorrerá em 2.0 V

BORV27 > O reset ocorrerá em 2.7 V

BORV42 > O reset ocorrerá em 4.2 V

BORV45 > O reset ocorrerá em 4.5 V

V.24 – O modo Power-Down, ou ‘Sleep’

Este é um modo especial de funcionamento no qual o processador para a execução do programa, desligando o circuito interno de clock passando para um modo de economia de energia.

Muito útil para aplicações movidas a baterias.

Consumo:

- Em modo XT com clock de 4 MHz, o consumo típico do 18F458 é de 1,7 mA
- Em modo sleep, o consumo médio é de 2 uA (0,002 mA, ou quase 900 vezes menos)

Importante: Apenas o chip passa a economizar energia. O restante do circuito permanece ligado.

Todos os periféricos que dependem do clock interno serão desligados (Ver no databook quais periféricos continuam funcionando durante o sleep)

O processador sairá do modo sleep das seguintes maneiras:

1. Reset externo no pino MCLR\
2. Estouro do timer do watch dog
3. Pedido de interrupção de alguns periféricos (Ver no databook quais periféricos podem tirar o pic do

modo sleep)

No caso de reset pelo pino MCLR, ou pelo estouro do watch dog, o processador reiniciará o programa como se fosse ligado naquele momento, pelo início da função main().

Para sabermos se o pic iniciou pelo reset ao ligar (normal), pelo reset forçado no pino ou pelo watch dog, usamos função RESTART_CAUSE(), cujos retornos possíveis são:

WDT_TIMEOUT	> Ressetou pelo watch dog
MCLR_FROM_SLEEP	> Estava em sleep e foi ressetado no pino
NORMAL_POWER_UP	> Boot normal ao ligar
BROWNOUT_RESTART	> Reset pelo brown-out (Veremos depois)

Se o pic sair do modo sleep devido ao pedido de alguma interrupção (das que podem realizar esta função), o processo é um tanto diferente:

- Apenas as interrupções habilitadas podem tirar o pic do sleep. Se uma interrupção existir em nosso programa, mas estiver desabilitada ao entrar em sleep, mesmo que seja um periférico que não dependa do clock, o pic não sairá do sleep;
- Caso a habilitação Global não esteja ativada, o pic continuará o programa da instrução seguinte a que o colocou em sleep;
- Caso a habilitação global esteja ativada, o pic executará UMA instrução após sair do sleep e irá imediatamente para a interrupção.

Como acionar o modo sleep ?

Usando a instrução `SLEEP( );`

Importante: Sempre que a instrução SLEEP () for executada, o timer do watch dog será ressetado, garantindo assim um tempo conhecido caso o watch dog venha a tirar o pic do modo sleep. Equivale a usarmos a seguinte sequência:

```
Restart_Wdt ( );
Sleep ( );
```

V.25 – Power-up Timer

Este circuito interno permite ajustarmos o hardware do reset para ‘esperar’ um tempo, logo após a energização, para que o pic seja “resetado”, visando garantir que a fonte já esteja estabilizada quando o pic começar a rodar o programa.

O tempo nominal de espera é de 72 ms.

Toda a configuração é feita via linha de fusíveis e NÃO pode ser alterada pelo programa, e as possibilidades são:

PUT	> Ligado
NOPUT	> Desligado

V.26 – Oscilator Start-up Timer

Este circuito interno acrescenta, após o tempo do power-up timer (se este estiver habilitado) um delay fixo de 1.024 Tcy, com o objetivo de garantir a estabilização do cristal.

Este modo é automático para os seguintes modos de oscilação:

XT - LP - HS - HS+PLL (apenas 18F458)

V.27 – Módulo MSSP como SPI

A comunicação SPI faz parte do módulo chamado de Master Synchronous Serial Port (MSSP), que pode funcionar no modo SPI ou I²C (nunca nos dois modos simultaneamente).

Sua principal aplicação é a comunicação entre periféricos ou entre microcontroladores entre si.

Na comunicação SPI usamos 3 linhas de comunicação, a saber:

- Serial Data Out, ou apenas SDO, no pino RC5
- Serial Data In, ou SDI, no pino RC4
- Serial Clock, ou SCK, no pino RC3

Dependendo do periférico podemos usar um outro pino qualquer do pic como seletor ou CS (Chip Selector).

A comunicação é feita de 8 em 8 bits de forma síncrona, isto é, usa o sinal de CLOCK para sincronizar toda a comunicação.

Veremos depois dois exemplos completos com um potenciômetro digital da Microchip, MCP 41010, o primeiro usando o módulo SPI interno ao pic e o segundo com funções escritas especialmente para esta finalidade, que neste caso permitem o uso de qualquer pino do pic (útil para o caso do pic não ter modulo SPI ou do mesmo já estar sendo usado em I²C).

As funções para usar o MSSP no modo SPI são:

Setup_Spi () Spi_Read () Spi_Write ()

V.28 – Módulo MSSP como I²C

A comunicação I²C, como vimos no item anterior, faz parte do módulo chamado de Master Synchronous Serial Port (MSSP), que pode funcionar no modo SPI ou I²C (nunca nos dois modos simultaneamente).

Sua principal aplicação também é a comunicação entre periféricos ou entre microcontroladores entre si.

Na comunicação I²C usamos 2 linhas de comunicação, a saber:

- Serial Data, ou SDA, no pino RC4 (Os dados entram e saem pelo mesmo pino)
- Serial Clock, ou SCL, no pino RC3

Os dispositivos I²C costumam ter internamente endereços base pré-configurados, e pinos para seleção binária de endereços individuais.

Por exemplo, um dispositivo pode ter endereços-base 0xA0 e 0xA1, e mais dois pinos de seleção, o que daria 4 opções ($2^2 = 4$ combinações).

Assim sendo, os endereços possíveis seriam:

- 0xA0 e 0xA1 - 0xA2 e 0xA3
- 0xA4 e 0xA5 - 0xA6 e 0xA7

V.29 – Módulo CCP como ‘Capture Mode’

Temos um bloco de captura, CCP1.

No modo Capture, o valor de 16 bits do Timer 1 (ou do Timer 3) será “capturado”, isto é, copiado em um registro especial, chamado CCP_1 e já declarado no arquivo .H do pic, na ocorrência de um dos seguintes eventos:

- A cada pulso de descida no pino RC2/CCP1
- A cada pulso de subida no pino RC2/CCP1
- A cada 4 pulsos de subida no pino RC2/CCP1
- A cada 16 pulsos de subida no pino RC2/CCP1

Sua principal aplicação é a medida do tempo entre dois eventos (ou a frequência). Devido a ser totalmente configurado por software, ele pode ter seu modo de funcionamento alterado a qualquer momento conforme nossa necessidade.

Vários exemplos podem ser vistos no documento: “Pic Micro CCP and ECCP Tips ‘n Tricks” e em vários outros documentos no site www.microchip.com

V.30 – Módulo CCP em ‘Compare mode’

Neste modo, o valor de 16 bits escrito por nosso programa em um registro especial, chamado CCP_1 e já declarado no arquivo .H do pic, será constantemente comparado, pelo próprio hardware interno, ao valor do Timer 1 (ou do Timer 3).

Assim que ficarem iguais, um dos eventos abaixo poderá ocorrer no pino RC2/CCP1:

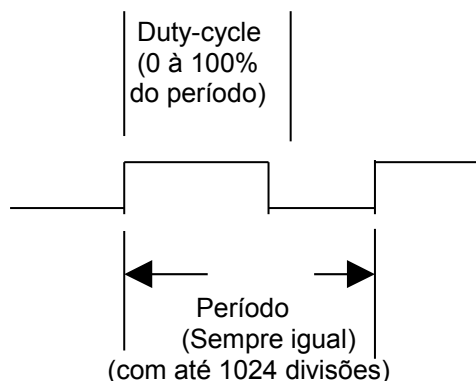
- Irá ao nível 1
- Irá ao nível 0
- Ficará inalterado
- Terá seu nível invertido

Sua principal aplicação esta na geração de uma base de tempo repetitiva, com o mínimo de intervenção do usuário, podendo ainda acionar ou não o pino RC2/CCP1.

V.31 – Módulo CCP em ‘Pwm Mode’

No modo PWM, o pino CCP1 produzirá uma onda quadrada com até 1024 divisões entre ciclos (resolução interna de até 10 bits).

Um sinal PWM tem um “período” de repetição (constante) e um tempo (variável) em que a saída fica em nível lógico 1, chamado de “duty cycle”.



Sua principal aplicação está no controle de potências, como, por exemplo, acionar uma lâmpada ou uma outra carga com energia variável de 0 a 100%, e ainda no controle de velocidade de motores DC.

Pode ainda ser usado para gerar sinais analógicos ou até mesmo sinais de áudio, com o auxílio de filtros externos.

Vários exemplos podem ser vistos no documento: "Pic Micro CCP and ECCP Tips 'n Tricks" e em vários outros documentos no site www.microchip.com

Veja a seção "Dicas de PIC" no site <http://www.newtoncbraga.com.br/index.php/dicas-de-pic>

V.31.1 – Funções para controle para o módulo CCP

```

setup_ccpX ( )
set_pwmX_duty ( )
setup_power_pwm ( )
setup_power_pwm_pins ( )
set_power_pwmX_duty ( )
set_power_pwm_override ( )

```

V.32 – Low voltage Detector (LVD)-

Apenas na linha 18F

No brown-out detector temos um hardware interno que ressetava (e mantinha ressetado) o processador, caso a tensão de alimentação estivesse abaixo de um dos 4 níveis programados.

Neste caso, o LVD não resseta o processador. Este circuito fica monitorando a tensão de alimentação e comparando com níveis pré-determinados (ou uma tensão qualquer externa, através do pino RA5/LIDEM) e quando estiver abaixo do valor programado, aciona um pedido de interrupção. Desta forma o programa pode ser desviado para uma rotina de tratamento adequada.

Como exemplo, podemos imaginar um circuito alimentado a bateria, que ao atingir um nível de tensão ligeiramente superior ao mínimo necessário para o funcionamento, realizará alguma função como, por exemplo, desligar motores, acionar alarmes, salvar dados na EEPROM interna, entre outros, antes da bateria atingir um nível crítico não operacional.

A ativação ou não da função, e os ajustes de tensão são totalmente controlados por software, pela função "setup_low_volt_detect ()"

Exemplo 1: Para acionar quando a tensão for menor que a tensão no pino LVDIN, usar:

```
setup_low_volt_detect ( LVD_LVDIN );
```

Exemplo 2: Para acionar quando a tensão for menor que 3,6V, usar:

```
setup_low_volt_detect ( LVD_36 );
```

para outros níveis de detecção, ver o arquivo **18F458.H**

V.33 – Comparador analógico (linha 16F87xA e 18F458)

O módulo comparador é composto de 2 comparadores de tensão analógicos, configuráveis via software.

As entradas do comparador são multiplexadas com os pinos de I/O (entrada e saída) a saber:

16F877A:

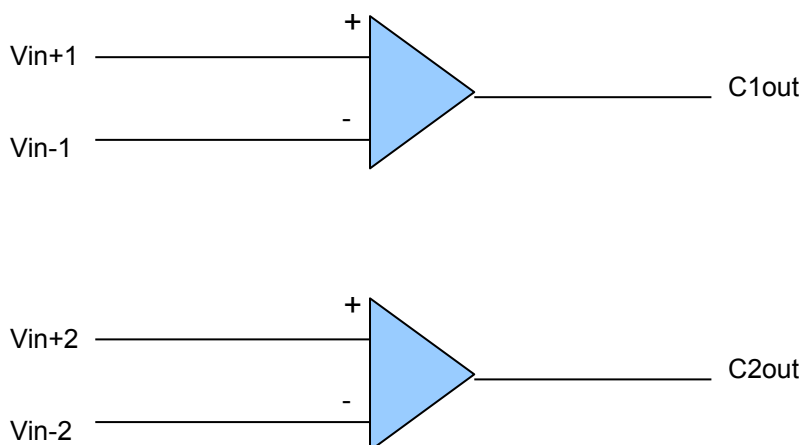
Neste caso temos uma observação especial. Na linha 16F, o pic 16F877 comum NÃO TEM comparadores internos, apenas o **16F877A**.

RA0/Vin-1 e RA3/Vin+1 para o comparador 1 e com os pinos RA1/Vin-2 e RA2/Vin+2 para o comparador 2, e as saídas C1Out e C2Out podem ser direcionadas para os pinos RA4 e RA5 respectivamente ou apenas internamente, via bits de sinalização (os comparadores podem ainda pedir interrupção se ajustados para tal).

18F458 - (e 18F448 >>> ou seja, apenas nos modelos de 40 pinos):

RD0/Vin+1 e RD1/Vin-1 para o comparador 1 e com os pinos RD2/Vin+2 e RD3/Vin-2, e as saídas C1Out e C2Out podem ser direcionadas para os pinos RE1 e RE2 respectivamente ou apenas internamente, via bits de sinalização (os comparadores podem ainda pedir interrupção se ajustados para tal).

Configuração dos comparadores:



MUITO IMPORTANTE

Após o reset, o hardware do comparador é automaticamente configurado para o modo analógico, e os pinos que servem para entrada dos comparadores já estão configurados para tal.

Assim sendo, mesmo que não desejemos usar o comparador analógico, devemos configura-lo.

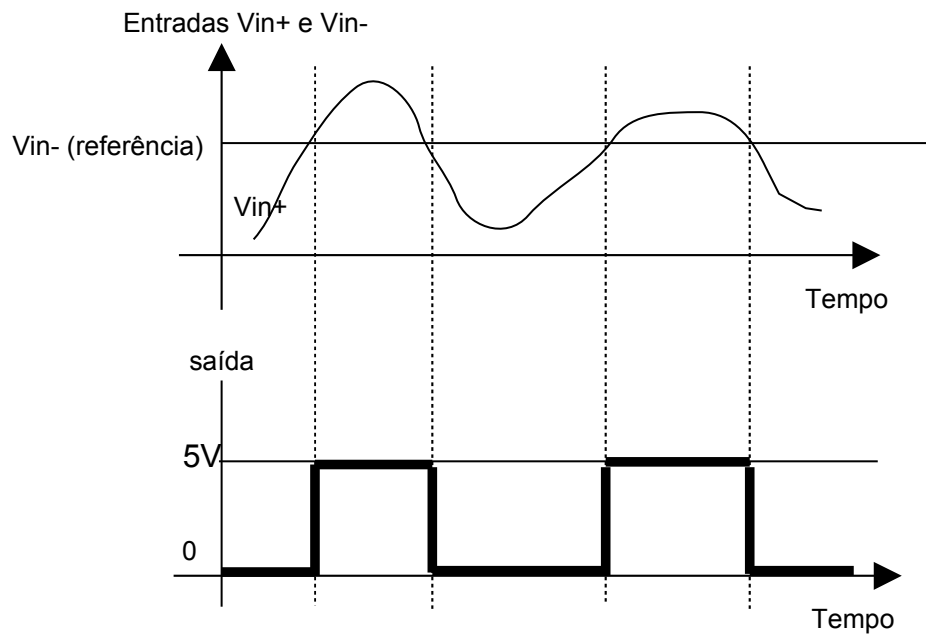
Verifique que em todos os exemplos práticos temos no início da função MAIN() o comando:

Setup_Comparators ('parametro')

Onde o "parâmetro" depende de nossa configuração.

Princípio de funcionamento:

Sempre que a tensão do pino VIn+ for superior a tensão do pino VIn- (este ajuste pode ser invertido por software), a saída estará em nível 1, caso contrario estará em nível 0 (este ajuste também pode ser invertido por software).



V.33.1 - Funções para controle do comparador:

Setup_comparator (ajuste) ;

onde “ajuste” pode ter várias opções, como:

- NC_NC_NC_NC - para desligar o comparador (devemos fazer este ajuste se não formos usar os comparadores)
- D1_D0_NC_NC_OUT_ON_E1 - Apenas o comparador 1 ligado, com a saída no pino RE1

Para ver todas as opções possíveis consulte o databook do pic em questão e seu arquivo .H

V.34 – Interfaces CAN, USB, ETHERNET,.....-

Apenas na linha 18F

A interfaces especiais estão presentes em vários modelos da linha 18F

Devido a grande complexidade técnica e a necessidade de outros pré-requisitos para seu entendimento, não estudaremos estas interface neste material.

Para visitar o site da microchip especifico sobre alguma interface, basta acessar a página inicial e após a '/' colocar o periférico desejado

Exemplo: www.microchip.com/USB

VI - A linguagem C e os PIC's

Veremos nesta parte como escrever programas em C para os PIC's, acessando registros, portas e memória, além de estudar algumas funções criadas pela CCS para facilitar o trabalho do desenvolvedor de sistemas.

Não vamos estudar todas as funções criadas, apenas as mais usadas, de forma a permitir ao estudante aprofundar-se com facilidade posterior quando do uso do software completo e sem limitações.

Para este estudo usaremos uma versão de demonstração do compilador PCW, funcional por 30 dias.

Nota importante: para manter compatibilidade com chips e usuários mais antigos da linha PIC, em algumas funções o TIMER 0 é chamado pelo seu nome antigo, RTCC.

VI.1 - Diretivas de compilação

Ver item II.3 para teoria sobre as diretivas de compilação.

VI.1.1 - #asm #endasm

Permite a inclusão de trechos em assembler dentro do programa em C.

Muito cuidado deve ser tomado com o uso esta diretiva, pois toda ação gerada em assembler é de inteira responsabilidade do usuário, e pode vir a prejudicar outras variáveis ou posições de RAM usadas pelo compilador.

Exemplo:

```
x = a;
#asm
    bsf    PORTB,3    // estas duas instruções geram um pulso
    bcf    PORTB,3    // no pino RB3.
#endasm
x += 5;
```

IMPORTANTE: Observe que embora seja um trecho em assembler o comentário esta no formato da linguagem C.

VI.1.2 - #case

Indica para o compilador ser "sensitivo" a diferenças entre letras maiúsculas e minúsculas.

Embora a linguagem C de forma padrão seja sensível ao tipo de letra, sem esta diretiva, o compilador ignora as diferenças.

Exemplo: com o #case

Teste, *teste* e *TESTE* são variáveis diferentes

Exemplo: sem o #case

Teste, *teste* e *TESTE* são a mesma variável

VI.1.3 - #define 'nome' 'seqüência'

Permite a substituição, no momento da compilação, do 'nome' pela cadeia de caracteres (string) definida em 'seqüência'.

Exemplo:

```
#define largura      4

portb = largura;      // será visto pelo compilador como portb = 4
largura = 5;          // ERRO. Não é uma variável.
```

Sua principal vantagem está em não precisarmos trocar constantes em todo o programa, bastando alterar na definição, ou ainda para facilitar a escrita de funções mais complexas.

Outras aplicações serão vistas nos exemplos.

VI.1.4 - #include <arquivo>

O arquivo indicado será adicionado ao fonte no 'momento da compilação'.
Ideal para incluir trechos pré-definidos, evitando sua digitação a cada programa novo.

Exemplo típico é o arquivo 16F877A.H que contém as definições sobre o processador usado.

Exemplo: #include < 16F877a.H >

VI.1.5 - #fuses 'opções'

Avisa ao compilador para incluir no arquivo .HEX informações sobre os fusíveis de configuração, facilitando na hora de se gravar o chip.

Algumas opções são:

```
Oscilador:      XT (cristal até 4 MHz), LP (cristal baixa potência),
                 HS (cristal alta velocidade), RC (oscilador RC), ...
WatchDog:       WDT (ligado) e NOWDT (desligado)
Código:         PROTECT (protegido) e NOPROTECT (não protegido)
Power-up:       PUT (Com power-up) e NOPUT (Sem power-up)
```

Ajuste usado nos exemplos com o 18F458

```
#fuses HS, noWRTD, noWRTB, CPD, noCPB, noWRT, noWRTC, noEBTR
#fuses NOOCSSEN, noSTVREN, NOLVP, PROTECT, noWDT, WDT128
#fuses noBROWNOUT, BORV27, PUT, nodebug, noEBTRB
```

Veja nos arquivos .H quais os fusíveis suportados pelo modelo de PIC que estiver usando.

VI.1.6 - #ifdef #endif

Usado para "compilação condicional", isto é, permite selecionar se o trecho entre as duas diretivas será incluído no momento da compilação ou não.

Exemplo:

```
#define EmModoreal
.
#ifdef EmModoreal
```

```
    delay_ms(200);
#endif
```

No exemplo, caso o `#define EmModoreal` tenha sido declarado, a chamada para a rotina de tempo será incluída no programa final. Caso contrário não será incluída.

Neste caso poderia servir para tirarmos o atraso durante os testes no simulador, sem termos de ficar apagando várias linhas.

VI.1.7 - `#INT_####` , onde `####` indica o nome da rotina

Indica para o compilador que o trecho a seguir refere-se a uma rotina de interrupção.

O compilador gerará um código padrão para entrar e sair da rotina, salvando o estado da máquina e zerando o flag de requisição de interrupção ao sair da mesma.

Importante: Para manter compatibilidade com a linha 16XXX, apenas uma interrupção poderá ser atendida de cada vez. Para permitir que uma interrupção possa ser interrompida por outra mais importante, ver o parâmetro HIGH abaixo.

Algumas das interrupções reconhecidas são:

<code>INT_EXT</code>	(RB0/Int0)
<code>INT_EXT1</code>	(RB1/Int1)
<code>INT_TIMER0</code>	(overflow do timer 0)

Para uma lista completa do nome das diretivas de interrupção das várias dezenas de chips existentes, consultar o manual da CCS e o arquivo .H do chip utilizado.

O nome da rotina é definido pelo seu programa.

Exemplo:

```
#INT_EXT    // indica que a rotina a seguir é a da interrupção. externa
void SincronismoDeRede( )
{
    .....
}
```

VI.1.7.1 – Declarando interrupções de alta prioridade

Apenas na linha 18F

Se na declaração da rotina usarmos a palavra HIGH a rotina será de alta prioridade e poderá interromper outra interrupção já em andamento.

Ex:

```
#INT_timer0 HIGH
void Relógio()
{
    // rotinas da interrupção do timer 0
}
```

VI.1.8 - `#Priority` – Prioridades por software

Permite ao programador escolher a prioridade no atendimento das interrupções habilitadas. A primeira da lista é a mais prioritária (para as que não usam o parâmetro HIGH visto acima).

Exemplo:

```
#priority    timer0, ext           // Timer 0 é mais prioritária que a interrupção externa.
```

VI.1.9 - #ROM

Permite a gravação de constantes na memória de programa ou na EEPROM de dados interna no momento da compilação.

Não influencia no programa, apenas informa ao gravador para gravar os valores no endereço indicado.

Exemplo:

Determinando os 10 primeiros bytes da eeprom de dados (início em F00000H na linha 18F):

```
#ROM int8    0xF00000 = { 0,1,2,3,4,5,6,7,8,9 }
```

VI.1.10 - #use delay (clock= '*valor do clock em Hz*')

Informa ao compilador o valor do clock da CPU em Hertz. Serve para permitir o uso de funções de tempo baseadas no clock da máquina.

Caso o watch dog seja usado, devemos informar por esta diretiva, para que o mesmo seja resetado nas rotinas de tempo.

Exemplos:

1) Sem o watch dog: `#use delay(clock=4000000)`

2) Com o watch dog: `#use delay(clock=10000000, Restart_Wdt)`

VI.1.11 - #use Fast_IO(*port*)

Informa ao compilador que o controle da direção dos ports (registro TRIS do *port* indicado) será executado pelo programa. Gera operações de entrada e saída mais rápidas.

Exemplo:

```
#use fast_io(A)           // quem deve controlar o trisa é o programa.
```

Neste caso devemos controlar os registros TRIS manualmente (ver item VI.2.7, número 8)

VI.1.12 - #use standard_IO(*port*) - *Default*

A cada acesso as portas ou pinos de I/O o compilador gerará código para o controle do tris correspondente.

Permite maior flexibilidade nos casos de bits com dupla função ou usuários com pouca experiência, mas consome um pouco mais de tempo (alguns micro ou nanosegundos) e memória de programa.

É O IDEAL PARA QUEM ESTÁ INICIANDO.

VI.1.13 - `#use rs232 (BAUD = taxa, XMIT = pinoTx, RCV = pinoRx, BITS = n )`

Ajusta os pinos indicados como **pinoRX** e **pinoTX** para funcionarem como entrada e saída serial com velocidade indicada em **taxa**, e com palavra de **n** bits.

O compilador tem incorporado rotinas que permitem a transmissão e recepção no formato assíncrono típico da comunicação serial.

Para o uso destas rotinas a diretiva acima deve ser utilizada.

Nos PIC's com USART interna, se os pinos indicados forem os pinos da USART o software usará o Hardware interno.

Embora o nome da diretiva nos de a sugestão de usar RS232, devemos lembrar que os sinais do PIC são de nível TTL.

Importante: antes do uso desta diretiva o `#use delay` é obrigatório.

Exemplo:

Recebe pelo pino RB7 e depois envia o mesmo byte, com clock de 4 MHz, pelo pino RA4 a 9600 bps e palavra de 8 bits:

```
#use delay (clock=4000000)
#use rs232 (BAUD = 9600, XMIT = pin_A4, RCV = pin_B7, BITS = 8 )

X = getchar ( );           // fica esperando um byte chegar pelo serial
                           // e então escreve na variável X
putchar ( X );             // e depois envia o mesmo de volta.
```

Observação: os nomes de pinos e outras constantes serão vistos ao final e estão no arquivo .H correspondente ao chip utilizado.

VI.1.14 - `#byte nome = endereço do byte` `#bit nome = endereço do byte . número do bit`

Permite o acesso a posições de RAM (Registros e memória geral) pela indicação de seu endereço real (lembre-se de que o gerenciamento das variáveis em C já é realizado pelo compilador).

Exemplo: Sabemos que o port D está no endereço F83H da memória RAM (ver o databook do 18F458), e se desejarmos acessar o port D como se fosse uma variável devemos usar:

```
#byte portd = 0xF83 (veja o anexo III – sistemas de numeração)
```

Vale o mesmo para variáveis representando bits:

```
#bit RD0 = portd.0 // RD0 agora é uma variável em C
```

Importante: Para tal devemos usar o sistema de acesso `fast_io( )`

VI.2 - Funções escritas para os PIC's

Neste item veremos algumas das funções escritas especialmente para a família PIC. Uma descrição completa de todas as funções pode ser encontrada no manual do usuário da CCS.

VI.2.1 - Funções matemáticas

Para o uso de funções matemáticas especiais, como raiz quadrada, seno, exponencial, entre outras, é obrigatório incluir a biblioteca `math.h` (seja curioso e de uma olhada nesta biblioteca), da seguinte forma:

```
#include < math.h >
```

VI.2.2 - Funções de manipulação de bit

1) `bit_clear (variável , bit )`

Zera o *bit* indicado na *variável* indicada. Como podemos ter variáveis de 8, 16 ou 32 bits, o número do bit pode ir de 0~7 ou de 0~15 ou de 0~31.

Exemplo:

```
int8 x;
x = 0b10001000
bit_clear(x,7); // x = 00001000
```

2) `bit_set (variável , bit )`

Seta o *bit* indicado na *variável* indicada. Como podemos ter variáveis de 8, 16 ou 32 bits, o número do bit pode ir de 0~7 ou de 0~15 ou de 0~31.

Exemplo:

```
int8 x;
x = 0b10001000
bit_set(x,0); // x = 10001001
```

3) `bit_test (variável , bit )`

Efetua um teste no *bit* indicado na *variável* indicada. Como podemos ter variáveis de 8, 16 ou 32 bits, o número do bit pode ir de 0~7 ou de 0~15 ou de 0~31.

Se o bit estiver em '1' a função retorna um *verdadeiro (true)* e caso contrário retorna *falso (false)*.

Para o caso de desejamos testar se está em '0' basta usarmos um `!` (NOT lógico) antes do teste.

Exemplo:

```
if ( bit_test(x,1) || ! bit_test(x,0) )           // Se o bit 1 de x for 1
{                                                  // OU ( || )
    .....                                       // o bit 0 de x for 0
}                                                  // executa o bloco.
```

4) `Output_High( pino ) -- Output_Low( pino ) -- Output_Toggle ( pino)`

Servem para setar , zerar e para inverter o estado de pinos de I/O.

5) `bit = input( pino )`

Lê o estado do pino indicado e escreve na variável indicada.

Exemplo: espera o pino RB0 ir de 0 para 1.

```
while ( ! input (pin_b0) ); // realiza bit test . É o teste Ideal.
```

ou


```
while ( pin_b0 == 0);           // realiza teste lógico em byte. Evitar !!
```

Observação: os nomes de pinos e outras constantes serão vistos ao final e estão no arquivo .H correspondente ao chip utilizado.

VI.2.3 - Funções de tempo

1) Delay_cycles (x)

Gera um delay equivalente a ' x ' *ciclos de instrução*.

```
Exemplo:    delay_cycles (1);           // equivale a 1 nop
            delay_cycles (4);           // equivale a 4 nop's
```

2) Delay_Ms (x)

Gera um delay de ' x ' milissegundos.

O programa deve conter a diretiva "#use delay(...)" antes de se usar esta função.

Exemplo:

A seqüência abaixo gera um pulso no pino RB2 com 10 ms (aproximadamente).

```
output_high (pin_b2);           // RB2 = 1
delay_ms (10);                  // espera 10 ms
output_low (pin_b2);            // RB2 = 0
```

3) Delay_Us (x)

Gera um delay de ' x ' microssegundos.

O programa deve conter a diretiva "#use delay(...)" antes de se usar esta função.

VI.2.4 - Funções para interrupções

1) enable_interrupts (tipo)

Permite habilitar interrupções individualmente ou globalmente. Quando habilitando globalmente apenas o bit de mascaramento geral será afetado, e não as habilitações individuais.

Alguns *tipos* são os seguintes:

```
global (GIE),          int_ext (RB0),          int_timer0,
```

Exemplo: habilitando a interrupção do timer 0 e do A/D

```
enable_interrupts (int_timer0);
enable_interrupts (int_ad);
enable_interrupts (global);
```

2) disable_interrupts (tipo)

Funciona de forma semelhante ao "enable_interrupts ()", desabilitando individualmente ou globalmente as interrupções.

3) ext_int_edge (qual, modo)

A variável qual indica qual das 3 interrupções externas estamos ajustando. Os valores podem ser 0 (int0), 1 (int1) ou 2 (int2).

A variável modo indica se a interrupção externa indicada será reconhecida quando ocorrer a descida do sinal (modo = H_TO_L) ou na subida do sinal (modo = L_TO_H).

Exemplo: reconhece interrupção externa 1 na subida.

```
ext_int_edge ( 1, L_TO_H);
```

4) Clear_interrupt (Tipo)

Permite “zerar” o pedido de uma interrupção pendente. Útil para os casos em que não precisaremos mais atender a uma certa interrupção, mas a mesma já está a espera do atendimento.

VI.2.5 - Funções para o canal A/D**1) read_adc ()**

Lê o canal analógico do chip.

IMPORTANTE: Como podemos ajustar a resolução em 8 ou 10 bits, devemos fazer com que o resultado seja direcionado para a variável de tamanho adequado.

Antes de usar esta função devemos usar “setup_adc()”, “set_adc_channel()” e setup_adc_ports().

Exemplo:

```
Temperatura = read_adc( );           // lê o canal já selecionado e
                                     // escreve na variável
```

2) setup_adc (modo)

Ajusta o conversor A/D. Alguns dos modos possíveis são:

adc_off,	adc_clock_div_2,	adc_clock_div_8,
adc_clock_div_32,	adc_clock_div_64,	adc_clock_div_internal

Para saber qual o melhor ajuste, consultar o databook do modelo utilizado para ver os detalhes de 'tempo de conversão X clock'.

3) set_adc_channel (canal)

Especifica qual o canal que será lido pela função read_adc().

4) setup_adc_ports (ajuste) OU setup_port_A (ajuste)

Ajusta os pinos para serem entradas analógicas ou não. Seu ajuste depende de cada chip em particular, devendo o programador cuidar para não escolher um ajuste errado para um certo modelo (consultar o arquivo .H correspondente).

Veja alguns ajustes para o 18F458

NO_ANALOGS	Todos os pinos analógicos desligados
ALL_ANALOG	Todos os pinos analógicos ligados
AN0	Apenas o pino RA0 analógico

Exemplo: Lendo o canal 0:

```
setup_port_a( AN0 );
setup_adc( ADC_CLOCK_INTERNAL );
set_adc_channel( 0 );
value = Read_ADC( );
```

VI.2.6 - Funções para EEPROM de dados interna

1) **Read_Eeprom (endereço)**

Lê o dado presente na posição *endereço* da EEPROM interna.
O limite para o *endereço* depende do modelo (de 0 a 255 no 18F458).

Exemplo: Posição_0 = read_eeprom(0);

2) **Write_Eeprom (endereço , valor)**

Escreve no *endereço* da EEPROM interna o *valor* indicado.

Importante: Esta instrução leva em média 4 ms para ser finalizada.

(Lembre-se que para a CPU rodando a 10 MHz internamente, o Tcy será de 100 ns, o que implica que a escrita na EEPROM leva em média o equivalente a 40.000 Tcy)

VI.2.7 - Funções do Timer 0 (São similares para os demais timers)

1) **get_timer0 ()**

Lê o valor atual do registro TMR0.

Exemplos:

```
if ( get_timer0 ( ) == 0 ) { ..... } // executa o bloco se TMR0 = 0
```

```
Valor_do_timer = get_timer0 ( ); // copia TMR0 para a variável
```

>>>>> Funciona de forma idêntica para os timers 1, 2 e 3

2) **set_timer0 (valor)**

Ajusta o valor do registro TMR0.

Lembre-se que sempre que escrevemos no registro TMR0 a contagem fica suspensa por dois ciclos de máquina.

Exemplo: Se o registro TMR0 for maior que 100 zera o registro.

```
if ( get_timer0( ) > 100 )
    set_timer0( 0 );
```

3) setup_timer_0 (Modo | prescaler | Tamanho);

Ajusta o modo de funcionamento do timer 0, se é 8 ou 16 bits e a divisão do prescaler.

Modos para ajuste do timer 0:

rtcc_internal --> timer 0 clock interno
 rtcc_ext_h_to_l --> timer 0 clock externo, transição de 1 para 0
 rtcc_ext_l_to_h --> timer 0 clock externo, transição de 0 para 1

Tamanhos da contagem do timer 0 (se não indicar, será de 16 bits):

RTCC_8_BIT --> timer 0 de 8 bits, similar a família 16F

Constantes para ajustes do prescaler:

rtcc_div_1 --> Sem prescaler para o timer 0
 rtcc_div_2 --> Prescaler para o timer 0, dividindo por 2
 rtcc_div_4 --> Prescaler para o timer 0, dividindo por 4
 rtcc_div_8 --> Prescaler para o timer 0, dividindo por 8
 rtcc_div_16 --> Prescaler para o timer 0, dividindo por 16
 rtcc_div_32 --> Prescaler para o timer 0, dividindo por 32
 rtcc_div_64 --> Prescaler para o timer 0, dividindo por 64
 rtcc_div_128 --> Prescaler para o timer 0, dividindo por 128
 rtcc_div_256 --> Prescaler para o timer 0, dividindo por 256

Exemplo 1: Ajustando o timer 0 modo interno em 1:4, 8 bits

```
setup_timer_0(RTCC_INTERNAL | RTCC_8_BIT | rtcc_div_4);
```

Exemplo 2: Ajustando o timer 0 modo externo, na descida do sinal, prescaler em 1:16, contagem 16 bits

```
setup_timer_0(rtcc_ext_h_to_l rtcc_div_16);
```

VI.2.8 - Funções do canal de comunicação serial

1) getc(), getch() OU getchar() (*)

São a mesma função. Entra numa rotina para esperar a chegada de um caractere pelo pino RCV definido na diretiva #use RS232.

Exemplo: espera a chegada do caractere 'A' :

```
while ( getch( ) != 'A' );
```

2) putc(valor) OU putchar(valor) (*)

São a mesma função. Envia "valor" serialmente pelo pino XMIT definido na diretiva #use RS232.

Exemplo: Enviar a palavra 'OLA' pelo serial.

```
putchar ( 'O' );                   OU                   putchar ( "ola");
putchar ( 'L' );
putchar ( 'A' );
```

(*) **Observação:** Estas funções somente usarão o canal serial de hardware presente em alguns modelos se os pinos XMIT e RCV forem os pinos da USART interna. Caso contrário, serão usadas funções de software escritas para este fim. Neste caso devemos levar em conta que embora ocupem apenas uma linha de código em C, podem ocupar bastante memória e demorar vários milissegundos para serem executadas.

VI.2.9 - Funções de uso geral

1) **port_b_pullups (opção)**

Controla os pull-ups da porta B (apenas para os bits ajustados entrada).

Temos como *opção* TRUE, ligando os pull-ups (RBPU\ = 0) e FALSE, desligando os pull-ups (RBPU\ = 1).

2) **restart_wdt()**

Reseta o watch dog.

3) **Sleep()**

Coloca o processador no modo de economia de energia.

- 4) **set_tris_A (ajuste)** ----> É obrigatório o uso da diretiva #fast_io(A)
set_tris_B (ajuste) ----> É obrigatório o uso da diretiva #fast_io(B)
set_tris_C (ajuste) ----> É obrigatório o uso da diretiva #fast_io(C)
set_tris_D (ajuste) ----> É obrigatório o uso da diretiva #fast_io(D)
set_tris_E (ajuste) ----> É obrigatório o uso da diretiva #fast_io(E)

Ajusta o registro TRIS indicado. Deve ser usado sempre que as diretivas #fast_io (port), #byte ou #bit forem utilizadas, pois estas determinam nosso controle sobre os ports.

Exemplo: Ajustando bits do portd todos para saída

```
set_tris_d ( 0 );
```

Ajustando os bits 0, 1 e 4 do portA para saída

```
set_tris_A ( 0b11101100 );
```

5) **Output_X (valor de 8 bits);**

Escreve o valor de 8 bits nos pinos do port X indicado.

Diferente da função output_high() e output_low(), que escrevem em apenas 1 bit, esta função permite escrever em todos os bits do port indicado de uma só vez.

Aconselha-se usar estas funções em standard_io() somente se TODOS os pinos do port forem saída, pois em standard_io() os pinos serão ajustados para serem TODOS pinos de saída antes da real escrita no port.

6) **valor8 = Input_X ();**

Lê o valor dos pinos do port X indicado e armazena na variável valor8.

Aconselha-se usar estas funções em standard_io() somente se TODOS os pinos do port forem entrada, pois em standard_io() os pinos serão ajustados para serem TODOS pinos de entrada antes da real leitura no port.

VII - Os arquivos de definições dos PICs

Os arquivos de definições dos pics, mais conhecidos como HEADER FILES, são os arquivos com terminação .H

Estes arquivos servem para indicar ao compilador o nome das variáveis, dos fusíveis, das funções, entre outros.

Você deve sempre consultar o arquivo de definições do pic que estiver usando.

E nosso caso os arquivos relevantes são os seguintes:

- 16F877.H
- 16F877A.H
- 18F452.H
- 18F458.H

Se você usou o caminho padrão de instalação dos programas, estes arquivos estarão na pasta

C: \ arquivos de programas \ picc \ devices

VIII – Exemplos práticos dos principais periféricos estudados no cap V

VIII.1 – Introdução

Neste capítulo veremos vários exemplos com o pic 18F458. Cada exemplo será apresentado com uma breve introdução teórica sobre o mesmo, e para acompanhamento o leitor deve usar o Mplab abrindo o projeto (arquivo .MCP) na pasta correspondente ao exemplo.

Todos os programas exemplo estão prontos para serem usados, no link abaixo, e em cada exemplo estará indicado o local do mesmo, no formato

cap8\..... – <http://www.newtoncbraga.com.br/livros/cap8.zip>

onde cada capítulo tem sua pasta de arquivos (no exemplo acima, capítulo 9), e cada exemplo esta em uma pasta separada.

VIII.2 – Usando displays de cristal liquido como auxilio de aprendizado

Para permitir fácil visualização dos resultados de vários exemplos, usaremos como dispositivo de saída um display de cristal liquido com 2 linhas de 16 caracteres cada, bem usual no mercado, mais conhecido como LCD 16X2.

Veremos neste item seu principio básico de funcionamento e as rotinas usadas para ajustes e acesso ao display.

VIII.2.1 - O display LCD 16X2

Os displays LCD são divididos em 2 tipos: Displays inteligentes e displays básicos.

Nos displays básicos devemos gerar todos os sinais de controle para acesso a cada ponto ou segmento.

Já nos displays inteligentes, cada unidade possui um chip dedicado apenas para permitir o acesso ao mesmo. Neste caso basta que nosso programa acesse o controlador do display para escrever caracteres, mudar o cursor de lugar, apagar, etc...

Rotinas prontas para o LCD

Temos 1 arquivo extra neste programa: **Lcd_Lib_D.c**

Este arquivo devera ser anexado ao nosso programa imediatamente antes do **main()** com a diretiva **#include**, a saber:

```
#include <apostilaPICrev4\lib\lcd_lib_D.c>
```

IMPORTANTE: Estas rotinas estão escritas para acesso ao displays pelo port D, Bits 2 a 7

Não esqueça de fazer o ajuste

```
set_tris_D(0b00000011);
```

caso esteja usando o Fast_io(D);

Sobre as bibliotecas:

Damos o nome de “bibliotecas” (do inglês Library) para funções prontas e que geralmente são acrescentadas ao nosso programa, evitando que tenhamos de redigitar trechos de código já testados.

Em nosso caso, para todas as experiências que usarão o display LCD, vamos apenas acrescentar a biblioteca LCD_LIB_D.C e chamar as funções da mesma.

Funções escritas para o LCD

Temos as seguintes funções prontas para o LCD:

1) void LCD_INIT(void):

Função obrigatória antes de se usar qualquer outra. Inicializa o LCD para uso em 4 bits, cursor deslocando a direita.

2) void LCDEnviaByte(void):

Envia para o LCD um byte dividido em 2 pacotes de 4 bits. Antes devemos indicar se o byte é de dado ou de comando.

3) void LCDClear(void):

Limpa todos os caracteres do display.

4) void Cursor (int8 linha,coluna):

Posiciona o cursor para o próximo caractere na linha e coluna indicadas, onde LINHA pode ser 1 ou 2, e COLUNA de 1 à 16.

5) void ApagarLCD(void):

Desliga apenas o cristal, mantendo os caracteres na memória do LCD

6) void AcenderLCD(void):

Religa o cristal, reapresentando os caracteres na memória do LCD.

7) void CursorApagado(void):

Não mostra o cursor no display.

8) void CursorPiscando(void):

Mostra o cursor sempre piscando no display.

9) void EnviaOCaractere(int8 qual);

Envia o caractere indicado para a posição atual do cursor.

Devido as características especiais do compilador C usado, caso enviemos uma string de caracteres, o compilador gera automaticamente um loop onde enviará cada byte da string.

Por exemplo, se executarmos

```
EnviaOCaractere ( "Teste");
```

O compilador fará a seguinte sequência equivalente:

```
EnviaOCaractere ('T');
```

```
EnviaOCaractere ('e');
```


EnviaOCaractere ('s');
EnviaOCaractere ('t');
EnviaOCaractere ('e');

VIII.3 – Exemplos do Capítulo II

Nestes exemplos, na pasta **cap2** do arquivo **cap2.zip** no link <http://www.newtoncbraga.com.br/livros/cap2.zip>, temos os exemplos vistos na teoria da linguagem C prontos para serem simulados no Mplab.

São eles:

- Na pasta ...**\Ex1** temos o programa que calcula a soma dos números de 1 a 100.
- Na pasta ...**\Ex2** temos o programa que conta indefinidamente de 1 a 10. Sempre que chegar a 10 incrementa a variável X e recomeça.
- Na pasta ...**\Ex3** temos o programa que incrementa indefinidamente a variável chamada contador e escreve nos leds colocados no port D.
- Na pasta ...**\Programa Resumo Geral** temos o programa que mostra a estrutura de todos os itens vistos no capítulo II sobre a linguagem C.
- Na pasta ...**\abreviacoes** temos o programa de exemplo para o uso de ABREVIACÕES.
- Na pasta ...**\break** temos o programa de exemplo para o uso do BREAK.
- Na pasta ...**\continue** temos o programa de exemplo para o uso do CONTINUE.
- Na pasta ...**\estruturas** temos o programa de exemplo para o uso de ESTRUTURAS.
- Na pasta ...**\return** temos o programa de exemplo para o uso do RETURN.
- Na pasta ...**\typecastings** temos o programa de exemplo para o uso de TYPECASTINGS.
- Na pasta ...**\unions** temos o programa de exemplo para o uso de UNIONS.

VIII.4 – Exemplo dos Capítulo III

Na pasta **cap3\Programa Mínimo** do arquivo disponível para download (<http://www.newtoncbraga.com.br/livros/cap3>) temos a listagem do programa mínimo para você desenvolver seus estudos de simulação no Mplab.

VIII.5 – Exemplos práticos para treinamento

Temos a seguir uma lista de cada exemplo desenvolvido para estudo, com uma breve descrição do mesmo e a pasta onde está localizado.

Para facilitar o estudo, cada exemplo a seguir terá início em uma nova página.

Esquemas elétricos de cada exemplo

Para preferencias de hardware, consulte o esquema de cada experiência no link abaixo

<http://www.newtonbraga.com.br/livros/esquemas.zip>

onde temos os esquemas de cada experiência.

Consulte-os sempre que necessário.

Aconselhamos baixar o arquivo <http://www.newtonbraga.com.br/livros/cap8.zip> para baixar os exemplos dos itens abaixo.

VIII.5.1 – Usando saídas digitais

Objetivo:

Estudar as funções de acesso aos pinos como saída tanto bit a bit quanto byte a byte.

Localização dos arquivos:

O fonte e o projeto para esta experiência (Sdig.c) está na seguinte pasta:

cap8\01-Sdig do arquivo **cap8.zip**

Fundamentação Teórica

É função básica de qualquer microcontrolador acessar pinos individualmente ou até mesmo em uma porta inteira para controles de entrada ou saída.

Os pinos de Entrada e saída são muitas vezes descritos apenas como “pinos de E/S”, ou ainda pela sua denominação em inglês, “Input/Output”, ou I/O.

Geralmente os pinos de E/S tem primariamente a função de escrever ou ler sinais com nível de tensão TTL, isto é, nível 0 (de 0 até 0,8V) e nível 1 (de 2 ate 5V).

No caso dos microcontroladores pic e desta nossa experiência, vamos ver a função para escrever 0 ou 1 em um bit individualmente, e então veremos a função para escrever em um port inteiro.

Função **output_high (qualpino)**

Esta função faz com que o pino indicado em **qualpino** vá para nível lógico **1**.

Função **output_low (qualpino)**

Esta função faz com que o pino indicado em **qualpino** vá para nível lógico **0**.

Função **output_X (valorde8bits)**

Esta função escreve no port **X** o **valorde8bits** indicado.

Detalhes do programa exemplo:

Neste programa simples, apenas ficamos alterando o estado de alguns pinos e escrevendo valores no port D para visualização nos leds.

VIII.5.2 – Usando entradas e saídas digitais

Objetivo:

Usando os conceitos adquiridos na experiência 1, veremos como ler sinais digitais e para sinalização usar as funções de saída nos leds no port D.

Veremos as funções de acesso aos pinos de entrada tanto bit a bit quanto byte a byte.

Localização dos arquivos:

O fonte e o projeto para esta experiência (ESDig.c) esta na seguinte pasta:
cap8\02-ESDig do arquivo **cap8.zip**

Fundamentação Teórica

Chamamos de entrada digital aquela cujos sinais são, como já dissemos, de nível TTL.

No caso dos microcontroladores pic e desta nossa experiência, vamos ver a função para ler o estado de 1 bit individualmente, e então veremos a função para ler um port inteiro.

Função **input (qualpino)**

Esta função retorna o estado do pino indicado em **qualpino**.

Função **input_X ()**

Esta função retorna o valor presente nos pinos do port **X** indicado.

Detalhes do programa exemplo:

Neste programa, vamos realizar as seguintes funções:

- Se a tecla T7 estiver pressionada (nível 0), apagamos o led 1
- Se a tecla T4 estiver pressionada (nível 0), acendemos o led 1
- Se a tecla T1 estiver pressionada (nível 0), faremos portD = 0 (apagaremos todos os LEDs)
- Se a tecla T* estiver pressionada (nível 0), faremos portD = 255 (acendemos todos os LEDs)

VIII.5.3 – Usando o conversor de analógico para digital com resolução de 8 bits

Objetivo:

Estudar o funcionamento do conversor A/D configurado para 8 bits e escrever o valor convertido nos 8 leds do port D.

Localização dos arquivos:

O fonte e o projeto para esta experiência (Ad8Pd.c) esta na seguinte pasta:
cap8\03-Ad8pd do arquivo **cap8.zip**

Fundamentação Teórica

Chamamos de sinal analógico aquele cuja variação no tempo é contínua, e não discreta ou ponto a ponto. Podemos dizer que no mundo real todos os sinais são analógicos. Já no âmbito dos microprocessadores, os sinais são digitais, isto é, variam no tempo em “pacotes” ou em passos, chamados de pontos discretos.

No caso dos microcontroladores pic e desta nossa experiência, vamos ver como utilizar o conversor analógico-digital interno para ler uma tensão cujo valor vai de 0 a 5 V (este valor é gerado por um trimpot) e indicar o valor convertido para verificação no port D.

Como vamos usar o conversor com resolução de 8 bits (veremos em outro exemplo o conversor com resolução de 10 bits), teremos 256 variações possíveis para o sinal, de forma que para 5 V de fundo de escala teremos aproximadamente 20 mV por divisão:

$$2^8 = 256 \quad \text{e} \quad 5.0 \text{ V} / 256 \text{ divisões} = 19,53125 \text{ mV/divisão}$$

As 256 opções são contadas de 0 a 255, assim vemos que na conversão A/D o sinal não varia mais linearmente, isto é, indo do 0 ao 5V, vamos em “passos” de aproximadamente 20 mV.

Tensão Analógica (aproximada)	Valor Digital em decimal	Valor digital em binário
0 a 19 mV	0	00000000
20 a 39 mV	1	00000001
40 a 59 mV	2	00000010
60 a 79 mV	3	00000011
.	.	.
.	.	.
4940 a 4939 mV	253	11111101
4960 a 4979 mV	254	11111110
4980 a 5000 mV	255	11111111

Funções para acesso ao canal A/D:

Conforme a apostila de linguagem C para pics, devemos usar as seguintes funções para poder usar o conversor A/D:

1- Logo após o #include <..> devemos usar a diretiva

```
#device adc=8
```

para ajustar o compilador para 8 bits (quando for 10 bits usaremos =10)

2- Logo que entrarmos no main() devemos ajustar o canal A/D, inclusive para que possamos usar os demais pinos do port A como E/S digital.

```
setup_adc (adc_clock_div_32); // para ajustar a velocidade dce conversão.
setup_adc_ports(RA0_analog); // RA0 analógico, demais pinos dos ports
                             // A e E digitais.
setup_adc_channel (0);       // já indicamos que vamos ler o canal 0.
```

3- E sempre que desejarmos ler o valor analógico usaremos a função

```
variável_de_8_bits = read_adc ( );
```

que retorna o valor do canal analógico.

Detalhes do programa exemplo:

Este programa realiza a seguinte função:

1. Lê o valor do canal analógico e salva na variável ValorAD8
2. Copia esta variável no port D
3. Espera 50 ms (temos uma media de 20 leituras por segundo)

VIII.5.4 – Display LCD 16x2 com interface de 4 bits

Objetivo:

Estudar as funções para acesso ao display LCD com apenas 4 bits de via de dados.

Desta forma poderemos usar o display como interface de saída visual em vários exemplos.

Localização dos arquivos:

O fonte e o projeto para esta experiência (Lcd4.c) esta na seguinte pasta:
cap8\04-Lcd4 do arquivo **cap8.zip**

Fundamentação Teórica

Neste exemplo vamos apenas escrever uma variável, de 0 à 9, no display, usando as funções já vistas no item VIII.2.1

Detalhes do programa exemplo:

Este programa realiza a seguinte função:

1. Conta de 0 a 9 com intervalo de 1 segundo.
2. Escreve no display LCD

VIII.5.5 - Usando o conversor de analógico para digital com resolução de 10 bits e visualização dos resultados no LCD

Objetivo:

Nesta experiência vamos ler o canal analógico com resolução de 10 bits e escrever no LCD o valor de 10 bits em decimal e a tensão equivalente de 0 a 5V.

Localização dos arquivos:

O fonte e o projeto para esta experiência (AdLcd.c) esta na seguinte pasta:
cap8\05-AdLcd do arquivo **cap8.zip**

Fundamentação Teórica

Esta experiência combina a leitura do A/D com o acesso ao display LCD, conforme já visto nas experiências anteriores.

E como apresentamos no displays números maiores que 9 ?

Para que possamos mostrar no display números maiores que 9, como por exemplo o fundo de escala do A/D (1023) que é uma variável de 16 bits, devemos DECOMPOR a mesma em dígitos individuais, que serão armazenados em 4 bytes (milhar, centena, dezena e unidade) e posteriormente enviados ao LCD. Para tal, usaremos a função **Decompõe ()**, que nada tem de especial, apenas comandos da linguagem C. Estude esta função detalhadamente para assimilar o conceito da divisão e do resto da divisão de números inteiros.

Sobre o ajuste da resolução do A/D:

Observe na listagem do programa que usamos a diretiva `#device ACD=10` neste caso, para obtermos a resolução de 10 bits

Detalhes do programa exemplo:

Este programa realiza a seguinte função:

- Lê o canal 0 com resolução de 10 bits
- calcula a tensão equivalente de 0 à 5 Volts
- decompõe em 4 dígitos o valor do A/D
- decompõe em 2 dígitos o valor da tensão
- escreve no LCD o valor de 10 bits e a tensão equivalente.

VIII.5.6 – Usando a EEPROM de dados

Objetivo:

Vamos nesta experiência usar as funções de escrita e leitura na EEPROM de dados interna. Usaremos o display LCD para acompanhar o funcionamento do programa.

Localização dos arquivos:

O fonte e o projeto para esta experiência (eeprom.c) esta na seguinte pasta: **cap8\06-eeprom** do arquivo **cap8.zip**

Fundamentação Teórica

A EEPROM de dados tem a seguinte característica básica:

Permite a fácil escrita de valores e sua leitura, como uma memória RAM, combinado ao fato de não perder os valores com o desligamento da energia.

O único inconveniente está no tempo de escrita que pode chegar a ate 10 ms dependendo do chip.

Funções de acesso a eeprom

- `write_eeprom ( end8, valor8 );`
Escreve no endereço end8 (endereço de 8 bits) a variável valor8 (variável de 8 bits)
- `valor8 = read_eeprom ( end8 );`
Lê o endereço end8 (endereço de 8 bits) e devolve como resultado um valor de 8 bits.

Detalhes do programa exemplo:

Este programa realiza a seguinte função:

- No reset recupera do endereço 0 da EEPROM a variável 'valor8'
- Tecla T7 = decrementa valor8 (ate 0)
- Tecla T4 = incrementa valor8 (ate 255)
- Tecla T1 = salva valor8 na eeprom, endereço 0
- Tecla T* = lê valor8 da eeprom
- Decompõe a variável em 3 dígitos e mostra no display LCD (funções já estudadas).

VIII.5.7 – Usando o timer 0 em 8 bits com clock externo

Objetivo:

Vamos ajustar o timer 0 para usar sinal externo, com prescaler dividindo por 32 (a cada 32 pulsos externos o timer incrementa 1 vez) e ler o valor do mesmo para mostrar no display.

Vamos usar um oscilador externo (baseado no 555) para gerar pulsos no pino RA4/T0CKI.

Localização dos arquivos:

O fonte e o projeto para esta experiência (t0.c) esta na seguinte pasta:

cap8\07-T0 do arquivo **cap8.zip**

Fundamentação Teórica

Como já vimos na teoria, o timer 0 é um contador de 8 bits ou de 16 bits. Usaremos nesta experiência o timer em 8 bits, com sinal externo.

Nesta experiência o timer 0 será incrementado a cada 4 pulsos externos, cuja frequência pode ser alterada via trimpot do circuito baseado no 555. A cada pressionar da tecla S1 o seu valor será lido e mostrado no display LCD. Depois o valor do timer será zerado reiniciando o contador.

Funções de acesso ao timer0

- `set_timer0 ( valor8 );`

Escreve no registro de contagem do timer a variável valor8.

- `valor8 = get_timer0 ( );`

Lê o registro de contagem do timer 0 e devolve como resultado um valor de 8 bits

Detalhes do programa exemplo:

Este programa realiza a seguinte função:

- . Ao pressionar a tecla T* lê o valor do timer 0 (de 0 a 255)
- . Zera o timer 0
- . decompõe em 3 dígitos
- . escreve no lcd

VIII.5.8 – Usando o timer 1 (16 bits) com clock interno

Objetivo:

Gerar uma base de tempo de 10 ms fixa, que pode ser usada como base de tempo para varreduras, aquisição de dados, ...

Localização dos arquivos:

O fonte e o projeto para esta experiência (t1.c) esta na seguinte pasta:

cap8\08-T1 do arquivo **cap8.zip**

Fundamentação Teórica

Vamos ajustar o timer 1 para usar sinal interno, isto é, sua velocidade será fixa baseada no Tcy (ciclo de máquina) com prescaler em 1. Com clock de 10 MHz, teremos $T_{cy} = 0,4 \mu s / \text{ciclo}$.

A cada 10 ms o timer 1 estoura e pede uma interrupção.

A cada 50 interrupções (0,5 seg) inverte o estado do pino RD7, gerando um sinal visual com intervalo de 1 segundo entre piscadas.

Para obtermos 10 ms (10.000 us), o timer devera contar 25.000 Tcy
(25000 x 0,4 us = 10000 us).

Como o timer 1 é crescente, a cada interrupção vamos somar ao valor do Timer 1 um offset para que contando MAIS 25000, resulte 65.536 (que seria o 65.535 mais 1 para estourar).

O offset tem então o valor de 40.536, pois se o timer iniciar em 45.536 vai estourar (ira de 65.535 para 0) apos 25.000 pulsos.

Mais. A cada interrupção incrementar uma variável auxiliar para contar o tempo e gerar o sinal visual de 1 segundo

Funções de acesso ao timer1

- set_timer1 (valor16);

Escreve no registro de contagem do timer 1 a variável valor16 (variável de 16 bits)

- valor16 = get_timer1 ();

Lê o registro de contagem do timer 1 e devolve como resultado um valor de 16 bits

VIII.5.9 – Usando a interrupção externa INT0 no pino RB0

Objetivo:

Vamos ver como se implementa uma função de interrupção externa para sincronismo de um evento externo.

Localização dos arquivos:

O fonte e o projeto para esta experiência (intext.c) esta na seguinte pasta:
cap8\09-intext do arquivo **cap8.zip**

Fundamentação Teórica

Chamamos de interrupção externa qualquer evento que ocorra fora do chip e que sinalize ao processador a ocorrência deste evento sem que o mesmo tenha realizado uma varredura para ver o sinal.

Estaremos nesta experiência simulando um encoder incremental. Neste tipo de encoder dois sinais são gerados: O sinal de Sincronismo, indicando o movimento do mesmo, e um sinal de Referência, indicando a direção do movimento (esquerda/direita, sobe/desce, sentido horário/anti-horário,).

O sinal de Sincronismo esta sendo gerado por um 555 astável com frequência variável e a referencia pelo pressionar da tecla S1.

Detalhes do programa exemplo:

Este programa realiza a seguinte função:

- A cada interferência externa lê o estado da tecla T*.
 - Se T* estiver pressionada, decrementa o contador
 - Se T* não pressionada, incrementa o contador
- Apresenta no display o valor da simulação

VIII.5.10 – Enviando um byte pelo canal serial

Objetivo:

Vamos ver como se implementa a função que envia um byte pelo canal serial a 9600 bps.

Como auxílio para esta experiência utilizar no PC o programa **Recebe_1_byte.exe** e um cabo simples, descrito abaixo.

Localização dos arquivos:

O fonte, o projeto para esta experiência (txd.c), e o programa auxiliar estão na seguinte pasta:
cap8\10-txd do arquivo **cap8.zip**

Fundamentação Teórica

A cada 100 ms o valor do canal analógico será lido com resolução de 8 bits, mostrado nos leds do port D e depois enviado ao PC pelo canal serial usando a interface Rs-232 a 9600 bps.

Importante: Lembre-se de que os sinais do pic são TTL e não podem ser ligados diretamente ao PC. Veja esquema da experiência ao final da apostila.

Funções de acesso a transmissão serial

Para usar o canal serial devemos antes do main() incluir as diretivas:

```
#use delay(clock=1000000)
```

para indicar qual o clock utilizado e

```
#use rs232 (BAUD=9600, XMIT=PIN_C6, RCV=PIN_C7, BITS=8)
```

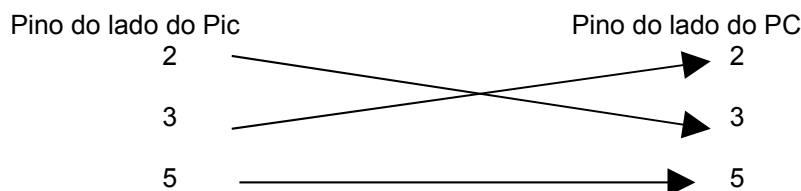
para que o compilador C já ajuste o canal serial interno do chip nas condições indicadas, e no programa devemos usar a função

```
putchar ( valor8 );
```

que escreve no registro de transmissão serial a variável valor8 (variável de 8 bits).

O cabo de comunicação:

O cabo para esta experiência é muito simples. São dois conectores DB9 fêmea interligados da seguinte forma:



Detalhes do programa exemplo:

Este programa realiza a seguinte função:

- A cada 100 ms lê o canal RA0 e envia para o programa do PC, que indicara o valor recebido via texto e via gráfico de barra

VIII.5.11 – Recebendo um byte pelo canal serial

Objetivo:

Vamos ver como se implementa a função que recebe um byte pelo canal serial a 9600 bps usando a interrupção de recepção do serial.

Como auxílio para esta experiência utilizar no PC o programa **Envia_1_byte.exe**

Localização dos arquivos:

O programa, o projeto para esta experiência (rxd.c), e o programa auxiliar estão na seguinte pasta: **cap8\11-rxd** do arquivo **cap8.zip**

Fundamentação Teórica

O programa no PC apresenta uma barra que pode ser ajustada pelo usuário com o mouse, e a posição (de 0 a 255) é indicada em um texto.

A cada ajuste realizado pelo usuário, o valor é enviado pelo serial e recebido pelo PIC, e este valor é indicado nos leds do portD em binário.

Funções de acesso a recepção serial

Para usar o canal serial devemos antes do main() incluir as diretivas:

```
#use delay(clock=10000000)
```

para indicar qual o clock utilizado e

```
#use rs232 (BAUD=9600, XMIT=PIN_C6, RCV=PIN_C7, BITS=8)
```

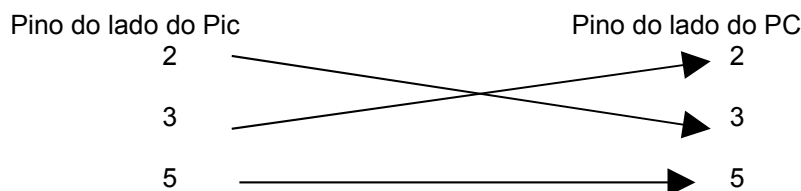
para que o compilador C já ajuste o canal serial interno do chip nas condições indicadas, e no programa devemos usar na interrupção **RDA** a função

```
x = getchar ( valor8 );
```

que lê o byte do buffer de recepção serial e escreve no registro X.

O cabo de comunicação:

O cabo para esta experiência é muito simples. São dois conectores DB9 fêmea interligados da seguinte forma:



Detalhes do programa exemplo:

Este programa realiza a seguinte função:

- A cada ajuste do usuário no programa do PC, o mesmo envia para a placa o byte indicado
- A placa recebe o byte e escreve nos leds do port D em binário.

VIII.5.12 – Comunicação serial SPI por hardware

Objetivo:

Estudar a comunicação serial padrão SPI com o auxílio de um potenciômetro digital da Microchip, modelo MCP 41010, usando o canal SPI interno.

Localização dos arquivos:

O fonte e o projeto para esta experiência (spi_hw.c) estão na seguinte pasta:
cap8\12-SPI_HW do arquivo **cap8.zip**

Fundamentação Teórica

Podemos ver os potenciômetros digitais como uma rede resistiva controlada por um hardware, e desta forma podemos selecionar o ajuste desta rede interna de forma a combinar estes resistores e obter um efeito similar ao de um trimpot. No caso do MCP41010, temos um trimpot de 10 Kohms, com 256 divisões possíveis.

Nosso programa envia pelo canal serial SPI 2 bytes, 1 de comando e outro indicando qual a posição desejamos para nosso trimpot. Esta variação não é obrigatoriamente linear como nos trimpots mecânicos ou como em outros modelos disponíveis no mercado. Podemos da posição 10 ir para a posição 100 diretamente e de forma muito rápida.

Podemos considerar que o trimpot digital é na verdade um conversor D/A integrado.

Valores ao ligar o chip:

Ao alimentar o trimpot, o mesmo ira automaticamente para a posição central (128), de forma que nosso software deve posicioná-lo na posição desejada, que pode ser fixa no programa ou ter sido recuperada da EEPROM.

Características da experiência:

Ao reset do pic, o trimpot será ajustado na posição 128. Na saída do mesmo, teremos então 2,5 V (ver esquema desta experiência).

Ao pressionar a tecla T*, vamos diminuir (ate 0) o valor do ajuste com delay de 50 ms entre ajustes.

Ao pressionar a tecla T7, vamos aumentar (ate 255) o valor do ajuste com delay de 50 ms entre ajustes.

O ajuste do trimpot poderá ser observado nos leds do port D (em binário) e com um voltímetro na saída do mesmo.

VIII.5.13 – Comunicação serial I²C

Objetivo:

Estudar a comunicação serial padrão I²C com o auxílio de uma memória EEPROM 24C64.

Localização dos arquivos:

O fonte e o projeto para esta experiência (i2c.c) estão na seguinte pasta:
cap8\13-I2C do arquivo **cap8.zip**

Fundamentação Teórica

Neste exemplo temos uma EEPROM serial 24C64 ligada ao canal MSSP do pic em modo I²C.

Como já vimos na teoria, as EEPROMs são úteis para o armazenamento de informações que não podem ser perdidas com o fim da alimentação.

A memória interna dos pics está hoje limitada a 256 bytes, o que em algumas aplicações não é o suficiente, como coletores de dados, por exemplo.

Neste caso, usamos uma EEPROM com mais capacidade, mas externa ao pic. A grande vantagem das EEPROMs I²C está no fato de utilizar apenas 2 pinos do pic, e quando usamos linguagem C, o software fica muito simples de ser implementado.

Só para comparação, enquanto o pic tem 256 bytes (0,25K), a memória 26C64 tem 8.192 bytes (1K). Temos ainda memórias com 128 KBytes (modelo 24LC1025)

A EEPROM que usaremos tem 3 pinos de endereçamento externo, podendo ir de 0 a 7, e endereço fixo 0xA0.

Assim, podemos ter até 8 EEPROMs em paralelo, nos endereços de 0xA0 até 0xAE (de 2 em 2).

Funções para uso da eeprom i2c:

- `init_ext_DEVICE()`
Deve ser usada antes de qualquer acesso a EEPROM
- `write_ext_device(a, d);`
Escreve no endereço A o dado D
- `read_ext_device(a);`
Lê no endereço A o dado armazenado

IMPORTANTE: Para usar este dispositivo o include a seguir é obrigatório.

```
#include <apostilaPICrev4\lib\I2C_LIB.c>
```

Neste arquivo estão as funções de acesso ao dispositivo.

Detalhes do programa exemplo:

Se pressionar T* decrementa a variável `valor8`, salva e em seguida recupera da EEPROM, mostrando no LCD os valores enviados e recebidos.

Se pressionar T7 incrementa a variável `valor8`, salva e em seguida recupera da EEPROM, mostrando no LCD os valores enviados e recebidos.

VIII.5.14 – CCP em modo “PWM”

Objetivo:

Estudar o funcionamento do módulo CCP em PWM

Localização dos arquivos:

O fonte e o projeto para esta experiência (`pwm.c`), e o programa auxiliar estão na seguinte pasta: **cap8\14-PWM** do arquivo **cap8.zip**

Fundamentação Teórica

Conforme visto na teoria, no PWM temos um sinal com frequência constante, mas com tempo em nível alto variável.

Possui varias aplicações, geralmente em controles de potência.

O controle da frequência é determinado pelo timer 2, e o período em alto pelo registro de duty-cycle.

Detalhes do programa exemplo:

Em nosso exemplo vamos acionar um LED, controlando o brilho dos mesmos.

A cada 'n' interrupções do timer 2, a função `set_pwm1_duty( )` mudará o ajuste do pwm, variando a potência emitida pelos LEDs.

- Se a tecla T7 (pino RB2) estiver pressionada (nível 0), aumenta o brilho no modo 2
- Se a tecla T4 (pino RB3) estiver pressionada (nível 0), diminui o brilho no modo 2
- Se a tecla T1 (pino RB4) estiver pressionada (nível 0), seleciona o modo 2 onde o brilho do LED e' controlado pelas teclas T7 e T4
- Se a tecla T* (pino RB5) estiver pressionada (nível 0), seleciona o modo 1, onde o brilho do LED vai do 0 ao 100% e retorna

VIII.5.15 – Multiplexação de displays de 7 segmentos

Objetivo:

Estudar a multiplexação de displays, isto é, ver como podemos ligar 4 displays em paralelo e acendê-los 1 de cada vez, dando a impressão de que todos estão acesos.

Localização dos arquivos:

O fonte e o projeto para esta experiência (ad7seg.c) estão na seguinte pasta:
cap8\15-ad7seg do arquivo **cap8.zip**

Fundamentação Teórica

O que é a multiplexação de displays ?

Consiste em ligarmos em paralelo os segmentos do mesmo e acionarmos um de cada vez, na ordem, desta forma:

- acende o display 1 e apaga os displays 2, 3 e 4
- espera um tempo
- acende o display 2 e apaga os displays 1, 3 e 4
- espera um tempo
- acende o display 3 e apaga os displays 1, 2 e 4
- espera um tempo
- acende o display 4 e apaga os displays 1, 2 e 3
- espera um tempo
- repete tudo

A taxa de repetição geralmente é controlada pela interrupção periódica de um timer.

Detalhes do programa exemplo:

Este programa realiza a seguinte função:

No reset ou após dar um toque na tecla T7, estamos no modo 0 (voltímetro) , onde indicamos nos displays o valor do A/D equivalente de 0 à 5.115 V

Se a tecla T4 for pressionada, funcionara no modo A/D puro, mostrando o valor de 0 a 1023 para escala de 0 a 5V.

Se mantivermos pressionada a tecla T1, a taxa de varredura será reduzida até o limite de percepção visual, onde teremos a impressão de que os displays estão piscando.

Se mantivermos pressionada a tecla T*, a taxa de varredura será bem lenta, e poderemos ver com facilidade o processo de “troca” do display aceso.

Então entra num ciclo repetitivo, a saber:

- Lê o canal analógico e calcula a tensão equivalente se for o caso
- Repete a cada 100 ms

Rotina de interrupção periódica do timer 0:

- Decompõe o valor lido / calculado
- Apaga todos os displays
- Determina qual será aceso
- Ajusta os segmentos do display selecionado
- Acende o display selecionado
- Volta

VIII.5.16 – Teclado matricial 3x4

Objetivo:

Ver como se realiza a multiplexação de teclas com um teclado de 12 teclas dispostas em uma matriz de 3 colunas e 4 linhas.

Localização dos arquivos:

O fonte e o projeto para esta experiência (tec3x4.c), bem como o programa auxiliar, estão na seguinte pasta:

cap8\16-tec3x4 do arquivo **cap8.zip**

Fundamentação Teórica

Na varredura de teclado matricial, o que fazemos é habilitar uma coluna de cada vez, e para cada coluna habilitada, ver se há alguma tecla pressionada (ver esquema desta experiência).

Detalhes do programa exemplo:

Este programa realiza a seguinte função:

Caso a sequência digitada seja 1 2 3 4 5 6 7 8, apenas os 3 últimos dígitos serão mostrados no display e armazenados na memória (apenas as 3 últimas teclas numéricas):

000, 001, 012, 123, 234, 345, 456, 567, 678.

A tecla ‘ * ’ zera os dígitos no display e a tecla ‘ # ’ enviará pelo serial o valor da memória.

O programa no PC receberá estes números e mostrará qual o valor final digitado.

VIII.5.17 – Comunicação serial SPI por software

Objetivo:

Estudar a comunicação serial padrão SPI com o auxílio de um potenciômetro digital da Microchip, modelo MCP 41010, usando funções de software para gerar o padrão SPI.

Localização dos arquivos:

O fonte e o projeto para esta experiência (spi_sw.c) estão na seguinte pasta:

cap8\17-SPI_SW – do arquivo **cap8.zip**

Fundamentação Teórica

Vale a mesma para o item IX.5.12

Características da experiência:

Nesta experiência vamos enviando valores seqüências de 0 a 255, e depois reiniciando do 0, para o potenciômetro, gerando assim uma forma de onda “serrote” na saída do mesmo.

Anexos

Instalação dos programas:

- 1) Baixe e instale o Mplab versão 8.XX (não instale o Mplab-X) em http://www.microchip.com/stellent/idcplg?IdcService=SS_GET_PAGE&nodeId=1406&dDocName=en019469&part=SW007002
- 2) No site www.ccsinfo.com baixe o demo do compilador C em <http://www.ccsinfo.com/ccsfreedemo.php>
- 3) No site www.ccsinfo.com baixe o plugin para o Mplab em http://www.ccsinfo.com/downloads/setup_mplab_plugin.exe

Instale nesta ordem.

- Esquemas das Experiências: <http://www.newtoncbraga.com.br/livros/esquemas.zip>
- Exemplos do capítulo 2 : <http://www.newtoncbraga.com.br/livros/cap2.zip>
- Exemplos do capítulo 3 : <http://www.newtoncbraga.com.br/livros/cap3.zip>
- Exemplos do capítulo 8 : <http://www.newtoncbraga.com.br/livros/cap4.zip>

Curso Rápido e Completo - Teórico e Prático

Temos ainda:

- **Placa didática e de desenvolvimento PICLAB 5**
Permite realizar TODAS as experiências da apostila, além de inúmeras outras, pois pode ser ampliada via conector de expansão
- **Gravador USB**
Grava in-circuit e ainda executa os programas diretamente na placa Piclab 5 ou na sua própria placa final

Para maiores informações sobre nossos produtos, visite o site do autor:

www.vidal.com.br